

ĐẠI HỌC ĐÀ NẴNG

TRƯỜNG ĐẠI HỌC BÁCH KHOA

KHOA CÔNG NGHỆ THÔNG TIN

ĐỒ ÁN TỐT NGHIỆP

NGÀNH: CÔNG NGHỆ THÔNG TIN

CHUYÊN NGÀNH: KHOA HỌC DỮ LIỆU VÀ TRÍ TUỆ
NHÂN TẠO

ĐỀ TÀI:

NGHIÊN CỨU PHÁT TRIỂN HỆ THỐNG
CHĂM BÀI TRỰC TUYẾN THÔNG MINH
TÍCH HỢP AI ĐỂ PHÂN TÍCH MÃ NGUỒN,
SINH TEST CASE VÀ HỖ TRỢ ĐÁNH GIÁ
NĂNG LỰC NGƯỜI DÙNG.

Người hướng dẫn: TS. PHẠM MINH TUẤN

Sinh viên thực hiện: NGUYỄN ĐỨC NHÃ

Số thẻ sinh viên: 102220031

Lớp: 22T_KHDL

Đà Nẵng, 06/2026

ĐẠI HỌC ĐÀ NẴNG
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA CÔNG NGHỆ THÔNG TIN

ĐỒ ÁN TỐT NGHIỆP
NGÀNH: CÔNG NGHỆ THÔNG TIN
CHUYÊN NGÀNH: KHOA HỌC DỮ LIỆU VÀ TRÍ TUỆ
NHÂN TẠO

ĐỀ TÀI:
NGHIÊN CỨU PHÁT TRIỂN HỆ THỐNG
CHẤM BÀI TRỰC TUYẾN THÔNG MINH
TÍCH HỢP AI ĐỂ PHÂN TÍCH MÃ NGUỒN,
SINH TEST CASE VÀ HỖ TRỢ ĐÁNH GIÁ
NĂNG LỰC NGƯỜI DÙNG.

Người hướng dẫn: TS. PHẠM MINH TUẤN

Sinh viên thực hiện: NGUYỄN ĐỨC NHÃ

Số thẻ sinh viên: 102220031

Lớp: 22T_KHDL

Đà Nẵng, 06/2026

This image shows a full page of white paper with horizontal dotted lines. The lines are evenly spaced and run across the width of the page, providing a guide for handwriting practice. There are no margins, text, or other markings on the page.

[illegible]

TÓM TẮT

Tên đề tài: Nghiên cứu phát triển hệ thống chấm bài trực tuyến thông minh tích hợp AI để phân tích mã nguồn, sinh test case và hỗ trợ đánh giá năng lực người dùng

Sinh viên thực hiện: Nguyễn Đức Nhã

Số thẻ SV: 102220031

Lớp: 22T_KHDL

Trong quá trình học tập và rèn luyện lập trình, việc đánh giá lời giải một cách tự động, chính xác và nhanh chóng đóng vai trò quan trọng đối với sinh viên ngành Công nghệ thông tin. Các hệ thống chấm bài trực tuyến giúp người học luyện tập thuật toán, nộp mã nguồn, nhận kết quả chấm và cải thiện kỹ năng lập trình thông qua quá trình thực hành thường xuyên. Tuy nhiên, nhiều hệ thống Online Judge truyền thống chủ yếu trả về kết quả đúng hoặc sai, chưa hỗ trợ nhiều trong việc phân tích mã nguồn, gợi ý cải thiện lời giải, sinh dữ liệu kiểm thử và đánh giá năng lực người dùng một cách toàn diện.

Đề tài “Nghiên cứu phát triển hệ thống chấm bài trực tuyến thông minh tích hợp AI để phân tích mã nguồn, sinh test case và hỗ trợ đánh giá năng lực người dùng” tập trung nghiên cứu, xây dựng và phát triển hệ thống BKDNOJ – BKDN Online Judge. Hệ thống cung cấp các chức năng chính của một nền tảng chấm bài lập trình trực tuyến như quản lý bài tập, quản lý người dùng, nộp bài, chạy thử chương trình, chấm bài tự động, hiển thị kết quả, quản lý bộ dữ liệu kiểm thử và hỗ trợ tổ chức các cuộc thi lập trình.

Điểm nổi bật của đề tài là việc mở rộng hệ thống theo hướng thông minh, kết hợp giữa cơ chế chấm bài tự động truyền thống với các chức năng hỗ trợ hiện đại. Hệ thống tích hợp AI nhằm hỗ trợ phân tích mã nguồn, đưa ra nhận xét về lời giải, phát hiện một số vấn đề trong cách cài đặt và gợi ý hướng cải thiện phù hợp. Bên cạnh đó, hệ thống còn hỗ trợ sinh test case cho bài toán, giúp người ra đề có thêm công cụ trong quá trình xây dựng và kiểm tra chất lượng bộ dữ liệu kiểm thử.

Ngoài các chức năng AI, BKDNOJ còn được phát triển thêm IDE trực tuyến, cho phép người dùng viết mã, chạy thử chương trình với dữ liệu mẫu hoặc dữ liệu tùy chỉnh ngay trong giao diện làm bài. Luồng chạy thử được thiết kế tách biệt với luồng nộp bài chính thức, giúp người dùng kiểm tra chương trình thuận tiện hơn mà không ảnh hưởng đến kết quả chấm chính thức hay bảng xếp hạng. Hệ thống cũng bổ sung cơ chế khóa tập trung

trong contest nhằm hạn chế gian lận khi thi trực tuyến, đặc biệt trong bối cảnh các công cụ AI ngày càng phổ biến.

Về mặt kỹ thuật, BKDNOJ được xây dựng theo kiến trúc nhiều thành phần, bao gồm ứng dụng web, cơ sở dữ liệu, bộ nhớ đệm, tiến trình xử lý nền, WebSocket, Bridge, Nginx và Judge Server. Hệ thống sử dụng Docker và Docker Compose để hỗ trợ triển khai, vận hành và quản lý các dịch vụ. Quy trình chấm bài được thực hiện tự động từ khi người dùng nộp mã nguồn, hệ thống chuyển bài nộp đến máy chấm, biên dịch và chạy chương trình với các bộ dữ liệu kiểm thử, sau đó trả kết quả về giao diện người dùng theo thời gian thực.

Kết quả của đề tài là hệ thống BKDNOJ có khả năng đáp ứng các chức năng chính của một hệ thống chấm bài trực tuyến, đồng thời được mở rộng theo hướng thông minh thông qua việc tích hợp AI, IDE trực tuyến và cơ chế hỗ trợ kiểm soát gian lận trong contest. Hệ thống có ý nghĩa thực tiễn trong việc hỗ trợ học tập, giảng dạy, luyện tập lập trình và tổ chức các kỳ thi lập trình trong môi trường giáo dục.

NHIỆM VỤ ĐỒ ÁN TỐT NGHIỆP

Họ tên sinh viên: Nguyễn Đức Nhã

Số thẻ sinh viên: 102220031

Lớp: 22T_KHDL

Khoa: Công nghệ thông tin

Ngành: Khoa học dữ liệu và Trí tuệ nhân tạo

- Tên đề tài đồ án:* Nghiên cứu phát triển hệ thống chấm bài trực tuyến thông minh tích hợp AI để phân tích mã nguồn, sinh test case và hỗ trợ đánh giá năng lực người dùng.
- Đề tài thuộc diện:* ☐ Có ký kết thỏa thuận sở hữu trí tuệ đối với kết quả thực hiện
- Các số liệu và dữ liệu ban đầu:*
Không có số liệu và dữ liệu ban đầu
- Nội dung các phần thuyết minh và tính toán:*

Chương 1. Giới thiệu đề tài: Trình bày bối cảnh thực hiện, lý do chọn đề tài, mục tiêu, nhiệm vụ, đối tượng, phạm vi và phương pháp nghiên cứu của đồ án.

Chương 2. Cơ sở lý thuyết: Trình bày các kiến thức nền tảng liên quan đến hệ thống Online Judge, quy trình chấm bài tự động, kiến trúc web nhiều thành phần, cơ sở dữ liệu, Docker, WebSocket, máy chấm và ứng dụng AI trong hỗ trợ lập trình.

Chương 3. Phân tích và thiết kế hệ thống: Trình bày yêu cầu hệ thống, các nhóm người dùng, sơ đồ Use Case, kiến trúc tổng thể, thiết kế cơ sở dữ liệu và các luồng xử lý chính như nộp bài, chạy thử, chấm điểm, chức năng AI và khóa tập trung trong kỳ thi.

Chương 4. Xây dựng và triển khai hệ thống: Trình bày quá trình xây dựng các chức năng chính của BKDNOJ, bao gồm chức năng nền tảng, IDE trực tuyến, luồng chạy thử, các chức năng AI, cơ chế khóa tập trung và quá trình triển khai hệ thống bằng Docker.

Chương 5. Thực nghiệm và đánh giá: Trình bày mục tiêu thực nghiệm, môi trường kiểm thử, các kịch bản kiểm thử, kết quả đạt được, đánh giá ưu điểm, hạn chế và mức độ đáp ứng của hệ thống so với mục tiêu đề ra.

Kết luận và hướng phát triển: Tổng kết các kết quả đạt được, nêu những hạn chế còn tồn tại và đề xuất hướng phát triển tiếp theo cho hệ thống BKDNOJ.

5. *Các bản vẽ, đồ thị (ghi rõ các loại và kích thước bản vẽ):*

Không có bản vẽ, đồ thị

6. *Họ tên người hướng dẫn:* TS. Phạm Minh Tuấn

7. *Ngày giao nhiệm vụ đồ án:* / / 202...

8. *Ngày hoàn thành đồ án:* / / 202...

Đà Nẵng, ngày tháng năm 2026

Trưởng Bộ môn

Người hướng dẫn

Phạm Minh Tuấn

LỜI NÓI ĐẦU

Đồ án tốt nghiệp với đề tài này là kết quả của quá trình học tập, nghiên cứu và rèn luyện của tôi trong những năm học tại Khoa Công nghệ Thông tin, Trường Đại học Bách khoa – Đại học Đà Nẵng. Trong suốt quá trình đó, tôi đã có cơ hội tiếp thu các kiến thức nền tảng và chuyên sâu về công nghệ thông tin, đồng thời vận dụng những kiến thức đã học vào việc phân tích, thiết kế, xây dựng và hoàn thiện một hệ thống phần mềm có tính ứng dụng thực tế.

Trong bối cảnh việc học tập và rèn luyện lập trình ngày càng trở nên quan trọng, các hệ thống chấm bài trực tuyến đóng vai trò thiết yếu trong việc hỗ trợ sinh viên nâng cao tư duy thuật toán, kỹ năng giải quyết vấn đề và năng lực lập trình. Xuất phát từ nhu cầu đó, đề tài BKDNOJ – BKDN Online Judge được thực hiện với mục tiêu xây dựng một nền tảng chấm bài lập trình trực tuyến, hỗ trợ người dùng luyện tập giải bài, nộp bài, chạy thử chương trình, xem kết quả chấm, tham gia các kỳ thi lập trình, đồng thời hỗ trợ quản trị viên trong việc quản lý bài tập, bộ dữ liệu kiểm thử, cuộc thi và người dùng.

Trong quá trình thực hiện đồ án, tôi đã tìm hiểu về kiến trúc của hệ thống Online Judge, quy trình chấm bài tự động, quản lý bài tập, quản lý cuộc thi, xử lý bài nộp, cũng như các vấn đề liên quan đến cơ sở dữ liệu, giao diện người dùng, bảo mật và vận hành hệ thống. Đây không chỉ là cơ hội để tôi củng cố kiến thức chuyên môn đã học, mà còn giúp tôi rèn luyện kỹ năng phân tích yêu cầu, thiết kế hệ thống, lập trình, kiểm thử và triển khai một sản phẩm phần mềm hoàn chỉnh.

Để hoàn thành đồ án này, tôi đã nhận được sự quan tâm, giúp đỡ và đồng hành quý báu từ thầy cô, gia đình và bạn bè. Trước hết, tôi xin gửi lời cảm ơn sâu sắc đến TS. Phạm Minh Tuấn, giảng viên hướng dẫn trực tiếp của đồ án, người đã tận tình định hướng, góp ý và hỗ trợ tôi trong suốt quá trình thực hiện đề tài.

Tôi xin chân thành cảm ơn Ban Chủ nhiệm Khoa Công nghệ Thông tin cùng toàn thể quý thầy cô đã giảng dạy, truyền đạt kiến thức, tạo điều kiện thuận lợi và giúp tôi có được nền tảng chuyên môn vững chắc trong suốt thời gian học tập tại trường.

Mặc dù đã cố gắng hoàn thiện hệ thống và nội dung báo cáo, do giới hạn về thời gian, kinh nghiệm thực tế và phạm vi của đồ án, báo cáo không tránh khỏi những thiếu sót. Tôi rất mong nhận được sự góp ý từ quý thầy cô để đề tài được hoàn thiện hơn trong tương lai.

CAM ĐOAN

Tôi xin cam đoan:

1. Toàn bộ nội dung được trình bày trong đồ án tốt nghiệp này là kết quả nghiên cứu, tìm hiểu và thực hiện của cá nhân tôi dưới sự hướng dẫn trực tiếp của TS. Phạm Minh Tuấn
2. Các tài liệu, công trình nghiên cứu, số liệu và nguồn tham khảo được sử dụng trong đồ án đều được trích dẫn đầy đủ, rõ ràng và tuân thủ các quy định về đạo đức học thuật
3. Đồ án không sao chép trái phép nội dung từ bất kỳ công trình nào khác. Các nội dung kế thừa, tham khảo hoặc trích dẫn đều được ghi rõ nguồn gốc theo đúng quy định.
4. Nếu có bất kỳ sai phạm nào liên quan đến bản quyền, trích dẫn hoặc tính trung thực học thuật, tôi xin hoàn toàn chịu trách nhiệm trước Hội đồng và các quy định của Nhà trường.

Sinh viên thực hiện

Nguyễn Đức Nhã

MỤC LỤC

LỜI NÓI ĐẦU.....	i
CAM ĐOAN.....	ii
MỤC LỤC	iii
DANH MỤC BẢNG BIỂU	viii
DANH MỤC HÌNH ẢNH.....	ix
DANH SÁCH CHỮ VIẾT TẮT.....	xi
MỞ ĐẦU	1
CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI.....	3
1.1. Giới thiệu đề tài.....	3
1.1.1 Bối cảnh thực hiện đề tài	3
1.1.2 Lý do chọn đề tài	3
1.2. Mục tiêu nghiên cứu và nhiệm vụ đề tài.....	4
1.2.1 Mục tiêu nghiên cứu.....	4
1.2.2 Nhiệm vụ đề tài	4
1.3. Đối tượng và phạm vi	5
1.3.1 Đối tượng.....	5
1.3.2 Phạm vi.....	5
1.4. Phương pháp nghiên cứu	6
1.4.1 Phương pháp nghiên cứu tài liệu.....	6
1.4.2 Phương pháp phân tích và thiết kế hệ thống.....	6
1.4.3 Phương pháp xây dựng và triển khai hệ thống.....	6
1.4.4 Phương pháp kiểm thử và đánh giá.....	6
1.5. Ý nghĩa.....	7
1.5.1 Ý nghĩa khoa học	7

1.5.2 Ý nghĩa thực tiễn.....	7
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT.....	8
2.1. Tổng quan về hệ thống chấm bài trực tuyến.....	8
2.1.1 Khái niệm hệ thống Online Judge	8
2.1.2 Vai trò của Online Judge trong học tập và thi lập trình.....	8
2.1.3 Quy trình hoạt động cơ bản của hệ thống chấm bài trực tuyến.....	8
2.2 Quy trình chấm bài tự động.....	9
2.2.1 Quy trình nộp bài và xử lý bài nộp.....	9
2.2.2 Biên dịch và thực thi mã nguồn.....	10
2.2.3 So sánh kết quả và xác định trạng thái chấm.....	10
2.2.4 Các kết quả chấm.....	11
2.3 Kiến trúc và công nghệ nền tảng của hệ thống	12
2.3.1 Kiến trúc web nhiều thành phần	12
2.3.2 Cơ sở dữ liệu, bộ nhớ đệm và tác vụ nền	12
2.3.3 WebSocket và cập nhật kết quả theo thời gian thực	13
2.3.4 Docker và triển khai hệ thống nhiều dịch vụ	13
2.4 Môi trường chạy thử và chấm bài.....	14
2.4.1 Máy chấm và môi trường thực thi mã nguồn.....	14
2.4.2 Test case trong đánh giá lời giải	15
2.4.3 IDE trực tuyến và luồng chạy thử chương trình.....	16
2.4.4 Vấn đề an toàn khi thực thi mã nguồn người dùng.....	17
2.5 Trí tuệ nhân tạo trong hỗ trợ lập trình	18
2.5.1 Khái quát về AI và mô hình ngôn ngữ lớn.....	18
2.5.2 AI trong phân tích mã nguồn.....	18
2.5.3 AI trong hỗ trợ tạo đề bài.....	19
2.5.4 AI trong sinh test case	19

2.5.5 AI trong hỗ trợ đánh giá năng lực người dùng.....	20
2.5.6 AI trong dự đoán dạng đề dựa trên bài nộp đúng.....	20
2.6 Cơ chế hỗ trợ hạn chế gian lận trong kỳ thi trực tuyến	21
2.6.1. Vấn đề gian lận trong thi lập trình trực tuyến.....	21
2.6.2. Cơ chế khóa tập trung trong kỳ thi.....	21
2.6.3. Giới hạn của cơ chế khóa tập trung trên nền web.....	22
2.7 Tổng kết chương.....	22
CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG	23
3.1 Tổng quan hệ thống BKDNOJ.....	23
3.1.1 Mục tiêu thiết kế hệ thống.....	23
3.1.2 Các nhóm người dùng của hệ thống.....	23
3.2 Phân tích yêu cầu hệ thống.....	24
3.2.1 Yêu cầu chức năng.....	25
3.2.2 Yêu cầu phi chức năng	26
3.3 Sơ đồ Use Case của hệ thống.....	27
3.3.1 Sơ đồ Use Case tổng quát.....	28
3.3.2 Đặc tả một số Use Case chính	29
3.4. Thiết kế kiến trúc tổng thể hệ thống.....	33
3.4.1. Kiến trúc tổng thể.....	33
3.4.2. Vai trò của các thành phần trong hệ thống	35
3.5. Thiết kế cơ sở dữ liệu.....	36
3.5.1. Sơ đồ cơ sở dữ liệu tổng quát	37
3.5.2. Các nhóm dữ liệu chính.....	38
3.6. Thiết kế luồng nộp bài và chấm bài.....	39
3.6.1. Luồng nộp bài chính thức	40
3.6.2. Các trạng thái của bài nộp.....	42

3.6.3. Thiết kế chế độ chấm điểm.....	43
3.7. Thiết kế IDE trực tuyến và luồng chạy thử	47
3.7.1. Mục tiêu của IDE trực tuyến	47
3.7.2. Luồng chạy thử chương trình	48
3.7.3. Phân biệt chạy thử và nộp bài chính thức.....	50
3.8. Thiết kế các chức năng AI.....	51
3.8.1. Quản lý API key AI.....	52
3.8.2. AI phân tích mã nguồn	54
3.8.3. AI hỗ trợ tạo đề bài	55
3.8.4. AI sinh test case.....	57
3.8.5. AI gợi ý dạng bài	59
3.9. Thiết kế cơ chế khóa tập trung trong kỳ thi	61
3.9.1. Mục tiêu của cơ chế khóa tập trung.....	61
3.9.2. Luồng xử lý khi người dùng tham gia kỳ thi	62
3.10. Tổng kết chương.....	64
CHƯƠNG 4. XÂY DỰNG VÀ TRIỂN KHAI HỆ THỐNG.....	66
4.1 Môi trường và công nghệ sử dụng.....	66
4.2. Xây dựng các chức năng nền tảng.....	67
4.3. Xây dựng IDE trực tuyến và luồng chạy thử.....	72
4.4. Xây dựng các chức năng AI	74
4.5. Xây dựng cơ chế khóa tập trung trong kỳ thi	81
4.6. Triển khai hệ thống	83
4.7. Tổng kết chương.....	85
CHƯƠNG 5. THỰC NGHIỆM VÀ ĐÁNH GIÁ	86
5.1. Mục tiêu thực nghiệm.....	86
5.2. Thiết lập môi trường thực nghiệm	86

5.3. Kịch bản thực nghiệm.....	86
5.4. Kết quả thực nghiệm	87
5.5. Đánh giá hệ thống.....	88
KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	90
1. Kết quả đạt được.....	90
2. Hạn chế.....	91
3. Hướng phát triển.....	91
TÀI LIỆU THAM KHẢO	93

DANH MỤC BẢNG BIỂU

Bảng 2.2.4: Các trạng thái của kết quả chấm
Bảng 2.3.2: Thành phần các tác vụ nền trong hệ thống
Bảng 2.3.4: Các dịch vụ hệ thống
Bảng 2.4.2: Các loại test case
Bảng 3.1.2: Các nhóm người dùng và tác nhân của hệ thống
Bảng 3.2.1: Yêu cầu chức năng của hệ thống
Bảng 3.2.2: Yêu cầu phi chức năng của hệ thống
Bảng 3.3.2: Đặc tả một số Use Case chính
Bảng 3.4.2: Các thành phần trong hệ thống
Bảng 3.5.2: Các nhóm dữ liệu chính của BKDNOJ
Bảng 3.6.2: Trạng thái của bài nộp
Bảng 3.6.3.1: Các chế độ chấm điểm
Bảng 3.6.3.2: Bảng giá trị batch
Bảng 3.7.3: Phân biệt giữa chạy thử và nộp bài chính thức
Bảng 3.8.1: Quản lý API key
Bảng 3.8.4: Thành phần và vai trò trong AI sinh test case
Bảng 3.8.5: Dữ liệu đầu vào và mục đích của AI gợi ý dạng bài
Bảng 4.1: Môi trường và các công nghệ sử dụng
Bảng 4.6: Danh sách container đang chạy bằng Docker Compose
Bảng 5.3: Nhóm và nội dung kiểm thử
Bảng 5.4.1: Kết quả thực nghiệm với các chức năng chính
Bảng 5.4.2: Bảng kiểm thử tải đọc API

DANH MỤC HÌNH ẢNH

- Hình 2.1.1: Quy trình nộp bài và xử lý bài nộp
- Hình 2.4.3: Sơ đồ sử dụng IDE trực tuyến
- Hình 3.3.1: Sơ đồ Use Case tổng quát của hệ thống
- Hình 3.4.1: Kiến trúc tổng thể hệ thống BKDNOJ
- Hình 3.5.1: Sơ đồ cơ sở dữ liệu rút gọn của hệ thống BKDNOJ
- Hình 3.6.1: Sơ đồ tuần tự của luồng nộp bài chính thức
- Hình 3.7.2: Sơ đồ tuần tự của luồng chạy thử chương trình
- Hình 3.8: Sơ đồ các chức năng AI
- Hình 3.8.2: Sơ đồ tuần tự của AI phân tích mã nguồn
- Hình 3.8.3: Sơ đồ tuần tự AI hỗ trợ tạo đề bài
- Hình 3.8.4: Sơ đồ tuần tự của AI sinh test case
- Hình 3.8.5: Gợi ý dạng bài với AI
- Hình 3.9.2: Sơ đồ tuần tự cơ chế khóa tập trung
- Hình 4.2.1: Giao diện xem danh sách bài tập
- Hình 4.2.2: Giao diện xem lịch sử bài nộp
- Hình 4.2.3: Xem chi tiết kết quả bài nộp
- Hình 4.2.4: Giao diện quản lý bài tập
- Hình 4.2.5: Giao diện quản lý test case
- Hình 4.2.6: Giao diện thêm bài tập
- Hình 4.2.7: Giao diện quản lý kỳ thi
- Hình 4.3.1: Giao diện IDE trực tuyến
- Hình 4.3.2: Thao tác chạy thử chương trình với IDE trực tuyến
- Hình 4.4.1: Giao diện quản lý API Key
- Hình 4.4.2: Giao diện AI phân tích mã nguồn
- Hình 4.4.3: Giao diện AI hỗ trợ tạo đề bài

Hình 4.4.4: Giao diện AI sinh test case

Hình 4.4.5: Quá trình xử lý AI sinh test case

Hình 4.4.6: Giao diện AI dự đoán dạng đề

Hình 4.4.7: Tiến độ năng lực người dùng

Hình 4.5.1: Giao diện bắt đầu tham gia kỳ thi với khóa tập trung

Hình 4.5.2: Giao diện trước khi vào khóa tập trung

Hình 4.5.3: Giao diện bảng xếp hạng cho biết số lần vi phạm

Hình 4.6: Giao diện trạng thái của máy chấm

DANH SÁCH CHỮ VIẾT TẮT

Chữ viết tắt	Cụm từ đầy đủ	Ý nghĩa trong tài liệu
AI	Artificial Intelligence	Trí tuệ nhân tạo, được tích hợp để hỗ trợ phân tích mã nguồn, tạo đề bài, sinh test case và đánh giá năng lực người dùng.
API	Application Programming Interface	Giao diện lập trình ứng dụng, dùng để hệ thống kết nối với các dịch vụ AI bên ngoài.
API key	Application Programming Interface Key	Khóa truy cập dùng để xác thực khi gọi đến các dịch vụ AI.
BKDNOJ	BKDN Online Judge	Tên hệ thống chấm bài trực tuyến được xây dựng trong đề tài.
OJ	Online Judge	Hệ thống chấm bài lập trình trực tuyến.
DMOJ	Distributed Modern Online Judge	Nền tảng Online Judge mã nguồn mở được sử dụng làm nguồn tham khảo kỹ thuật.
VNOJ	VNOI Online Judge	Hệ thống Online Judge của VNOI, được sử dụng làm nguồn tham khảo kỹ thuật.
IDE	Integrated Development Environment	Môi trường phát triển tích hợp, trong tài liệu là IDE trực tuyến cho phép viết mã và chạy thử trên trình duyệt.
UI	User Interface	Giao diện người dùng.
UX	User Experience	Trải nghiệm người dùng khi thao tác với hệ thống.

DB	Database	Cơ sở dữ liệu dùng để lưu thông tin người dùng, bài tập, bài nộp, test case, kỳ thi và kết quả chấm.
SQL	Structured Query Language	Ngôn ngữ truy vấn dữ liệu trong hệ quản trị cơ sở dữ liệu quan hệ.
JSON	JavaScript Object Notation	Định dạng dữ liệu thường dùng khi trao đổi thông tin giữa hệ thống và dịch vụ bên ngoài.
HTML	HyperText Markup Language	Ngôn ngữ đánh dấu dùng để xây dựng giao diện web.
SVG	Scalable Vector Graphics	Định dạng ảnh vector, có thể xuất hiện trong tài nguyên giao diện hoặc hình minh họa.
PNG	Portable Network Graphics	Định dạng ảnh thường dùng cho hình minh họa hoặc ảnh giao diện.
JPG/JPEG	Joint Photographic Experts Group	Định dạng ảnh nén thường dùng cho ảnh đầu vào hoặc ảnh minh họa.
WebP	Web Picture Format	Định dạng ảnh hiện đại được hỗ trợ trong chức năng tạo đề bài từ ảnh.
AC	Accepted	Kết quả chấm cho biết chương trình đúng với các test case được chấm.
WA	Wrong Answer	Kết quả chấm cho biết chương trình chạy được nhưng cho kết quả sai.
TLE	Time Limit Exceeded	Kết quả chấm cho biết chương trình chạy vượt quá giới hạn thời gian.
MLE	Memory Limit Exceeded	Kết quả chấm cho biết chương trình sử dụng vượt quá giới hạn bộ nhớ.

RTE	Runtime Error	Kết quả chấm cho biết chương trình gặp lỗi trong quá trình thực thi.
CE	Compilation Error	Kết quả chấm cho biết mã nguồn không biên dịch thành công.
OLE	Output Limit Exceeded	Kết quả chấm cho biết chương trình sinh dữ liệu đầu ra vượt quá giới hạn cho phép.
IE	Internal Error	Lỗi nội bộ của hệ thống hoặc máy chấm trong quá trình xử lý bài nộp.
ICPC	International Collegiate Programming Contest	Mô hình thi lập trình thường dùng cách chấm đúng/sai toàn phần.
IOI	International Olympiad in Informatics	Mô hình thi lập trình thường có chấm điểm theo subtask hoặc test case.
CPU	Central Processing Unit	Bộ xử lý trung tâm, dùng trong phần theo dõi tài nguyên container hoặc máy chủ.
RAM	Random Access Memory	Bộ nhớ truy cập ngẫu nhiên, dùng trong đánh giá tài nguyên hệ thống.
I/O	Input/Output	Dữ liệu vào/ra của chương trình hoặc hệ thống.
VU	Virtual User	Người dùng ảo trong kiểm thử tải bằng công cụ k6.
MB	Megabyte	Đơn vị đo dung lượng bộ nhớ hoặc kích thước dữ liệu.
GB	Gigabyte	Đơn vị đo dung lượng bộ nhớ hoặc tài nguyên lưu trữ.

MỞ ĐẦU

Các hệ thống chấm bài trực tuyến đóng vai trò quan trọng trong việc hỗ trợ học tập, rèn luyện lập trình và tổ chức các kỳ thi thuật toán. Thông qua các hệ thống này, người học có thể nộp mã nguồn, nhận kết quả chấm tự động và cải thiện kỹ năng lập trình qua quá trình luyện tập thường xuyên. Tuy nhiên, nhiều hệ thống Online Judge truyền thống chủ yếu tập trung vào việc trả về kết quả chấm như đúng, sai hoặc lỗi biên dịch, chưa hỗ trợ sâu trong việc phân tích mã nguồn, sinh dữ liệu kiểm thử, chạy thử chương trình thuận tiện và đánh giá năng lực người dùng một cách toàn diện.

Trong bối cảnh trí tuệ nhân tạo ngày càng phát triển, việc tích hợp AI vào hệ thống chấm bài trực tuyến là một hướng tiếp cận phù hợp, có tính thực tiễn cao trong giáo dục lập trình. Bên cạnh đó, sự phổ biến của các công cụ AI cũng đặt ra yêu cầu mới đối với việc tổ chức thi lập trình trực tuyến, đặc biệt là vấn đề hạn chế gian lận và đảm bảo tính công bằng trong contest.

Đề tài “Nghiên cứu phát triển hệ thống chấm bài trực tuyến thông minh tích hợp AI để phân tích mã nguồn, sinh test case và hỗ trợ đánh giá năng lực người dùng” tập trung nghiên cứu và phát triển hệ thống BKDNOJ – BKDN Online Judge. Hệ thống cung cấp các chức năng chính của một nền tảng chấm bài trực tuyến như quản lý bài tập, quản lý người dùng, nộp bài, chấm bài tự động, quản lý test case, quản lý cuộc thi và hiển thị kết quả chấm theo thời gian thực.

Bên cạnh các chức năng cơ bản, BKDNOJ được mở rộng thêm các chức năng hỗ trợ hiện đại như IDE trực tuyến, luồng chạy thử chương trình độc lập, các chức năng AI hỗ trợ phân tích mã nguồn, sinh test case và cung cấp thêm thông tin phục vụ đánh giá năng lực lập trình của người dùng. Ngoài ra, hệ thống còn bổ sung cơ chế khóa tập trung trong contest nhằm hạn chế hành vi gian lận khi thi trực tuyến, giúp nâng cao tính nghiêm túc và công bằng trong quá trình tổ chức thi.

BKDNOJ được xây dựng trên nền tảng web với kiến trúc nhiều thành phần, bao gồm Django Site, cơ sở dữ liệu, Redis, Celery, WebSocket, Bridge, Nginx và Judge Server. Hệ thống được triển khai bằng Docker và Docker Compose nhằm hỗ trợ quá trình cài đặt, vận hành và quản lý các dịch vụ. Kết quả hướng tới là một hệ thống chấm bài trực tuyến có khả

năng vận hành ổn định, hỗ trợ học tập lập trình hiệu quả, đồng thời mở rộng theo hướng thông minh và phù hợp hơn với nhu cầu giảng dạy, luyện tập và tổ chức thi lập trình.

Đồ án tốt nghiệp được cấu trúc thành các phần chính như sau:

Chương 1. Giới thiệu đề tài: Trình bày bối cảnh thực hiện, lý do chọn đề tài, mục tiêu, nhiệm vụ, đối tượng, phạm vi và phương pháp nghiên cứu của đồ án.

Chương 2. Cơ sở lý thuyết: Trình bày các kiến thức nền tảng liên quan đến hệ thống Online Judge, quy trình chấm bài tự động, kiến trúc web nhiều thành phần, cơ sở dữ liệu, Docker, WebSocket, máy chấm và ứng dụng AI trong hỗ trợ lập trình.

Chương 3. Phân tích và thiết kế hệ thống: Trình bày yêu cầu hệ thống, các nhóm người dùng, sơ đồ Use Case, kiến trúc tổng thể, thiết kế cơ sở dữ liệu và các luồng xử lý chính như nộp bài, chạy thử, chấm điểm, chức năng AI và khóa tập trung trong kỳ thi.

Chương 4. Xây dựng và triển khai hệ thống: Trình bày quá trình xây dựng các chức năng chính của BKDNOJ, bao gồm chức năng nền tảng, IDE trực tuyến, luồng chạy thử, các chức năng AI, cơ chế khóa tập trung và quá trình triển khai hệ thống bằng Docker.

Chương 5. Thực nghiệm và đánh giá: Trình bày mục tiêu thực nghiệm, môi trường kiểm thử, các kịch bản kiểm thử, kết quả đạt được, đánh giá ưu điểm, hạn chế và mức độ đáp ứng của hệ thống so với mục tiêu đề ra.

Kết luận và hướng phát triển: Tổng kết các kết quả đạt được, nêu những hạn chế còn tồn tại và đề xuất hướng phát triển tiếp theo cho hệ thống BKDNOJ.

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI

1.1. Giới thiệu đề tài

1.1.1 Bối cảnh thực hiện đề tài

Trong những năm gần đây, các hệ thống chấm bài trực tuyến (Online Judge) đã trở thành công cụ quan trọng trong đào tạo và rèn luyện kỹ năng lập trình. Các hệ thống này giúp tự động hóa quá trình chấm bài, hỗ trợ người học luyện tập giải thuật và tạo môi trường tổ chức các cuộc thi lập trình một cách hiệu quả.

Tuy nhiên, phần lớn các hệ thống Online Judge hiện nay chủ yếu tập trung vào việc đánh giá kết quả đúng hoặc sai của bài làm, chưa hỗ trợ sâu trong việc phân tích mã nguồn, sinh dữ liệu kiểm thử hay đánh giá năng lực người dùng. Trong khi đó, sự phát triển mạnh mẽ của trí tuệ nhân tạo đã mở ra khả năng ứng dụng AI vào các hoạt động hỗ trợ học tập và đánh giá lập trình.

Xuất phát từ thực tế đó, đề tài “Nghiên cứu phát triển hệ thống chấm bài trực tuyến thông minh tích hợp AI để phân tích mã nguồn, sinh test case và hỗ trợ đánh giá năng lực người dùng” được thực hiện nhằm phát triển hệ thống BKDNOJ – BKDN Online Judge. Đề tài hướng đến việc xây dựng một hệ thống chấm bài trực tuyến hiện đại, kết hợp giữa cơ chế chấm bài tự động và các chức năng AI nhằm hỗ trợ người học, người ra đề và quản trị viên trong quá trình sử dụng hệ thống.

1.1.2 Lý do chọn đề tài

Trong quá trình học tập lập trình, sinh viên cần một môi trường để luyện tập, nộp bài và nhận kết quả đánh giá một cách nhanh chóng, khách quan. Các hệ thống chấm bài trực tuyến đã đáp ứng tốt nhu cầu này thông qua việc tự động biên dịch, chạy chương trình và trả về kết quả chấm. Tuy nhiên, đa số hệ thống hiện nay chủ yếu dừng lại ở việc xác định bài làm đúng hay sai, chưa hỗ trợ nhiều trong việc phân tích mã nguồn, sinh dữ liệu kiểm thử, gợi ý cải thiện lời giải hoặc đánh giá năng lực người dùng theo quá trình học tập.

Bên cạnh đó, trí tuệ nhân tạo đang được ứng dụng ngày càng nhiều trong giáo dục và phát triển phần mềm. Việc tích hợp AI vào hệ thống chấm bài trực tuyến có thể giúp mở rộng khả năng hỗ trợ người học và người ra đề, chẳng hạn như phân tích mã nguồn, sinh test case và cung cấp thêm thông tin phục vụ đánh giá năng lực lập trình. Ngoài ra, trong bối

cánh các công cụ AI ngày càng phổ biến, việc tổ chức thi lập trình trực tuyến cũng cần có thêm cơ chế hỗ trợ hạn chế gian lận và đảm bảo tính công bằng trong contest.

Vì vậy, tôi lựa chọn đề tài “Nghiên cứu phát triển hệ thống chấm bài trực tuyến thông minh tích hợp AI để phân tích mã nguồn, sinh test case và hỗ trợ đánh giá năng lực người dùng” nhằm nghiên cứu và phát triển hệ thống BKDNOJ – BKDN Online Judge theo hướng hiện đại hơn, vừa đáp ứng các chức năng cốt lõi của một hệ thống Online Judge, vừa bổ sung các tính năng có giá trị thực tiễn như AI hỗ trợ lập trình, IDE trực tuyến và cơ chế khóa tập trung trong contest.

1.2. Mục tiêu nghiên cứu và nhiệm vụ đề tài

1.2.1 Mục tiêu nghiên cứu

Mục tiêu nghiên cứu của đề tài là nghiên cứu, xây dựng và phát triển hệ thống **BKDNOJ – BKDN Online Judge** theo hướng một nền tảng chấm bài trực tuyến thông minh, có khả năng hỗ trợ quá trình học tập, rèn luyện lập trình và tổ chức các kỳ thi lập trình một cách hiệu quả.

Đề tài tập trung nghiên cứu mô hình hoạt động của hệ thống Online Judge, quy trình nộp bài, chấm bài tự động, quản lý bài tập, quản lý test case, quản lý người dùng và cuộc thi. Trên cơ sở đó, hệ thống được xây dựng và hoàn thiện nhằm đảm bảo các chức năng cốt lõi như tiếp nhận bài nộp, biên dịch và thực thi mã nguồn, trả kết quả chấm, cập nhật trạng thái bài nộp theo thời gian thực và hỗ trợ người dùng theo dõi quá trình luyện tập.

Bên cạnh các chức năng chấm bài truyền thống, đề tài còn hướng đến việc mở rộng hệ thống với các chức năng hiện đại như IDE trực tuyến, luồng chạy thử chương trình, tích hợp trí tuệ nhân tạo để phân tích mã nguồn, sinh test case và cung cấp thêm thông tin phục vụ đánh giá năng lực người dùng. Ngoài ra, hệ thống cũng nghiên cứu bổ sung cơ chế khóa tập trung trong contest nhằm hỗ trợ hạn chế gian lận và nâng cao tính công bằng trong quá trình tổ chức thi lập trình trực tuyến.

1.2.2 Nhiệm vụ đề tài

- Nghiên cứu mô hình hoạt động của hệ thống chấm bài trực tuyến, bao gồm quy trình nộp bài, biên dịch, chạy chương trình, chấm bài và trả kết quả cho người dùng.

- Phân tích các yêu cầu cần thiết của hệ thống BKDNOJ, xác định các nhóm người dùng, chức năng chính và các luồng xử lý quan trọng.
- Xây dựng và hoàn thiện các chức năng phục vụ học tập và thi lập trình như quản lý bài tập, quản lý test case, nộp bài, chấm bài tự động, chạy thử chương trình và tổ chức kỳ thi.
- Phát triển IDE trực tuyến và luồng chạy thử riêng, giúp người dùng kiểm tra chương trình ngay trong hệ thống mà không ảnh hưởng đến bài nộp chính thức.
- Tích hợp các chức năng AI nhằm hỗ trợ phân tích mã nguồn, sinh test case và cung cấp thêm thông tin phục vụ đánh giá năng lực lập trình của người dùng.
- Bổ sung cơ chế khóa tập trung trong kỳ thi nhằm hỗ trợ hạn chế gian lận và nâng cao tính công bằng khi tổ chức thi trực tuyến.
- Triển khai, kiểm thử và đánh giá hệ thống nhằm xác định mức độ đáp ứng của các chức năng so với mục tiêu đề tài.

1.3. Đối tượng và phạm vi

1.3.1 Đối tượng

- Đối tượng nghiên cứu của đề tài là hệ thống chấm bài lập trình trực tuyến **BKDNOJ – BKDN Online Judge**, bao gồm các thành phần phục vụ quá trình học tập, luyện tập và tổ chức thi lập trình. Đề tài tập trung vào mô hình hoạt động của hệ thống Online Judge, quy trình nộp bài, chấm bài tự động, quản lý bài tập, quản lý test case, quản lý người dùng, tổ chức kỳ thi và cập nhật kết quả theo thời gian thực.
- Bên cạnh đó, đề tài cũng nghiên cứu việc tích hợp các chức năng mở rộng vào hệ thống, bao gồm IDE trực tuyến, luồng chạy thử chương trình, các chức năng AI hỗ trợ phân tích mã nguồn, sinh test case, hỗ trợ đánh giá năng lực người dùng và cơ chế khóa tập trung trong kỳ thi nhằm hạn chế gian lận khi thi trực tuyến.

1.3.2 Phạm vi

- Phạm vi của đề tài tập trung vào việc nghiên cứu, xây dựng và phát triển hệ thống BKDNOJ trên nền tảng web. Hệ thống hướng đến các chức năng chính như quản lý

bài tập, nộp bài, chấm bài tự động, chạy thử chương trình, quản lý test case, tổ chức kỳ thi, quản lý người dùng và hiển thị kết quả chấm.

- Đề tài cũng mở rộng hệ thống với các chức năng hỗ trợ hiện đại như IDE trực tuyến, tích hợp AI để phân tích mã nguồn và sinh test case, quản lý API key, cơ chế khóa tập trung trong kỳ thi và hỗ trợ đánh giá năng lực người dùng. Trong phạm vi đồ án, hệ thống được triển khai và kiểm thử ở quy mô phục vụ học tập, luyện tập và tổ chức thi lập trình trong môi trường giáo dục; chưa tập trung vào triển khai ở quy mô rất lớn hoặc phát triển ứng dụng di động riêng.

1.4. Phương pháp nghiên cứu

1.4.1 Phương pháp nghiên cứu tài liệu

Đề tài tiến hành nghiên cứu tài liệu về hệ thống Online Judge, quy trình đánh giá lời giải, kiến trúc hệ thống web nhiều thành phần và các công nghệ sử dụng trong triển khai. Các nhóm tài liệu chính bao gồm nghiên cứu tổng quan về Online Judge, tài liệu DMOJ, tài liệu Django, Redis, Celery, WebSocket, Docker, Nginx, tài liệu API AI, tài liệu về cơ chế toàn màn hình/chuyển trạng thái trang web và công cụ kiểm thử tải k6.

1.4.2 Phương pháp phân tích và thiết kế hệ thống

Trên cơ sở các kiến thức đã nghiên cứu, đề tài tiến hành phân tích yêu cầu hệ thống, xác định các nhóm người dùng, chức năng chính và các luồng xử lý quan trọng. Từ đó, hệ thống BKDNOJ được thiết kế theo hướng đáp ứng nhu cầu học tập, luyện tập lập trình, tổ chức kỳ thi và mở rộng các chức năng hỗ trợ hiện đại.

1.4.3 Phương pháp xây dựng và triển khai hệ thống

Đề tài áp dụng phương pháp xây dựng hệ thống theo từng chức năng, bao gồm các chức năng chấm bài trực tuyến, IDE trực tuyến, luồng chạy thử chương trình, tích hợp AI hỗ trợ phân tích mã nguồn, sinh test case và cơ chế khóa tập trung trong kỳ thi. Hệ thống được triển khai bằng Docker và Docker Compose nhằm thuận tiện cho việc cài đặt, quản lý và vận hành các dịch vụ.

1.4.4 Phương pháp kiểm thử và đánh giá

Sau khi xây dựng các chức năng, đề tài tiến hành kiểm thử hệ thống thông qua các kịch bản sử dụng thực tế như nộp bài, chạy thử chương trình, chấm bài tự động, sử dụng chức năng AI và tham gia kỳ thi có bật cơ chế khóa tập trung. Kết quả kiểm thử được sử dụng để đánh giá mức độ đúng đắn, khả năng hoạt động ổn định và mức độ đáp ứng của hệ thống so với mục tiêu đề tài.

1.5. Ý nghĩa

1.5.1 Ý nghĩa khoa học

Đề tài góp phần nghiên cứu mô hình hoạt động của hệ thống chấm bài trực tuyến, bao gồm quy trình nộp bài, chấm bài tự động, tổ chức kỳ thi, quản lý test case và cập nhật kết quả theo thời gian thực. Thông qua việc xây dựng hệ thống BKDNOJ, đề tài làm rõ cách kết hợp giữa các thành phần trong một hệ thống Online Judge như ứng dụng web, cơ sở dữ liệu, WebSocket, tác vụ nền và máy chấm.

Bên cạnh đó, đề tài còn nghiên cứu việc tích hợp AI vào hệ thống chấm bài trực tuyến nhằm hỗ trợ phân tích mã nguồn, sinh test case và cung cấp thêm thông tin phục vụ đánh giá năng lực người dùng. Đây là hướng tiếp cận có ý nghĩa trong việc mở rộng khả năng hỗ trợ của các hệ thống học tập lập trình hiện nay.

1.5.2 Ý nghĩa thực tiễn

Về mặt thực tiễn, hệ thống BKDNOJ có thể hỗ trợ người học trong quá trình luyện tập lập trình thông qua các chức năng như nộp bài, chấm bài tự động, chạy thử chương trình bằng IDE trực tuyến và theo dõi kết quả học tập. Hệ thống cũng hỗ trợ giảng viên hoặc người ra đề trong việc quản lý bài tập, quản lý test case, tổ chức kỳ thi và sử dụng AI để hỗ trợ phân tích mã nguồn, sinh test case.

Ngoài ra, cơ chế khóa tập trung trong kỳ thi giúp hạn chế một số hành vi gian lận khi thi trực tuyến, góp phần nâng cao tính nghiêm túc và công bằng trong quá trình tổ chức thi. Việc triển khai hệ thống bằng Docker và Docker Compose cũng giúp quá trình cài đặt, vận hành và mở rộng hệ thống trở nên thuận tiện hơn trong môi trường giáo dục.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Tổng quan về hệ thống chấm bài trực tuyến

2.1.1 Khái niệm hệ thống Online Judge

Online Judge là hệ thống chấm bài lập trình trực tuyến, cho phép người dùng nộp mã nguồn để giải các bài toán lập trình và nhận kết quả đánh giá tự động. Sau khi nhận bài nộp, hệ thống sẽ biên dịch chương trình, thực thi với các bộ **test case** đã được chuẩn bị trước và so sánh kết quả với đáp án để xác định tính đúng đắn của lời giải. [1]

Các hệ thống Online Judge được sử dụng rộng rãi trong học tập, luyện tập thuật toán và tổ chức các kỳ thi lập trình. Nhờ khả năng tự động hóa quá trình chấm bài, hệ thống giúp giảm công việc chấm thủ công, đảm bảo tính khách quan và cung cấp phản hồi nhanh chóng cho người học. [1]

2.1.2 Vai trò của Online Judge trong học tập và thi lập trình

Online Judge đóng vai trò quan trọng trong việc hỗ trợ học tập và rèn luyện kỹ năng lập trình. Hệ thống cho phép người học thực hành giải quyết bài toán, nhận kết quả đánh giá nhanh chóng và từng bước cải thiện tư duy thuật toán cũng như kỹ năng lập trình.

Bên cạnh đó, Online Judge còn là nền tảng hiệu quả để tổ chức các kỳ thi lập trình. Việc chấm bài được thực hiện tự động dựa trên cùng một bộ **test case**, giúp đảm bảo tính khách quan, công bằng và giảm đáng kể khối lượng công việc của người tổ chức.

2.1.3 Quy trình hoạt động cơ bản của hệ thống chấm bài trực tuyến

Quy trình hoạt động của một hệ thống Online Judge thường bắt đầu khi người dùng nộp mã nguồn cho một bài toán. Hệ thống tiếp nhận bài nộp, thực hiện biên dịch chương trình và chạy chương trình với các bộ **test case** đã được chuẩn bị trước. [1], [2]

Kết quả đầu ra của chương trình sẽ được so sánh với đáp án chuẩn để xác định tính đúng đắn của lời giải. Đồng thời, hệ thống cũng kiểm tra các giới hạn về thời gian thực thi và bộ nhớ sử dụng. Sau khi quá trình chấm hoàn tất, kết quả sẽ được lưu trữ và trả về cho người dùng dưới dạng các trạng thái như Accepted, Wrong Answer, Time Limit Exceeded hoặc Compilation Error.

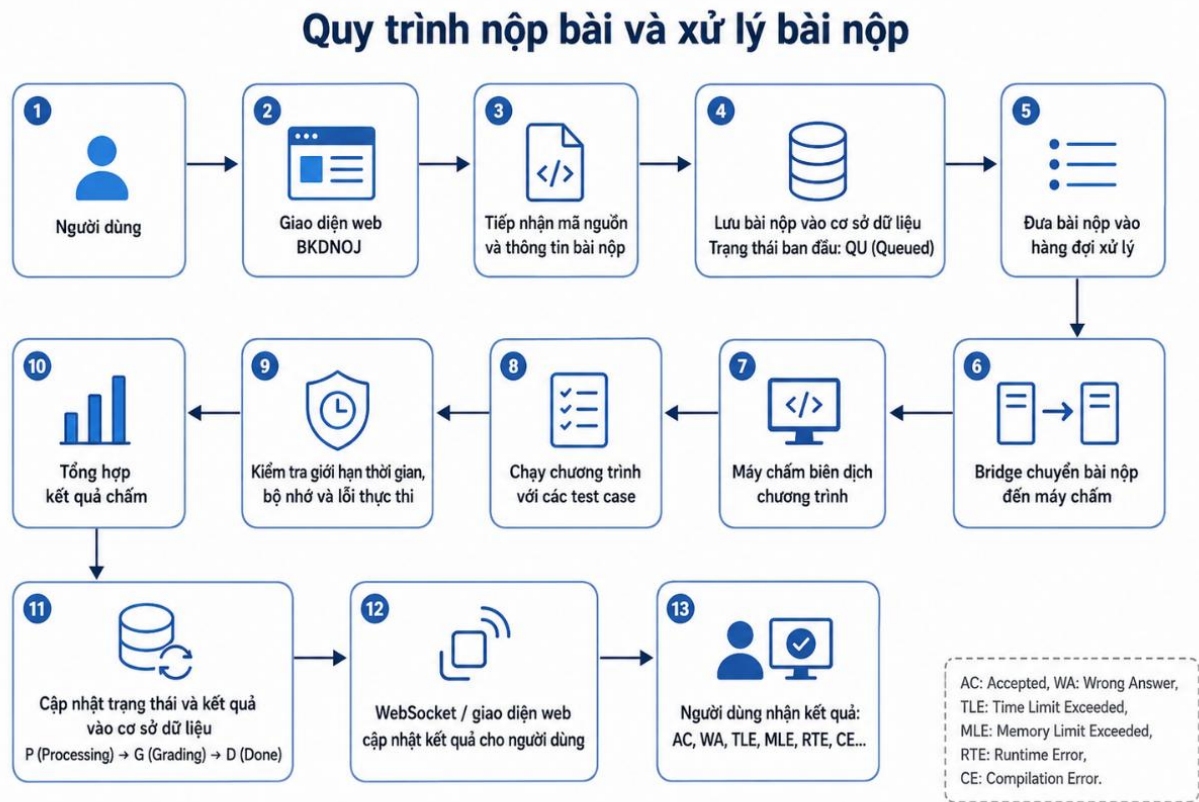
2.2 Quy trình chấm bài tự động

2.2.1 Quy trình nộp bài và xử lý bài nộp

Quy trình nộp bài trong hệ thống chấm bài trực tuyến bắt đầu khi người dùng chọn bài toán, ngôn ngữ lập trình và gửi mã nguồn lên hệ thống. Sau khi tiếp nhận bài nộp, hệ thống lưu thông tin bài nộp vào cơ sở dữ liệu, bao gồm người nộp, bài toán, ngôn ngữ lập trình, mã nguồn và trạng thái ban đầu của bài nộp.

Tiếp theo, bài nộp được đưa vào hàng đợi để chờ xử lý. Khi đến lượt, hệ thống chuyển bài nộp đến máy chấm để thực hiện biên dịch và chạy chương trình với các bộ **test case** đã được chuẩn bị trước. Trong quá trình này, hệ thống theo dõi các giới hạn về thời gian chạy, bộ nhớ sử dụng và lỗi phát sinh khi thực thi.

Sau khi quá trình chấm hoàn tất, kết quả của từng test case được tổng hợp để xác định kết quả cuối cùng của bài nộp. Hệ thống cập nhật trạng thái bài nộp, lưu kết quả chấm và trả thông tin về giao diện người dùng, giúp người dùng theo dõi được bài làm của mình một cách nhanh chóng và khách quan.



Hình 2.1.1: Quy trình nộp bài và xử lý bài nộp

2.2.2 Biên dịch và thực thi mã nguồn

Sau khi bài nộp được chuyển đến máy chấm, hệ thống tiến hành biên dịch mã nguồn dựa trên ngôn ngữ lập trình mà người dùng đã lựa chọn. Nếu quá trình biên dịch xảy ra lỗi, bài nộp sẽ được trả về trạng thái lỗi biên dịch và không tiếp tục chạy với các test case. [3]

Khi biên dịch thành công, chương trình được thực thi trong môi trường kiểm soát tài nguyên. Hệ thống lần lượt chạy chương trình với các test case đã được chuẩn bị trước, đồng thời theo dõi thời gian chạy, bộ nhớ sử dụng và các lỗi có thể phát sinh trong quá trình thực thi. Việc kiểm soát này giúp đảm bảo chương trình được chấm đúng theo giới hạn của bài toán và hạn chế rủi ro khi chạy mã nguồn do người dùng gửi lên.

2.2.3 So sánh kết quả và xác định trạng thái chấm

Sau khi chương trình được thực thi với từng test case, hệ thống thu nhận kết quả đầu ra của chương trình và so sánh với đáp án chuẩn đã được chuẩn bị trước. Việc so sánh có thể được thực hiện theo nhiều cách, chẳng hạn như so khớp chính xác, bỏ qua khoảng trắng không cần thiết hoặc sử dụng trình chấm riêng tùy theo yêu cầu của bài toán. [1], [3]

Dựa trên kết quả so sánh và quá trình thực thi, hệ thống xác định trạng thái chấm của bài nộp. Nếu kết quả đầu ra đúng với đáp án chuẩn và chương trình không vi phạm giới hạn thời gian, bộ nhớ, bài nộp sẽ được đánh giá là đúng. Ngược lại, nếu kết quả sai, chương trình chạy quá thời gian, vượt giới hạn bộ nhớ, lỗi khi thực thi hoặc lỗi biên dịch, hệ thống sẽ trả về trạng thái tương ứng cho người dùng.

2.2.4 Các kết quả chấm

Trong hệ thống Online Judge, sau khi quá trình chấm bài hoàn tất, bài nộp sẽ được gán một kết quả chấm tương ứng với trạng thái thực thi của chương trình. Các kết quả này giúp người dùng biết được lời giải của mình đúng, sai hoặc gặp lỗi ở bước nào trong quá trình biên dịch và chạy chương trình. [1], [3]

Các kết quả chấm được sử dụng trong hệ thống bao gồm:

Ký hiệu	Tên kết quả	Ý nghĩa
AC	Accepted	Chương trình cho kết quả đúng với tất cả các test case được chấm.
WA	Wrong Answer	Chương trình chạy được nhưng kết quả đầu ra không đúng với đáp án chuẩn.
TLE	Time Limit Exceeded	Chương trình chạy vượt quá giới hạn thời gian cho phép.
MLE	Memory Limit Exceeded	Chương trình sử dụng vượt quá giới hạn bộ nhớ cho phép.
RTE	Runtime Error	Chương trình gặp lỗi trong quá trình thực thi.
CE	Compilation Error	Mã nguồn không biên dịch thành công.

OLE	Output Limit Exceeded	Chương trình sinh ra lượng dữ liệu đầu ra vượt quá giới hạn cho phép.
IE	Internal Error	Hệ thống hoặc máy chấm gặp lỗi nội bộ trong quá trình xử lý bài nộp.

Bảng 2.2.4: Các trạng thái của kết quả chấm

2.3 Kiến trúc và công nghệ nền tảng của hệ thống

2.3.1 Kiến trúc web nhiều thành phần

Kiến trúc web nhiều thành phần là mô hình tổ chức hệ thống thành nhiều dịch vụ hoặc lớp xử lý khác nhau, mỗi thành phần đảm nhận một vai trò riêng trong quá trình vận hành. Thay vì toàn bộ chức năng được xử lý trong một ứng dụng duy nhất, hệ thống được tách thành các phần như giao diện người dùng, máy chủ web, ứng dụng xử lý nghiệp vụ, cơ sở dữ liệu, bộ nhớ đệm, tác vụ nền và các dịch vụ hỗ trợ khác.

Trong hệ thống chấm bài trực tuyến, kiến trúc nhiều thành phần giúp phân tách rõ ràng giữa phần tiếp nhận yêu cầu từ người dùng, phần quản lý dữ liệu, phần xử lý bài nộp và phần chấm bài. Ví dụ, ứng dụng web chịu trách nhiệm hiển thị giao diện, tiếp nhận bài nộp và trả kết quả; cơ sở dữ liệu lưu trữ thông tin người dùng, bài tập, bài nộp và test case; trong khi máy chấm đảm nhận việc biên dịch, chạy chương trình và xác định kết quả chấm.

Việc tổ chức hệ thống theo kiến trúc nhiều thành phần giúp hệ thống dễ quản lý, dễ triển khai và thuận tiện hơn khi mở rộng. Đối với BKDNOJ, mô hình này là nền tảng để kết hợp nhiều thành phần như Django Site, cơ sở dữ liệu, Redis, Celery, WebSocket, Bridge, Nginx và máy chấm nhằm phục vụ các chức năng nộp bài, chấm bài tự động, chạy thử chương trình và cập nhật kết quả theo thời gian thực.

2.3.2 Cơ sở dữ liệu, bộ nhớ đệm và tác vụ nền

Trong hệ thống web nhiều thành phần, cơ sở dữ liệu đóng vai trò lưu trữ các thông tin quan trọng của hệ thống như tài khoản người dùng, bài tập, bài nộp, test case, kết quả chấm, kỳ thi và các dữ liệu cấu hình khác. Đối với hệ thống chấm bài trực tuyến, cơ sở dữ liệu cần đảm bảo tính toàn vẹn dữ liệu, khả năng truy vấn nhanh và lưu trữ chính xác lịch sử hoạt động của người dùng.

Bên cạnh cơ sở dữ liệu, bộ nhớ đệm được sử dụng để tăng tốc độ xử lý và giảm tải cho hệ thống. Các dữ liệu được truy cập thường xuyên hoặc các trạng thái trung gian có thể được lưu trong bộ nhớ đệm nhằm giúp hệ thống phản hồi nhanh hơn. Trong BKDNOJ, Redis được sử dụng như một thành phần hỗ trợ lưu trữ tạm thời, quản lý hàng đợi và phục vụ cho các tác vụ cần xử lý bất đồng bộ. [6]

Tác vụ nền là các công việc không cần xử lý trực tiếp trong luồng yêu cầu của người dùng, chẳng hạn như xử lý hàng đợi, cập nhật dữ liệu, gửi thông báo hoặc thực hiện các công việc tốn thời gian. Việc tách các tác vụ này ra khỏi luồng xử lý chính giúp ứng dụng web duy trì khả năng phản hồi tốt hơn. Trong BKDNOJ, Celery được sử dụng để xử lý các tác vụ nền, phối hợp với Redis nhằm hỗ trợ hệ thống vận hành ổn định hơn trong các chức năng như chấm bài, cập nhật trạng thái và các xử lý liên quan. [7]

Thành phần	Vai trò trong hệ thống
Cơ sở dữ liệu	Lưu người dùng, bài tập, bài nộp, kỳ thi, kết quả chấm, ...
Redis	Bộ nhớ đệm, hàng đợi, lưu trạng thái trung gian
Celery	Xử lý tác vụ nền, giảm tải cho luồng xử lý web

Bảng 2.3.2: Thành phần các tác vụ nền trong hệ thống

2.3.3 WebSocket và cập nhật kết quả theo thời gian thực

Trong hệ thống chấm bài trực tuyến, trạng thái của bài nộp có thể thay đổi liên tục từ khi được đưa vào hàng đợi, đang xử lý, đang chấm cho đến khi hoàn tất. Nếu người dùng phải tải lại trang hoặc hệ thống liên tục gửi yêu cầu kiểm tra trạng thái, quá trình theo dõi kết quả sẽ kém hiệu quả và gây tốn tài nguyên.

WebSocket là cơ chế giao tiếp hai chiều giữa trình duyệt và máy chủ, cho phép máy chủ chủ động gửi dữ liệu mới đến trình duyệt khi có sự kiện xảy ra. Nhờ đó, người dùng có thể nhận được trạng thái bài nộp và kết quả chấm gần như ngay lập tức mà không cần tải lại trang. [5]

2.3.4 Docker và triển khai hệ thống nhiều dịch vụ

Docker là nền tảng cho phép đóng gói ứng dụng cùng với các thư viện, cấu hình và môi trường cần thiết vào các container. Nhờ đó, hệ thống có thể được triển khai nhất quán trên nhiều môi trường khác nhau, hạn chế lỗi do khác biệt về cấu hình máy chủ hoặc phiên bản phần mềm. [8]

Đối với hệ thống web nhiều thành phần, Docker Compose giúp định nghĩa và quản lý nhiều dịch vụ cùng lúc, chẳng hạn như ứng dụng web, cơ sở dữ liệu, Redis, Celery, WebSocket, Nginx và máy chấm. Mỗi dịch vụ được chạy trong một container riêng, có thể cấu hình mạng nội bộ, biến môi trường và vùng lưu trữ dữ liệu phù hợp. [9]

Trong BKDNOJ, Docker và Docker Compose được sử dụng để hỗ trợ quá trình cài đặt, triển khai và vận hành hệ thống. Cách triển khai này giúp các thành phần của hệ thống được tổ chức rõ ràng, dễ khởi động, dễ kiểm tra trạng thái và thuận tiện hơn khi mở rộng hoặc bảo trì.

Thành phần	Vai trò
Nginx	Tiếp nhận yêu cầu và chuyển tiếp đến các dịch vụ phù hợp
Django Site	Xử lý nghiệp vụ chính của hệ thống web
Cơ sở dữ liệu	Lưu trữ dữ liệu hệ thống
Redis	Bộ nhớ đệm và hỗ trợ hàng đợi
Celery	Xử lý tác vụ nền
WebSocket	Cập nhật kết quả theo thời gian thực
Bridge	Kết nối hệ thống web với máy chấm
Máy chấm	Biên dịch, thực thi và chấm bài nộp

Bảng 2.3.4: Các dịch vụ hệ thống

2.4 Môi trường chạy thử và chấm bài

2.4.1 Máy chấm và môi trường thực thi mã nguồn

Máy chấm là thành phần chịu trách nhiệm biên dịch, thực thi và đánh giá mã nguồn do người dùng gửi lên. Sau khi bài nộp được hệ thống tiếp nhận, máy chấm sẽ nhận mã nguồn,

ngôn ngữ lập trình, giới hạn thời gian, giới hạn bộ nhớ và các test case tương ứng để tiến hành chấm bài. [3]

Trong quá trình chấm, mã nguồn của người dùng được biên dịch theo ngôn ngữ đã chọn. Nếu biên dịch thành công, chương trình sẽ được chạy lần lượt với từng test case trong môi trường được kiểm soát. Môi trường này cần giới hạn thời gian thực thi, bộ nhớ sử dụng và dữ liệu đầu ra nhằm đảm bảo việc chấm bài diễn ra đúng theo yêu cầu của bài toán.

Do mã nguồn người dùng gửi lên là mã không tin cậy, môi trường thực thi cần có cơ chế cô lập và kiểm soát tài nguyên để hạn chế rủi ro cho hệ thống. Nhờ đó, máy chấm có thể đánh giá lời giải một cách tự động, khách quan và an toàn hơn.

2.4.2 Test case trong đánh giá lời giải

Trong hệ thống chấm bài trực tuyến, test case là bộ dữ liệu đầu vào và đầu ra tương ứng được sử dụng để kiểm tra tính đúng đắn của lời giải. Khi người dùng nộp mã nguồn, chương trình sẽ được chạy với từng test case; kết quả đầu ra của chương trình được so sánh với đáp án chuẩn để xác định bài làm đúng hay sai.

Test case có vai trò quan trọng trong việc đánh giá chất lượng lời giải. Một bộ test case tốt cần bao quát được nhiều trường hợp khác nhau của bài toán, bao gồm trường hợp đơn giản, trường hợp biên, trường hợp dữ liệu lớn và các tình huống dễ gây sai sót. Nhờ đó, hệ thống có thể phát hiện các lời giải chỉ đúng với một số trường hợp nhỏ nhưng chưa xử lý đầy đủ yêu cầu của bài toán. [1], [3]

Loại test case	Vai trò
Test cơ bản	Kiểm tra các trường hợp đơn giản, đúng theo ví dụ đề bài
Test biên	Kiểm tra các giá trị nhỏ nhất, lớn nhất hoặc trường hợp đặc biệt
Test dữ liệu lớn	Kiểm tra hiệu năng và giới hạn thời gian, bộ nhớ
Test phản ví dụ	Phát hiện các lời giải sai hoặc xử lý thiếu trường hợp

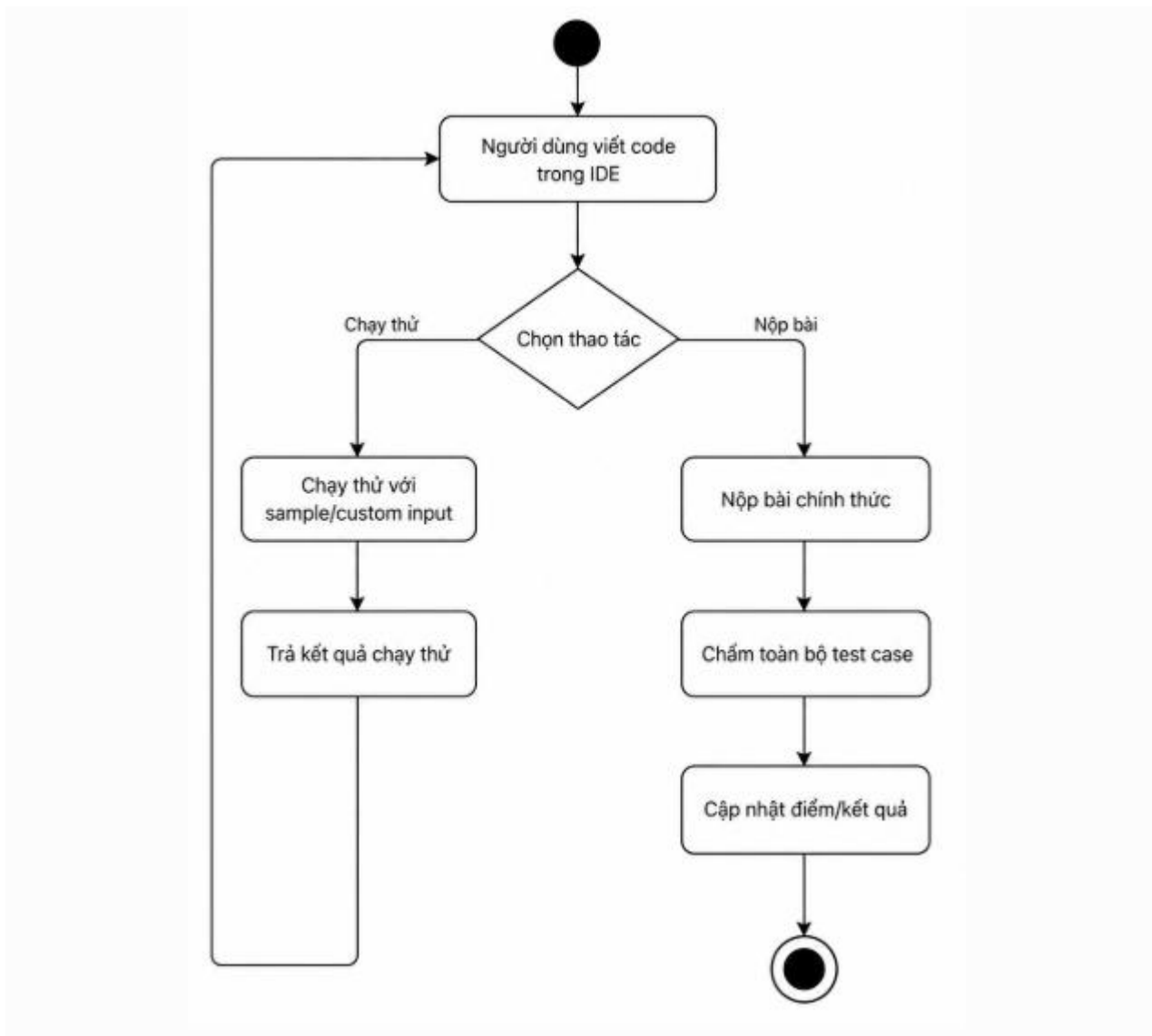
Bảng 2.4.2: Các loại test case

2.4.3 IDE trực tuyến và luồng chạy thử chương trình

IDE trực tuyến là môi trường cho phép người dùng viết mã, chỉnh sửa chương trình và chạy thử trực tiếp trên trình duyệt mà không cần sử dụng phần mềm lập trình bên ngoài. Trong hệ thống chấm bài trực tuyến, IDE giúp người học thao tác thuận tiện hơn khi đọc đề, viết lời giải, kiểm tra chương trình và nộp bài trong cùng một giao diện.

Luồng chạy thử chương trình cho phép người dùng kiểm tra mã nguồn với dữ liệu mẫu hoặc dữ liệu tự nhập trước khi gửi bài nộp chính thức. Kết quả chạy thử giúp người dùng phát hiện lỗi cơ bản, kiểm tra định dạng đầu ra và điều chỉnh lời giải trước khi chấm chính thức trên toàn bộ test case.

IDE trực tuyến và luồng chạy thử được xây dựng nhằm nâng cao trải nghiệm luyện tập lập trình. Luồng chạy thử được tách biệt với luồng nộp bài chính thức, vì vậy kết quả chạy thử không ảnh hưởng đến điểm số, bảng xếp hạng hoặc kết quả chấm cuối cùng. Điều này đặc biệt hữu ích trong các kỳ thi có cơ chế khóa tập trung, khi người dùng cần chạy thử chương trình ngay trong hệ thống mà không phải chuyển sang công cụ bên ngoài.



Hình 2.4.3: Sơ đồ sử dụng IDE trực tuyến

2.4.4 Vấn đề an toàn khi thực thi mã nguồn người dùng

Trong hệ thống chấm bài trực tuyến, mã nguồn do người dùng gửi lên là mã không tin cậy, vì hệ thống không thể biết trước chương trình đó có lỗi, chạy vô hạn hoặc chứa các thao tác gây ảnh hưởng đến máy chủ hay không. Do đó, việc thực thi mã nguồn người dùng cần được kiểm soát chặt chẽ để đảm bảo an toàn cho hệ thống.

Một số rủi ro có thể xảy ra khi chạy mã nguồn người dùng bao gồm chương trình chạy quá thời gian, sử dụng quá nhiều bộ nhớ, sinh dữ liệu đầu ra quá lớn, truy cập trái phép vào tệp

tin hệ thống hoặc thực hiện các thao tác không được phép. Nếu không có cơ chế kiểm soát, các chương trình này có thể làm giảm hiệu năng, gây treo máy chấm hoặc ảnh hưởng đến các bài nộp khác.

Vì vậy, môi trường thực thi trong hệ thống Online Judge cần có cơ chế giới hạn tài nguyên và cô lập chương trình khi chạy. Các giới hạn như thời gian thực thi, bộ nhớ sử dụng, kích thước đầu ra và quyền truy cập hệ thống giúp máy chấm xử lý bài nộp an toàn hơn. Đây là cơ sở quan trọng để hệ thống có thể chấm bài tự động một cách ổn định, khách quan và hạn chế rủi ro khi vận hành.

2.5 Trí tuệ nhân tạo trong hỗ trợ lập trình

2.5.1 Khái quát về AI và mô hình ngôn ngữ lớn

Trí tuệ nhân tạo là lĩnh vực nghiên cứu và phát triển các hệ thống có khả năng thực hiện một số nhiệm vụ cần đến trí tuệ của con người, chẳng hạn như nhận dạng, phân tích, suy luận, tạo nội dung và hỗ trợ ra quyết định. Trong những năm gần đây, AI được ứng dụng rộng rãi trong nhiều lĩnh vực, trong đó có giáo dục và phát triển phần mềm.

Mô hình ngôn ngữ lớn là một dạng mô hình AI được huấn luyện trên lượng dữ liệu văn bản rất lớn, có khả năng hiểu ngữ cảnh, sinh văn bản, trả lời câu hỏi, phân tích nội dung và hỗ trợ xử lý mã nguồn. Trong lĩnh vực lập trình, mô hình ngôn ngữ lớn có thể hỗ trợ giải thích mã nguồn, nhận xét lời giải, gợi ý cải thiện, tạo nội dung đề bài hoặc hỗ trợ sinh dữ liệu kiểm thử.

2.5.2 AI trong phân tích mã nguồn

AI có thể được sử dụng để hỗ trợ phân tích mã nguồn bằng cách đọc nội dung chương trình, nhận diện cách tổ chức lời giải và đưa ra nhận xét về quá trình cài đặt. Thông qua mô hình ngôn ngữ lớn, hệ thống có thể phân tích ý tưởng thuật toán, đánh giá độ rõ ràng của mã nguồn, xem xét độ phức tạp thời gian, độ phức tạp bộ nhớ và chỉ ra một số lỗi hoặc điểm chưa tối ưu trong lời giải.

Trong hệ thống chấm bài truyền thống, người dùng thường chỉ nhận được kết quả đúng hoặc sai sau khi nộp bài. Điều này giúp xác định lời giải có vượt qua test case hay không, nhưng chưa cho người dùng biết rõ nguyên nhân, điểm mạnh hoặc hạn chế trong cách giải của mình. Việc ứng dụng AI trong phân tích mã nguồn giúp bổ sung thêm góc nhìn tham

khảo, từ đó hỗ trợ người học hiểu rõ hơn về năng lực lập trình của bản thân, nhận biết các lỗi thường gặp và cải thiện chất lượng lời giải trong quá trình luyện tập.

Bên cạnh việc nhận xét mã nguồn, AI còn có thể hỗ trợ nhận diện dạng đề hoặc nhóm kiến thức liên quan đến bài toán, chẳng hạn như tham lam, quy hoạch động, đồ thị, tìm kiếm, cấu trúc dữ liệu hoặc xử lý chuỗi. Khi kết hợp với lịch sử các bài nộp đúng của người dùng, thông tin này có thể được sử dụng để đánh giá người dùng đang có thể mạnh hoặc còn hạn chế ở những dạng bài nào, từ đó hỗ trợ gợi ý hướng luyện tập phù hợp hơn.

2.5.3 AI trong hỗ trợ tạo đề bài

AI có thể hỗ trợ quá trình tạo đề bài bằng cách xử lý nội dung đầu vào từ ảnh hoặc văn bản do người dùng cung cấp. Đối với các đề bài lập trình đã có sẵn dưới dạng hình ảnh, việc nhập lại thủ công nội dung đề có thể mất thời gian và dễ xảy ra sai sót. AI có thể hỗ trợ nhận diện nội dung, trích xuất thông tin và viết lại đề bài theo định dạng rõ ràng, phù hợp với hệ thống chấm bài trực tuyến.

Trong quá trình hỗ trợ tạo đề bài, AI chủ yếu tập trung vào phân nội dung của đề như tên bài, mô tả bài toán, định dạng dữ liệu vào, dữ liệu ra và ví dụ minh họa. Việc chuẩn hóa nội dung theo cấu trúc thống nhất giúp người ra đề hoặc quản trị viên giảm bớt thao tác nhập liệu thủ công, đồng thời giúp đề bài được trình bày rõ ràng và dễ đọc hơn.

2.5.4 AI trong sinh test case

Trong hệ thống chấm bài trực tuyến, test case đóng vai trò quan trọng trong việc kiểm tra tính đúng đắn và độ ổn định của lời giải. Tuy nhiên, việc xây dựng bộ test case thủ công thường mất nhiều thời gian, đặc biệt đối với các bài toán có dữ liệu lớn, nhiều trường hợp biên hoặc cần kiểm tra nhiều hướng giải sai khác nhau. Vì vậy, AI có thể được sử dụng như một công cụ hỗ trợ người ra đề trong quá trình xây dựng dữ liệu kiểm thử.

AI trong sinh test case không nhất thiết trực tiếp tạo toàn bộ input và output, vì dữ liệu kiểm thử có thể rất lớn và cần đảm bảo đúng định dạng, đúng ràng buộc. Thay vào đó, AI có thể hỗ trợ đề xuất ý tưởng sinh dữ liệu, viết chương trình sinh input hoặc hỗ trợ xây dựng chương trình lời giải chuẩn để tạo output tương ứng. Cách tiếp cận này phù hợp hơn với hệ thống chấm bài lập trình, vì dữ liệu kiểm thử được tạo ra thông qua chương trình và có thể kiểm soát tốt hơn về kích thước, cấu trúc và tính hợp lệ.

Trong BKDNOJ, AI được sử dụng để hỗ trợ quá trình sinh test case cho bài toán, giúp người ra đề giảm bớt công sức khi xây dựng bộ dữ liệu kiểm thử. Tuy nhiên, các test case được sinh ra vẫn cần được người dùng kiểm tra lại trước khi sử dụng chính thức, nhằm đảm bảo dữ liệu đúng với yêu cầu đề bài, bao quát đủ các trường hợp cần kiểm thử và không gây sai lệch trong quá trình chấm bài. [12], [13]

2.5.5 AI trong hỗ trợ đánh giá năng lực người dùng

Trong quá trình học lập trình, việc đánh giá năng lực người dùng không chỉ dựa trên số lượng bài đã giải đúng, mà còn cần xem xét người dùng mạnh hoặc yếu ở những dạng bài nào, cách tiếp cận lời giải ra sao và mức độ ổn định trong quá trình luyện tập. Các hệ thống chấm bài truyền thống thường cung cấp kết quả chấm cho từng bài nộp, nhưng chưa thể hiện rõ năng lực của người dùng theo từng nhóm kiến thức hoặc kỹ năng lập trình.

AI có thể hỗ trợ đánh giá năng lực người dùng bằng cách phân tích mã nguồn, lịch sử bài nộp đúng và dạng đề của các bài toán mà người dùng đã giải. Thông qua đó, hệ thống có thể nhận diện các nhóm kiến thức mà người dùng thường xuyên xử lý tốt, chẳng hạn như tham lam, quy hoạch động, đồ thị, tìm kiếm, cấu trúc dữ liệu hoặc xử lý chuỗi. Ngược lại, hệ thống cũng có thể phát hiện những dạng bài mà người dùng còn ít luyện tập hoặc chưa đạt kết quả tốt.

2.5.6 AI trong dự đoán dạng đề dựa trên bài nộp đúng

Dạng đề là nhóm kiến thức hoặc kỹ thuật giải thuật mà một bài toán thuộc về, chẳng hạn như tham lam, quy hoạch động, đồ thị, tìm kiếm, cấu trúc dữ liệu hoặc xử lý chuỗi. Việc xác định đúng dạng đề giúp hệ thống phân loại bài toán rõ ràng hơn, đồng thời hỗ trợ người học lựa chọn bài tập phù hợp với năng lực và mục tiêu luyện tập.

AI có thể hỗ trợ dự đoán dạng đề bằng cách phân tích nội dung bài toán và các bài nộp đúng của người dùng. Khi một bài toán đã có các lời giải được chấm đúng, hệ thống có thể sử dụng những bài nộp này như dữ liệu tham khảo để nhận diện kỹ thuật giải chính được sử dụng. Từ đó, AI đưa ra gợi ý về dạng đề phù hợp, giúp người ra đề hoặc quản trị viên có thêm cơ sở để gắn nhãn bài toán.

Chức năng dự đoán dạng đề dựa trên bài nộp đúng được sử dụng như một công cụ hỗ trợ phân loại bài tập. Kết quả dự đoán của AI không được áp dụng một cách tự động hoàn toàn,

mà cần được người quản trị kiểm tra và xác nhận trước khi sử dụng chính thức. Điều này giúp đảm bảo việc phân loại bài toán chính xác hơn, đồng thời hỗ trợ quá trình đánh giá năng lực người dùng theo từng nhóm kiến thức.

2.6 Cơ chế hỗ trợ hạn chế gian lận trong kỳ thi trực tuyến

2.6.1. Vấn đề gian lận trong thi lập trình trực tuyến

Trong các kỳ thi lập trình trực tuyến, việc đảm bảo tính trung thực và công bằng là một yêu cầu quan trọng. Khác với thi trực tiếp trong phòng máy có giám sát, thi trực tuyến khiến người tổ chức khó kiểm soát hoàn toàn môi trường làm bài của thí sinh. Người dùng có thể chuyển sang tab khác, sử dụng tài liệu bên ngoài, trao đổi thông tin hoặc tận dụng các công cụ hỗ trợ không được phép trong quá trình làm bài.

Trong bối cảnh các công cụ trí tuệ nhân tạo ngày càng phổ biến, nguy cơ gian lận trong thi lập trình trực tuyến cũng tăng lên. Thí sinh có thể sử dụng AI để phân tích đề bài, gợi ý thuật toán hoặc tạo lời giải nếu hệ thống không có cơ chế kiểm soát phù hợp. Vì vậy, các hệ thống tổ chức kỳ thi trực tuyến cần có thêm các biện pháp hỗ trợ hạn chế gian lận, nhằm nâng cao tính nghiêm túc và công bằng trong quá trình thi.

2.6.2. Cơ chế khóa tập trung trong kỳ thi

Cơ chế khóa tập trung trong kỳ thi là một biện pháp hỗ trợ nhằm yêu cầu người dùng duy trì sự tập trung trong giao diện làm bài. Khi cơ chế này được bật, hệ thống có thể yêu cầu người dùng làm bài trong chế độ toàn màn hình, hạn chế việc chuyển trang, rời khỏi cửa sổ trình duyệt hoặc thoát khỏi giao diện thi trong quá trình làm bài. [14], [15]

Khi phát hiện người dùng mất tập trung khỏi giao diện thi, hệ thống có thể ghi nhận hành vi vi phạm để phục vụ quá trình theo dõi và xử lý. Cơ chế này không chỉ giúp cảnh báo người dùng trong lúc thi, mà còn cung cấp thêm thông tin cho người tổ chức kỳ thi khi cần đánh giá tính hợp lệ của quá trình làm bài.

Cơ chế khóa tập trung trong kỳ thi được sử dụng nhằm hỗ trợ hạn chế các hành vi gian lận khi thi trực tuyến. Cơ chế này đặc biệt phù hợp khi kết hợp với IDE trực tuyến và luồng chạy thử trong hệ thống, vì người dùng có thể viết mã, chạy thử chương trình và nộp bài ngay trong giao diện thi mà không cần chuyển sang công cụ bên ngoài.

2.6.3. Giới hạn của cơ chế khóa tập trung trên nền web

Mặc dù cơ chế khóa tập trung có thể hỗ trợ hạn chế một số hành vi gian lận, nhưng đây không phải là giải pháp chống gian lận tuyệt đối. Do hoạt động trên nền web, hệ thống chỉ có thể phát hiện một số hành vi như chuyển tab, mất tập trung hoặc thoát khỏi chế độ toàn màn hình. Những hành vi bên ngoài trình duyệt, thiết bị phụ hoặc trao đổi qua các kênh khác vẫn rất khó kiểm soát hoàn toàn.

Vì vậy, cơ chế khóa tập trung nên được xem là một biện pháp hỗ trợ, kết hợp với các phương pháp quản lý kỳ thi khác như quy định rõ ràng, giám sát phù hợp, thiết kế đề thi hợp lý và phân tích kết quả sau kỳ thi. Trong phạm vi hệ thống BKDNOJ, cơ chế này góp phần nâng cao tính nghiêm túc của kỳ thi trực tuyến, nhưng không thay thế hoàn toàn các giải pháp giám sát chuyên dụng.

2.7 Tổng kết chương

Chương 2 đã trình bày các cơ sở lý thuyết và công nghệ nền tảng liên quan đến việc xây dựng hệ thống chấm bài trực tuyến BKDNOJ. Nội dung chương đã giới thiệu tổng quan về hệ thống Online Judge, quy trình chấm bài tự động, các kết quả chấm, vai trò của test case, máy chấm và môi trường thực thi mã nguồn người dùng.

Bên cạnh đó, chương cũng trình bày các công nghệ và thành phần quan trọng trong hệ thống như kiến trúc web nhiều thành phần, cơ sở dữ liệu, bộ nhớ đệm, tác vụ nền, WebSocket, Docker và IDE trực tuyến. Đây là những nền tảng cần thiết để hệ thống có thể vận hành ổn định, hỗ trợ cập nhật kết quả theo thời gian thực và triển khai dưới dạng nhiều dịch vụ.

Ngoài ra, chương đã đề cập đến vai trò của trí tuệ nhân tạo trong hỗ trợ lập trình, bao gồm phân tích mã nguồn, hỗ trợ tạo đề bài, sinh test case, đánh giá năng lực người dùng và dự đoán dạng đề. Cơ chế hỗ trợ hạn chế gian lận trong kỳ thi trực tuyến cũng được trình bày nhằm làm rõ nhu cầu đảm bảo tính công bằng khi tổ chức thi trên nền web.

Những nội dung trong chương này là cơ sở để thực hiện việc phân tích yêu cầu, thiết kế kiến trúc, thiết kế cơ sở dữ liệu và các luồng xử lý chính của hệ thống BKDNOJ trong chương tiếp theo.

CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1 Tổng quan hệ thống BKDNOJ

3.1.1 Mục tiêu thiết kế hệ thống

Mục tiêu thiết kế hệ thống BKDNOJ là xây dựng một nền tảng chấm bài lập trình trực tuyến có khả năng hỗ trợ học tập, luyện tập và tổ chức các kỳ thi lập trình một cách hiệu quả. Hệ thống cần đảm bảo các chức năng cốt lõi như quản lý bài tập, quản lý test case, tiếp nhận bài nộp, chấm bài tự động, hiển thị kết quả và hỗ trợ người dùng theo dõi quá trình luyện tập.

Bên cạnh các chức năng chấm bài truyền thống, hệ thống được thiết kế để mở rộng thêm các chức năng hỗ trợ hiện đại như IDE trực tuyến, luồng chạy thử chương trình, tích hợp AI trong phân tích mã nguồn, hỗ trợ tạo đề bài, sinh test case, đánh giá năng lực người dùng và dự đoán dạng đề. Các chức năng này nhằm nâng cao trải nghiệm học tập, giảm công sức cho người ra đề và cung cấp thêm thông tin hỗ trợ quá trình đánh giá.

Ngoài ra, hệ thống cũng hướng đến việc hỗ trợ tổ chức kỳ thi lập trình trực tuyến với cơ chế khóa tập trung nhằm hạn chế gian lận, cập nhật kết quả theo thời gian thực và đảm bảo khả năng triển khai, vận hành thuận tiện thông qua kiến trúc nhiều thành phần. Việc thiết kế hệ thống cần đảm bảo tính ổn định, dễ mở rộng, dễ bảo trì và phù hợp với nhu cầu sử dụng trong môi trường giáo dục.

3.1.2 Các nhóm người dùng của hệ thống

Hệ thống BKDNOJ được thiết kế để phục vụ nhiều nhóm người dùng khác nhau trong quá trình học tập, luyện tập và tổ chức thi lập trình. Mỗi nhóm người dùng có vai trò và phạm vi chức năng riêng, từ việc sử dụng hệ thống để làm bài, quản lý bài tập, tổ chức kỳ thi cho đến vận hành các thành phần xử lý tự động.

Nhóm người học là nhóm sử dụng hệ thống để xem bài tập, viết mã, chạy thử chương trình, nộp bài và theo dõi kết quả chấm. Người học cũng có thể sử dụng các chức năng hỗ trợ như IDE trực tuyến, AI phân tích mã nguồn và theo dõi năng lực của bản thân thông qua quá trình luyện tập.

Nhóm người ra đề hoặc giảng viên có nhiệm vụ xây dựng và quản lý nội dung bài tập, test case và kỳ thi. Nhóm này có thể sử dụng các chức năng hỗ trợ như tạo đề bài bằng AI, sinh

test case, quản lý bài nộp và theo dõi kết quả làm bài của người học. Trong các kỳ thi trực tuyến, người ra đề hoặc giảng viên cũng có thể sử dụng cơ chế khóa tập trung để hỗ trợ hạn chế gian lận.

Nhóm quản trị viên chịu trách nhiệm quản lý toàn bộ hệ thống, bao gồm quản lý người dùng, phân quyền, cấu hình hệ thống, quản lý kỳ thi, quản lý máy chấm và các chức năng liên quan đến AI. Đây là nhóm có quyền cao nhất, đảm bảo hệ thống được vận hành ổn định và phù hợp với mục tiêu sử dụng.

Bên cạnh các nhóm người dùng trực tiếp, hệ thống còn có các tác nhân kỹ thuật như máy chấm và dịch vụ AI. Máy chấm đảm nhận việc biên dịch, thực thi và chấm bài nộp dựa trên test case. Dịch vụ AI hỗ trợ các chức năng như phân tích mã nguồn, tạo đề bài, sinh test case, đánh giá năng lực người dùng và dự đoán dạng đề.

Nhóm người dùng/tác nhân	Vai trò chính
Người học	Làm bài, chạy thử, nộp bài, xem kết quả
Người ra đề/Giảng viên	Quản lý bài tập, test case, kỳ thi
Quản trị viên	Quản lý hệ thống, người dùng, cấu hình
Máy chấm	Biên dịch, chạy chương trình, chấm bài
Dịch vụ AI	Hỗ trợ phân tích mã nguồn, tạo đề, sinh test case

Bảng 3.1.2: Các nhóm người dùng và tác nhân của hệ thống

3.2 Phân tích yêu cầu hệ thống

Phân tích yêu cầu hệ thống là bước quan trọng nhằm xác định các chức năng cần có, phạm vi hoạt động và các tiêu chí chất lượng mà hệ thống BKDNOJ cần đáp ứng. Dựa trên mục tiêu của đề tài, hệ thống cần phục vụ tốt quá trình học tập, luyện tập lập trình, tổ chức kỳ thi, chấm bài tự động, chạy thử chương trình, tích hợp AI và hỗ trợ hạn chế gian lận trong kỳ thi trực tuyến.

3.2.1 Yêu cầu chức năng

Yêu cầu chức năng mô tả các chức năng chính mà hệ thống cần cung cấp cho từng nhóm người dùng. Đối với BKDNOJ, các chức năng được thiết kế xoay quanh hoạt động quản lý bài tập, nộp bài, chấm bài, tổ chức kỳ thi, chạy thử chương trình bằng IDE trực tuyến và các chức năng AI hỗ trợ người học, người ra đề và quản trị viên.

Nhóm chức năng	Mô tả yêu cầu
Quản lý người dùng	Cho phép đăng ký, đăng nhập, quản lý thông tin cá nhân, phân quyền người dùng và theo dõi quá trình học tập.
Quản lý bài tập	Cho phép tạo, chỉnh sửa, phân loại, hiển thị và quản lý các bài tập lập trình trong hệ thống.
Quản lý test case	Cho phép người ra đề quản lý dữ liệu kiểm thử phục vụ quá trình chấm bài tự động.
Nộp bài và chấm bài	Cho phép người dùng nộp mã nguồn, hệ thống tự động biên dịch, thực thi và trả kết quả chấm.
Chạy thử chương trình bằng IDE trực tuyến	Cho phép người dùng viết mã và chạy thử chương trình với dữ liệu mẫu hoặc dữ liệu tự nhập trước khi nộp bài chính thức.
Chế độ chấm điểm linh hoạt	Hỗ trợ nhiều cách tính điểm khác nhau như chấm toàn bộ, chấm theo nhóm test case hoặc chấm theo từng test case.
Tổ chức kỳ thi	Cho phép tạo kỳ thi, quản lý bài trong kỳ thi, thời gian thi, thí sinh tham gia và kết quả thi.
Khóa tập trung trong kỳ thi	Hỗ trợ hạn chế gian lận bằng cách yêu cầu người dùng tập trung trong giao diện thi và ghi nhận hành vi vi phạm.
AI phân tích mã nguồn	Hỗ trợ phân tích mã nguồn, nhận xét lời giải, đánh giá cách tiếp cận và cung cấp thông tin phục vụ đánh giá năng lực người dùng.
AI hỗ trợ tạo đề bài	Hỗ trợ tạo nội dung đề bài theo định dạng chuẩn từ ảnh hoặc văn bản đầu vào.
AI sinh test case	Hỗ trợ người ra đề xây dựng dữ liệu kiểm thử thông qua chương trình sinh input và chương trình lời giải chuẩn.

AI dự đoán dạng đề	Hỗ trợ dự đoán dạng đề hoặc nhóm kiến thức của bài toán dựa trên các bài nộp đúng.
AI hỗ trợ đánh giá năng lực người dùng	Hỗ trợ tổng hợp thông tin từ bài nộp đúng, mã nguồn và dạng đề để đánh giá điểm mạnh, điểm yếu của người dùng.
Quản lý API key AI	Cho phép người dùng thêm, kiểm tra, quản lý API key và sử dụng các nhà cung cấp AI được hệ thống hỗ trợ.
Xếp hạng và bảng xếp hạng	Hỗ trợ tính điểm, xếp hạng người dùng và hiển thị bảng xếp hạng trong quá trình luyện tập hoặc thi lập trình.

Bảng 3.2.1: Yêu cầu chức năng của hệ thống

Nhìn chung, các yêu cầu chức năng của BKDNOJ không chỉ bao gồm các chức năng cơ bản của một hệ thống chấm bài trực tuyến, mà còn mở rộng thêm các chức năng hỗ trợ học tập, tổ chức kỳ thi, hạn chế gian lận và ứng dụng AI trong quá trình đánh giá năng lực lập trình.

3.2.2 Yêu cầu phi chức năng

Yêu cầu phi chức năng mô tả các tiêu chí về chất lượng, hiệu năng, bảo mật và khả năng vận hành của hệ thống. Đây là các yêu cầu không trực tiếp biểu hiện thành một chức năng cụ thể, nhưng ảnh hưởng lớn đến độ ổn định, độ tin cậy và trải nghiệm sử dụng của hệ thống.

Nhóm yêu cầu	Mô tả yêu cầu
Tính ổn định	Hệ thống cần hoạt động ổn định trong quá trình người dùng làm bài, chạy thử chương trình, nộp bài và tham gia kỳ thi.
Hiệu năng xử lý	Hệ thống cần tiếp nhận bài nộp, phân phối đến máy chấm, xử lý kết quả và phản hồi cho người dùng trong thời gian hợp lý.
Khả năng cập nhật thời gian thực	Trạng thái bài nộp, kết quả chấm, kết quả chạy thử và trạng thái các tác vụ xử lý cần được cập nhật nhanh chóng trên giao diện người dùng.
Bảo mật người dùng	Hệ thống cần kiểm soát đăng nhập, phân quyền, bảo vệ thông tin người dùng và hạn chế truy cập trái phép vào các chức năng quản trị.

Bảo mật API key AI	API key của người dùng cần được lưu trữ và xử lý an toàn, hạn chế việc lộ khóa truy cập đến các dịch vụ AI bên ngoài.
An toàn khi thực thi mã nguồn	Mã nguồn do người dùng gửi lên cần được biên dịch và thực thi trong môi trường được kiểm soát, có giới hạn thời gian, bộ nhớ và tài nguyên sử dụng.
Tính toàn vẹn dữ liệu	Dữ liệu về bài tập, test case, bài nộp, kết quả chấm, kỳ thi và người dùng cần được lưu trữ chính xác, hạn chế mất mát hoặc sai lệch trong quá trình xử lý.
Khả năng mở rộng	Kiến trúc hệ thống cần cho phép mở rộng các thành phần như máy chấm, dịch vụ nền, WebSocket và các chức năng AI khi số lượng người dùng hoặc bài nộp tăng lên.
Khả năng triển khai	Hệ thống cần được triển khai thuận tiện thông qua Docker và Docker Compose, giúp việc cài đặt, cấu hình và vận hành các dịch vụ dễ dàng hơn.
Tính dễ sử dụng	Giao diện hệ thống cần rõ ràng, dễ thao tác, hỗ trợ người dùng trong quá trình xem bài, viết mã, chạy thử, nộp bài và theo dõi kết quả.
Khả năng bảo trì	Hệ thống cần được tổ chức theo các thành phần rõ ràng, thuận tiện cho việc sửa lỗi, nâng cấp và phát triển thêm chức năng mới.
Khả năng giám sát và ghi log	Hệ thống cần có khả năng ghi nhận các sự kiện quan trọng như bài nộp, kết quả chấm, lỗi xử lý, tác vụ nền và hành vi vi phạm trong kỳ thi để hỗ trợ kiểm tra và vận hành.

Bảng 3.2.2: Yêu cầu phi chức năng của hệ thống

Các yêu cầu phi chức năng trên là cơ sở để lựa chọn kiến trúc nhiều thành phần cho BKDNOJ, bao gồm ứng dụng web, cơ sở dữ liệu, bộ nhớ đệm, tác vụ nền, WebSocket, Bridge và máy chấm. Việc đáp ứng các yêu cầu này giúp hệ thống vận hành ổn định, an toàn và phù hợp với môi trường học tập, luyện tập cũng như tổ chức kỳ thi lập trình trực tuyến.

3.3 Sơ đồ Use Case của hệ thống

Sau khi xác định các yêu cầu của hệ thống, sơ đồ Use Case được sử dụng để mô tả tổng quan các chức năng chính của BKDNOJ và mối quan hệ giữa các nhóm người dùng với hệ

thông. Sơ đồ tập trung thể hiện những chức năng mà người dùng có thể thực hiện thay vì mô tả chi tiết cách hệ thống xử lý bên trong.

Đối với BKDNOJ, sơ đồ Use Case khái quát các hoạt động như xem bài tập, chạy thử chương trình, nộp bài, xem kết quả chấm, tham gia kỳ thi, quản lý bài tập, quản lý test case, sử dụng các chức năng AI và quản trị hệ thống. Thông qua đó, có thể thấy được vai trò của từng tác nhân, bao gồm người học, giảng viên, quản trị viên, máy chấm và dịch vụ AI.

Sơ đồ Use Case là cơ sở để phân tích chi tiết các chức năng và quy trình xử lý của hệ thống trong các phần tiếp theo.

3.3.1 Sơ đồ Use Case tổng quát

Sơ đồ Use Case tổng quát của hệ thống BKDNOJ mô tả các tác nhân chính tham gia vào hệ thống và các chức năng tương ứng với từng tác nhân. Trong đó, người học là nhóm sử dụng hệ thống để phục vụ quá trình luyện tập lập trình, bao gồm xem bài tập, viết mã, chạy thử chương trình, nộp bài và theo dõi kết quả chấm. Đây là nhóm người dùng trung tâm của hệ thống, trực tiếp tương tác với các chức năng học tập và đánh giá.

Bên cạnh người học, giảng viên hoặc người ra đề là nhóm chịu trách nhiệm xây dựng nội dung học tập và tổ chức kỳ thi. Nhóm này có thể quản lý bài tập, quản lý test case, tạo kỳ thi, theo dõi bài nộp và sử dụng các chức năng AI để hỗ trợ tạo đề bài, sinh test case hoặc phân tích mã nguồn. Các chức năng này giúp giảm bớt thao tác thủ công và hỗ trợ quá trình quản lý nội dung trong hệ thống.

Quản trị viên là tác nhân có phạm vi quyền cao nhất, đảm nhận việc quản lý người dùng, phân quyền, cấu hình hệ thống, quản lý máy chấm và giám sát hoạt động chung của BKDNOJ. Ngoài các tác nhân là người dùng trực tiếp, hệ thống còn có máy chấm và dịch vụ AI. Máy chấm thực hiện biên dịch, thực thi và chấm bài nộp dựa trên test case, trong khi dịch vụ AI hỗ trợ các chức năng thông minh như phân tích mã nguồn, tạo đề bài, sinh test case, đánh giá năng lực người dùng và dự đoán dạng đề.

Sơ đồ Use Case tổng quát dưới đây thể hiện mối quan hệ giữa các tác nhân và các chức năng chính của hệ thống BKDNOJ.



Hình 3.3.1: Sơ đồ Use Case tổng quát của hệ thống

3.3.2 Đặc tả một số Use Case chính

Sau khi xây dựng sơ đồ Use Case tổng quát, một số Use Case quan trọng của hệ thống được đặc tả chi tiết hơn nhằm làm rõ tác nhân tham gia, điều kiện thực hiện, luồng xử lý chính và kết quả đạt được. Các Use Case được lựa chọn là những chức năng tiêu biểu, thể hiện rõ vai trò của BKDNOJ trong việc hỗ trợ luyện tập lập trình, chấm bài tự động, tổ chức kỳ thi và tích hợp các chức năng AI.

Use Case	Tác nhân chính	Điều kiện thực hiện	Luồng xử lý chính	Kết quả
Đăng ký, đăng nhập	Khách truy cập, người dùng	Người dùng truy cập hệ thống và có tài khoản hợp lệ nếu đăng nhập	Người dùng nhập thông tin tài khoản, hệ thống kiểm tra dữ liệu và xác thực thông tin đăng nhập	Người dùng đăng nhập thành công và sử dụng các chức năng theo quyền được cấp
Xem bài tập	Khách truy cập, Người dùng	Bài tập đã được công khai hoặc người dùng có quyền truy cập	Người dùng truy cập danh sách bài tập, chọn bài toán và xem nội dung đề bài, giới hạn thời gian, giới hạn bộ nhớ, dữ liệu vào, dữ liệu ra và ví dụ	Nội dung bài tập được hiển thị để người dùng đọc và làm bài
Chạy thử chương trình	Người dùng	Người dùng đã đăng nhập và đang ở trang bài tập có hỗ trợ IDE trực tuyến	Người dùng nhập mã nguồn, chọn ngôn ngữ lập trình và thực hiện chạy thử. Hệ thống tạo bản ghi chạy thử riêng, gửi mã nguồn đến máy chấm và trả kết quả về giao diện	Người dùng nhận kết quả chạy thử mà không ảnh hưởng đến điểm số, bài nộp chính thức hoặc bảng xếp hạng
Nộp bài chính thức	Người dùng	Người dùng đã đăng nhập, bài tập cho phép nộp bài và mã nguồn hợp lệ	Người dùng chọn ngôn ngữ, gửi mã nguồn, hệ thống tạo bài nộp, chuyển đến máy chấm thông qua Bridge và cập nhật kết quả sau khi chấm	Bài nộp được lưu, chấm tự động và trả kết quả chính thức cho người dùng

Xem kết quả chấm	Người dùng, giảng viên, quản trị viên	Đã có bài nộp được xử lý trong hệ thống	Người dùng truy cập lịch sử bài nộp hoặc trang kết quả, hệ thống hiển thị trạng thái và kết quả chấm tương ứng	Người dùng theo dõi được kết quả bài làm như Accepted, Wrong Answer, Time Limit Exceeded, Runtime Error hoặc Compilation Error
Quản lý bài tập	Giảng viên, quản trị viên	Người dùng có quyền quản lý bài tập	Người quản lý tạo mới hoặc chỉnh sửa bài tập, nhập nội dung đề, cấu hình giới hạn thời gian, giới hạn bộ nhớ, điểm số và các thông tin liên quan	Bài tập được lưu vào hệ thống và có thể được sử dụng để luyện tập hoặc đưa vào kỳ thi
Quản lý test case	Giảng viên, quản trị viên	Bài tập đã tồn tại và người dùng có quyền quản lý test case	Người quản lý thêm, sửa, xóa hoặc cập nhật dữ liệu kiểm thử cho bài toán	Bộ test case được lưu và sử dụng trong quá trình chấm bài tự động
Tổ chức kỳ thi	Giảng viên, quản trị viên	Người dùng có quyền tạo và quản lý kỳ thi	Người quản lý tạo kỳ thi, thiết lập thời gian, thêm bài toán, cấu hình thí sinh tham gia và theo dõi kết quả	Kỳ thi được tạo và Người dùng có thể tham gia theo cấu hình của hệ thống
Tham gia kỳ thi có khóa tập trung	Người dùng	Người dùng là thí sinh chính thức của kỳ thi và kỳ thi bắt cơ chế khóa tập trung	Người dùng truy cập kỳ thi, được chuyển đến trang trung gian, kích hoạt toàn màn hình và làm bài trong giao diện được kiểm soát. Hệ thống ghi nhận vi phạm nếu người dùng thoát	Thí sinh tham gia kỳ thi trong môi trường được kiểm soát, số lần vi phạm được lưu để giảng viên hoặc quản trị viên xem xét

			toàn màn hình, chuyển tab hoặc rời cửa sổ thi	
Quản lý API key AI	Người dùng	Người dùng đã đăng nhập và có API key từ nhà cung cấp AI	Người dùng thêm API key, chọn nhà cung cấp, kiểm tra kết nối và lưu cấu hình sử dụng	API key được lưu trữ để phục vụ các chức năng AI của người dùng
AI phân tích mã nguồn	Người dùng, giảng viên	Người dùng có mã nguồn cần phân tích và đã cấu hình API key AI hợp lệ	Người dùng gửi mã nguồn và thông tin bài toán, hệ thống gửi yêu cầu đến dịch vụ AI và nhận kết quả phân tích	Người dùng nhận được nhận xét về lời giải, độ phức tạp, lỗi có thể gặp và gợi ý cải thiện
AI hỗ trợ tạo đề bài	Giảng viên, quản trị viên	Người dùng có ảnh hoặc văn bản đầu vào của đề bài và API key AI hợp lệ	Người dùng cung cấp dữ liệu đầu vào, hệ thống gửi đến AI để trích xuất và chuẩn hóa nội dung đề	Hệ thống trả về bản nháp đề bài gồm tên bài, mô tả, dữ liệu vào, dữ liệu ra và ví dụ để người dùng kiểm tra trước khi lưu
AI sinh test case	Giảng viên, quản trị viên	Bài toán đã có nội dung cơ bản và người dùng có API key AI hợp lệ	Người dùng yêu cầu sinh test case, AI hỗ trợ tạo chiến lược sinh test, chương trình sinh input hoặc lời giải chuẩn. Hệ thống xử lý để tạo bộ dữ liệu kiểm thử	Bộ test case được tạo để người ra đề kiểm tra lại trước khi sử dụng chính thức
AI hỗ trợ đánh giá năng lực và dự đoán dạng đề	Người dùng, giảng viên, quản trị viên	Hệ thống có dữ liệu bài nộp đúng, mã nguồn hoặc thông tin bài toán	Hệ thống sử dụng dữ liệu bài nộp, mã nguồn và dạng đề để AI hỗ trợ nhận xét năng lực hoặc gợi ý nhóm kiến thức của bài toán	Người dùng có thêm thông tin tham khảo về điểm mạnh, điểm yếu và dạng bài cần luyện tập

Xem bảng xếp hạng	Người dùng, giảng viên, quản trị viên	Hệ thống có dữ liệu kết quả luyện tập hoặc kỳ thi	Người dùng truy cập bảng xếp hạng, hệ thống tổng hợp điểm số, thứ hạng và thông tin liên quan	Bảng xếp hạng được hiển thị, hỗ trợ theo dõi kết quả luyện tập hoặc kết quả kỳ thi
-------------------------	---	---	--	---

Bảng 3.3.2: Đặc tả một số Use Case chính

3.4. Thiết kế kiến trúc tổng thể hệ thống

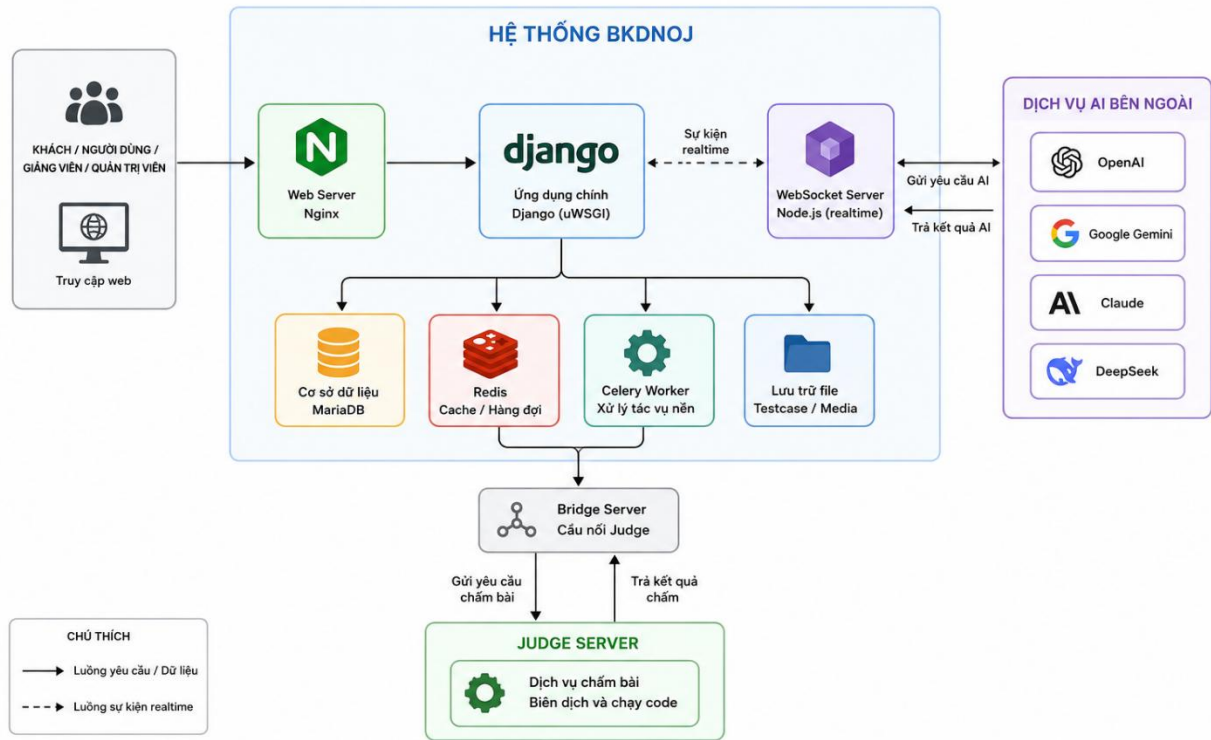
Sau khi xác định các nhóm chức năng và các tác nhân tương tác với hệ thống, bước tiếp theo là thiết kế kiến trúc tổng thể cho BKDNOJ. Kiến trúc hệ thống cần thể hiện được cách các thành phần chính phối hợp với nhau để xử lý các nghiệp vụ như hiển thị giao diện, tiếp nhận bài nộp, chấm bài tự động, chạy thử chương trình, cập nhật kết quả theo thời gian thực và thực hiện các chức năng AI.

BKDNOJ được thiết kế theo kiến trúc web nhiều thành phần. Trong đó, người dùng tương tác với hệ thống thông qua trình duyệt web. Các yêu cầu từ trình duyệt được chuyển đến máy chủ web, sau đó được xử lý bởi ứng dụng chính của hệ thống. Dữ liệu của hệ thống được lưu trữ trong cơ sở dữ liệu, các tác vụ cần xử lý nền được đưa vào hàng đợi, kết quả chấm và trạng thái xử lý được cập nhật lại cho người dùng thông qua cơ chế thời gian thực.

Việc tổ chức hệ thống thành nhiều thành phần riêng biệt giúp BKDNOJ dễ quản lý, dễ mở rộng và thuận tiện hơn trong quá trình triển khai. Mỗi thành phần đảm nhận một vai trò cụ thể, từ xử lý nghiệp vụ, lưu trữ dữ liệu, điều phối bài nộp, chấm bài, chạy tác vụ nền cho đến hỗ trợ các chức năng AI. Cách thiết kế này phù hợp với đặc thù của hệ thống chấm bài trực tuyến, nơi có nhiều tiến trình cần hoạt động đồng thời và cần đảm bảo tính ổn định trong quá trình vận hành.

3.4.1. Kiến trúc tổng thể

Kiến trúc tổng thể của hệ thống BKDNOJ bao gồm các thành phần chính như trình duyệt người dùng, Nginx, Django Site, cơ sở dữ liệu, Redis, Celery, WebSocket, Bridge, máy chấm và dịch vụ AI. Các thành phần này phối hợp với nhau để tạo thành một hệ thống hoàn chỉnh, có khả năng tiếp nhận yêu cầu từ người dùng, xử lý dữ liệu, chấm bài tự động và trả kết quả về giao diện.



Hình 3.4.1: Kiến trúc tổng thể hệ thống BKDNOJ

Trong sơ đồ kiến trúc tổng thể, trình duyệt người dùng là điểm bắt đầu của các thao tác như xem bài tập, chạy thử chương trình, nộp bài, xem kết quả chấm, tham gia kỳ thi và sử dụng các chức năng AI. Các yêu cầu này được gửi đến Nginx, đóng vai trò tiếp nhận và điều hướng request đến các dịch vụ phù hợp trong hệ thống.

Django Site là thành phần xử lý nghiệp vụ chính của BKDNOJ. Thành phần này chịu trách nhiệm quản lý người dùng, bài tập, test case, bài nộp, kỳ thi, kết quả chấm và các chức năng AI. Khi người dùng nộp bài, hệ thống lưu thông tin bài nộp vào cơ sở dữ liệu, sau đó chuyển yêu cầu xử lý đến các thành phần liên quan để thực hiện chấm bài.

Bridge đóng vai trò trung gian giữa hệ thống web và máy chấm. Thành phần này nhận bài nộp từ hệ thống, chuyển đến máy chấm để biên dịch và thực thi với các test case tương ứng. Sau khi quá trình chấm hoàn tất, kết quả được gửi ngược lại hệ thống để lưu trữ và hiển thị cho người dùng.

Máy chấm là thành phần đảm nhận việc biên dịch, thực thi mã nguồn và đánh giá kết quả chương trình. Đây là thành phần quan trọng trong hệ thống Online Judge, vì trực tiếp quyết

định trạng thái chấm của bài nộp như Accepted, Wrong Answer, Time Limit Exceeded, Runtime Error hoặc Compilation Error.

Redis và Celery được sử dụng để hỗ trợ các tác vụ nền và xử lý bất đồng bộ. Những tác vụ không nên xử lý trực tiếp trong request chính, chẳng hạn như các tác vụ AI, sinh test case hoặc một số công việc mất thời gian, có thể được đưa vào hàng đợi để xử lý nền. Điều này giúp giao diện người dùng phản hồi nhanh hơn và hệ thống vận hành ổn định hơn.

WebSocket được sử dụng để cập nhật trạng thái xử lý theo thời gian thực cho người dùng. Nhờ đó, người dùng có thể theo dõi trạng thái bài nộp, kết quả chấm, kết quả chạy thử hoặc trạng thái của các tác vụ đang xử lý mà không cần tải lại trang liên tục.

Dịch vụ AI là thành phần bên ngoài hoặc tích hợp thông qua API, được sử dụng để hỗ trợ các chức năng thông minh của hệ thống như phân tích mã nguồn, hỗ trợ tạo đề bài, sinh test case, đánh giá năng lực người dùng và dự đoán dạng đề. Các chức năng AI không thay thế quy trình chấm bài chính thức, mà đóng vai trò hỗ trợ người học, giảng viên và quản trị viên trong quá trình sử dụng hệ thống.

3.4.2. Vai trò của các thành phần trong hệ thống

Mỗi thành phần trong kiến trúc BKDNOJ đảm nhận một nhiệm vụ riêng nhằm đảm bảo hệ thống hoạt động ổn định và có khả năng mở rộng. Việc phân chia vai trò rõ ràng giúp quá trình thiết kế, triển khai, bảo trì và nâng cấp hệ thống trở nên thuận tiện hơn.

Thành phần	Vai trò
Trình duyệt người dùng	Là giao diện để người dùng tương tác với hệ thống, thực hiện các thao tác như xem bài, chạy thử, nộp bài, xem kết quả và sử dụng chức năng AI.
Nginx	Tiếp nhận request từ người dùng, điều hướng đến các dịch vụ phù hợp và hỗ trợ phục vụ tài nguyên tĩnh của hệ thống.
Django Site	Xử lý nghiệp vụ chính của hệ thống, bao gồm quản lý người dùng, bài tập, test case, bài nộp, kỳ thi, kết quả chấm và các chức năng AI.
Cơ sở dữ liệu	Lưu trữ dữ liệu quan trọng của hệ thống như tài khoản người dùng, bài tập, test case, bài nộp, kết quả chấm, kỳ thi và cấu hình hệ thống.

Redis	Hỗ trợ lưu trữ tạm, hàng đợi và các dữ liệu cần truy xuất nhanh trong quá trình vận hành hệ thống.
Celery	Xử lý các tác vụ nền hoặc tác vụ mất nhiều thời gian, giúp giảm tải cho request chính và cải thiện khả năng phản hồi của hệ thống.
WebSocket	Cập nhật trạng thái bài nộp, kết quả chấm, kết quả chạy thử và trạng thái tác vụ theo thời gian thực cho người dùng.
Bridge	Là thành phần trung gian kết nối giữa Django Site và máy chấm, chuyển bài nộp đến máy chấm và nhận kết quả trả về.
Máy chấm	Biên dịch, thực thi mã nguồn, chạy chương trình với test case và xác định kết quả chấm của bài nộp.
Dịch vụ AI	Hỗ trợ phân tích mã nguồn, tạo đề bài, sinh test case, đánh giá năng lực người dùng và dự đoán dạng đề.

Bảng 3.4.2: Các thành phần trong hệ thống

Từ bảng trên có thể thấy kiến trúc BKDNOJ được thiết kế theo hướng tách biệt các trách nhiệm chính của hệ thống. Thành phần web tập trung xử lý nghiệp vụ và giao diện, máy chấm đảm nhận việc đánh giá mã nguồn, các thành phần nền hỗ trợ xử lý bất đồng bộ, trong khi WebSocket giúp cập nhật trạng thái nhanh chóng cho người dùng. Nhờ đó, hệ thống có thể đáp ứng tốt hơn các yêu cầu về chấm bài tự động, hỗ trợ học tập, tổ chức kỳ thi và mở rộng các chức năng AI.

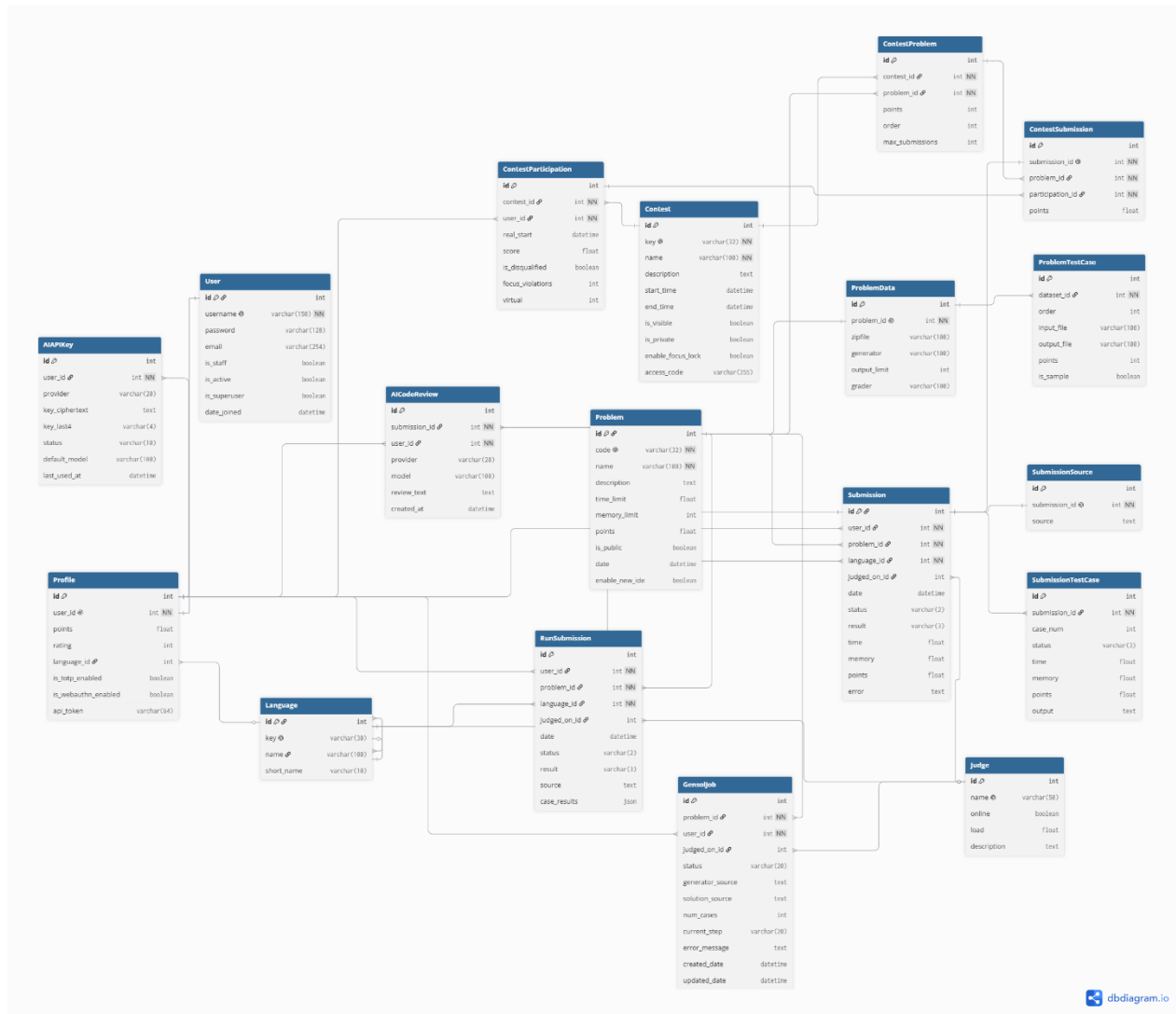
3.5. Thiết kế cơ sở dữ liệu

Cơ sở dữ liệu là thành phần quan trọng trong hệ thống BKDNOJ, chịu trách nhiệm lưu trữ và quản lý toàn bộ dữ liệu phục vụ quá trình vận hành hệ thống. Các dữ liệu này bao gồm thông tin người dùng, bài tập, test case, bài nộp, kết quả chấm, kỳ thi, cấu hình hệ thống, dữ liệu chạy thử chương trình và các thông tin liên quan đến chức năng AI.

Việc thiết kế cơ sở dữ liệu cần đảm bảo dữ liệu được tổ chức rõ ràng, hạn chế trùng lặp, thuận tiện cho việc truy xuất và có khả năng mở rộng khi hệ thống bổ sung thêm chức năng mới. Đối với hệ thống chấm bài trực tuyến, cơ sở dữ liệu không chỉ lưu các thông tin nghiệp vụ thông thường, mà còn phải hỗ trợ các luồng xử lý có tính liên tục như nộp bài, chấm bài tự động, cập nhật kết quả, tổ chức kỳ thi và đánh giá năng lực người dùng.

3.5.1. Sơ đồ cơ sở dữ liệu tổng quát

Sơ đồ cơ sở dữ liệu tổng quát thể hiện các nhóm bảng chính trong hệ thống BKDNOJ và mối quan hệ giữa chúng. Thông qua sơ đồ này, có thể thấy được cách dữ liệu trong hệ thống được tổ chức xoay quanh các thực thể trung tâm như người dùng, bài tập, bài nộp, test case, kỳ thi và kết quả chấm.



Hình 3.5.1: Sơ đồ cơ sở dữ liệu rút gọn của hệ thống BKDNOJ

Trong sơ đồ cơ sở dữ liệu tổng quát, nhóm dữ liệu người dùng đóng vai trò quản lý tài khoản, thông tin cá nhân và quyền hạn của người sử dụng hệ thống. Nhóm dữ liệu bài tập lưu trữ nội dung bài toán, giới hạn thời gian, giới hạn bộ nhớ, dạng đề và các thông tin cấu

hình liên quan. Mỗi bài tập có thể liên kết với nhiều test case để phục vụ quá trình chấm bài tự động.

Nhóm dữ liệu bài nộp là một trong những phần quan trọng nhất của hệ thống Online Judge. Khi người dùng nộp mã nguồn, hệ thống lưu lại thông tin bài nộp, ngôn ngữ lập trình, trạng thái xử lý, kết quả chấm và các dữ liệu liên quan. Bài nộp được liên kết với người dùng, bài tập và kết quả chấm, từ đó hỗ trợ việc theo dõi quá trình luyện tập cũng như thống kê năng lực của người học.

Bên cạnh đó, hệ thống còn có nhóm dữ liệu kỳ thi nhằm phục vụ việc tổ chức các cuộc thi lập trình trực tuyến. Nhóm dữ liệu này quản lý thông tin kỳ thi, danh sách bài trong kỳ thi, người tham gia, thời gian thi, kết quả và bảng xếp hạng. Đối với các kỳ thi có cơ chế khóa tập trung, hệ thống cần lưu thêm thông tin về trạng thái tham gia và số lần vi phạm của người dùng trong quá trình làm bài.

Ngoài các nhóm dữ liệu truyền thống, BKDNOJ còn mở rộng thêm các nhóm dữ liệu phục vụ IDE trực tuyến và chức năng AI. Dữ liệu chạy thử chương trình được tách biệt với bài nộp chính thức để đảm bảo kết quả chạy thử không ảnh hưởng đến điểm số và bảng xếp hạng. Dữ liệu AI được sử dụng để lưu thông tin API key, kết quả phân tích mã nguồn, tác vụ sinh test case, dữ liệu đánh giá năng lực và dự đoán dạng đề.

3.5.2. Các nhóm dữ liệu chính

Dựa trên các chức năng của hệ thống, cơ sở dữ liệu BKDNOJ có thể được chia thành nhiều nhóm dữ liệu chính. Mỗi nhóm đảm nhận một vai trò riêng và liên kết với các nhóm khác để tạo thành một hệ thống dữ liệu hoàn chỉnh.

Nhóm dữ liệu	Mô tả
Dữ liệu người dùng	Lưu thông tin tài khoản, hồ sơ cá nhân, vai trò, quyền hạn và các thông tin liên quan đến quá trình sử dụng hệ thống.
Dữ liệu bài tập	Lưu thông tin bài toán như tên bài, nội dung đề bài, giới hạn thời gian, giới hạn bộ nhớ, điểm số, dạng đề và các cấu hình liên quan.
Dữ liệu test case	Lưu thông tin về các bộ dữ liệu kiểm thử, dữ liệu vào, dữ liệu ra, nhóm test và cấu hình phục vụ quá trình chấm bài.

Dữ liệu bài nộp	Lưu thông tin mã nguồn người dùng nộp, ngôn ngữ lập trình, thời gian nộp, trạng thái xử lý và kết quả chấm.
Dữ liệu kết quả chấm	Lưu kết quả đánh giá bài nộp, bao gồm trạng thái chấm, điểm số, thời gian chạy, bộ nhớ sử dụng và kết quả trên từng test case nếu có.
Dữ liệu chạy thử chương trình	Lưu thông tin các lần chạy thử mã nguồn trong IDE trực tuyến, bao gồm dữ liệu vào, kết quả đầu ra và trạng thái chạy thử.
Dữ liệu kỳ thi	Lưu thông tin kỳ thi, thời gian bắt đầu, thời gian kết thúc, danh sách bài thi, thí sinh tham gia và kết quả trong kỳ thi.
Dữ liệu khóa tập trung	Lưu trạng thái tham gia kỳ thi, số lần vi phạm, thời điểm vi phạm và các thông tin phục vụ việc theo dõi hành vi trong kỳ thi.
Dữ liệu AI	Lưu thông tin liên quan đến các chức năng AI như phân tích mã nguồn, tạo đề bài, sinh test case, đánh giá năng lực và dự đoán dạng đề.
Dữ liệu API key AI	Lưu thông tin API key của người dùng để kết nối với các dịch vụ AI bên ngoài, phục vụ các chức năng AI trong hệ thống.
Dữ liệu xếp hạng	Lưu thông tin điểm số, kết quả luyện tập, kết quả kỳ thi và dữ liệu phục vụ bảng xếp hạng người dùng.
Dữ liệu cấu hình hệ thống	Lưu các thiết lập chung của hệ thống, cấu hình máy chấm, ngôn ngữ lập trình, quyền truy cập và các thông số vận hành khác.

Bảng 3.5.2: Các nhóm dữ liệu chính của BKDNOJ

Việc phân chia cơ sở dữ liệu thành các nhóm như trên giúp hệ thống dễ quản lý và mở rộng. Các nhóm dữ liệu cốt lõi như người dùng, bài tập, test case, bài nộp và kết quả chấm đảm bảo hoạt động chính của hệ thống Online Judge. Trong khi đó, các nhóm dữ liệu mở rộng như chạy thử chương trình, khóa tập trung, AI và đánh giá năng lực giúp BKDNOJ đáp ứng tốt hơn các yêu cầu mới trong học tập lập trình và tổ chức kỳ thi trực tuyến.

3.6. Thiết kế luồng nộp bài và chấm bài

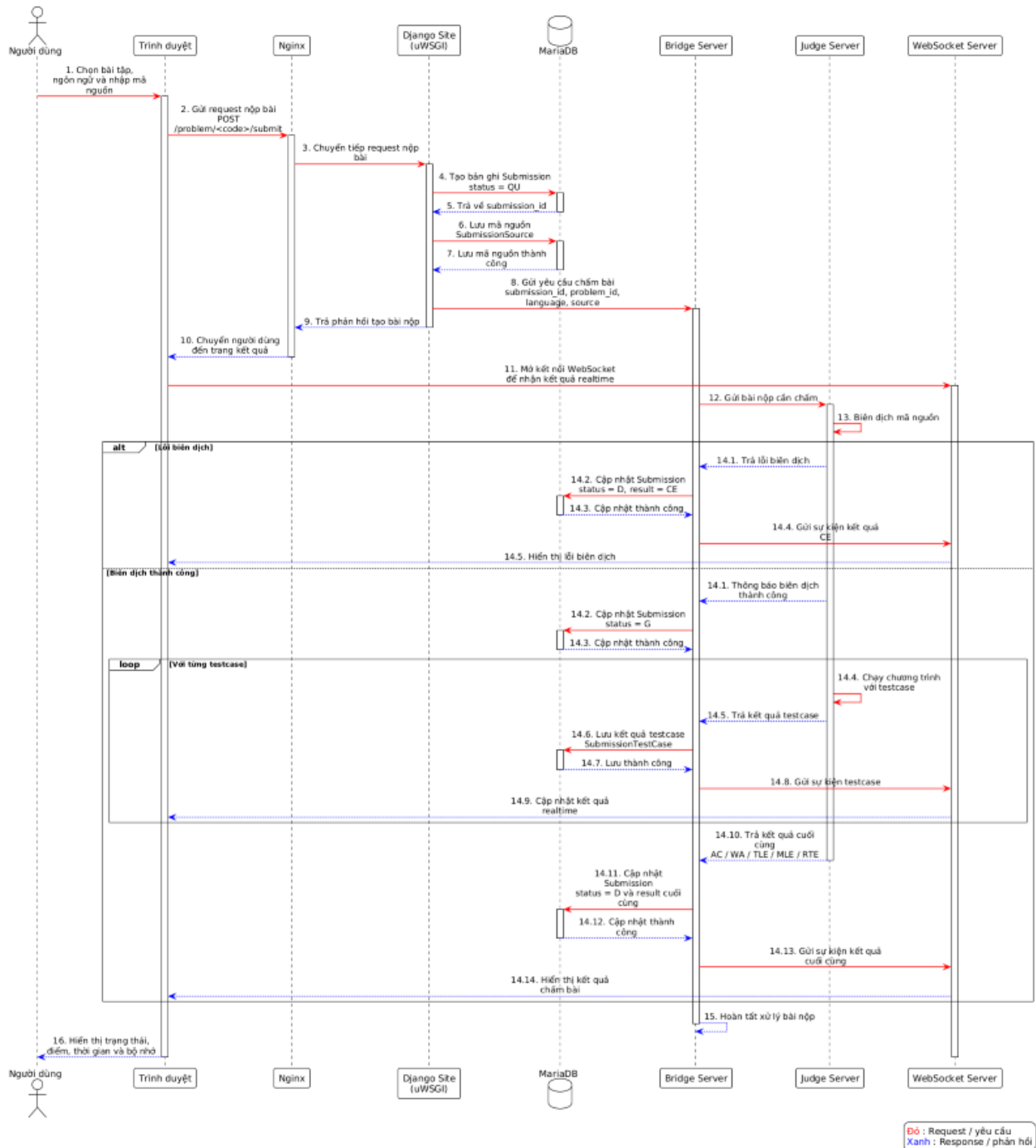
Luồng nộp bài và chấm bài là một trong những luồng xử lý quan trọng nhất của hệ thống BKDNOJ. Đây là luồng thể hiện cách hệ thống tiếp nhận mã nguồn từ người dùng, lưu trữ

thông tin bài nộp, chuyển bài nộp đến máy chấm, thực hiện biên dịch, chạy chương trình với test case và trả kết quả chấm về cho người dùng.

Việc thiết kế luồng nộp bài cần đảm bảo quá trình xử lý diễn ra rõ ràng, có khả năng theo dõi trạng thái và hạn chế ảnh hưởng đến các chức năng khác của hệ thống. Do bài nộp có thể mất thời gian để biên dịch và chấm trên nhiều test case, hệ thống không xử lý toàn bộ quá trình này trực tiếp trong một request ngắn, mà cần phối hợp giữa các thành phần như Django Site, cơ sở dữ liệu, Bridge, máy chấm và WebSocket để cập nhật kết quả cho người dùng.

3.6.1. Luồng nộp bài chính thức

Luồng nộp bài chính thức bắt đầu khi người dùng chọn bài toán, lựa chọn ngôn ngữ lập trình và gửi mã nguồn lên hệ thống. Sau khi nhận bài nộp, Django Site kiểm tra các thông tin cần thiết như người nộp, bài toán, ngôn ngữ lập trình và quyền nộp bài. Nếu dữ liệu hợp lệ, hệ thống tạo bản ghi bài nộp trong cơ sở dữ liệu với trạng thái ban đầu là chờ xử lý.



Hình 3.6.1: Sơ đồ tuần tự của luồng nộp bài chính thức

Sau khi bài nộp được lưu vào cơ sở dữ liệu, hệ thống chuyển thông tin bài nộp đến Bridge để điều phối quá trình chấm. Bridge đóng vai trò trung gian giữa hệ thống web và máy

chấm, có nhiệm vụ gửi mã nguồn, ngôn ngữ lập trình, giới hạn tài nguyên và thông tin test case đến máy chấm phù hợp.

Máy chấm nhận bài nộp và tiến hành biên dịch mã nguồn. Nếu quá trình biên dịch thất bại, bài nộp được trả về trạng thái lỗi biên dịch và không tiếp tục chạy với test case. Nếu biên dịch thành công, chương trình được thực thi lần lượt với các test case của bài toán trong môi trường được kiểm soát về thời gian chạy, bộ nhớ và tài nguyên sử dụng.

Trong quá trình chấm, máy chấm ghi nhận kết quả của từng test case, bao gồm trạng thái đúng sai, thời gian chạy, bộ nhớ sử dụng và các lỗi phát sinh nếu có. Sau khi hoàn tất, kết quả được gửi ngược về hệ thống thông qua Bridge. Django Site cập nhật kết quả vào cơ sở dữ liệu và hiển thị trạng thái cuối cùng của bài nộp cho người dùng.

Đối với người dùng, quá trình nộp bài cần được thể hiện một cách trực quan trên giao diện. Người dùng có thể theo dõi trạng thái bài nộp từ lúc chờ chấm, đang xử lý cho đến khi có kết quả cuối cùng. Việc cập nhật trạng thái có thể được thực hiện thông qua WebSocket, giúp người dùng nhận kết quả nhanh chóng mà không cần tải lại trang liên tục.

3.6.2. Các trạng thái của bài nộp

Trong quá trình nộp bài và chấm bài, mỗi bài nộp trong hệ thống BKDNOJ sẽ trải qua các trạng thái xử lý khác nhau. Các trạng thái này giúp hệ thống xác định bài nộp đang ở bước nào trong quy trình chấm, đồng thời giúp người dùng theo dõi được tiến trình xử lý bài làm của mình trên giao diện.

Trạng thái	Ý nghĩa
Queued	Bài nộp đã được hệ thống tiếp nhận và đưa vào hàng đợi, đang chờ được xử lý bởi hệ thống chấm.
Processing	Bài nộp đang được hệ thống xử lý ban đầu, bao gồm chuẩn bị dữ liệu, kiểm tra thông tin bài nộp và điều phối đến máy chấm phù hợp.
Grading	Bài nộp đang được máy chấm thực hiện chấm, bao gồm biên dịch mã nguồn, chạy chương trình với các test case và ghi nhận kết quả.
Completed	Quá trình chấm bài đã hoàn tất, hệ thống đã có kết quả cuối cùng và cập nhật kết quả cho người dùng.

Internal Error	Có lỗi nội bộ phát sinh trong quá trình xử lý hoặc chấm bài, chẳng hạn lỗi hệ thống, lỗi máy chấm hoặc lỗi kết nối giữa các thành phần.
Compile Error	Mã nguồn của người dùng không biên dịch thành công, do đó bài nộp không thể tiếp tục chạy với các test case.
Aborted	Quá trình chấm bài bị hủy hoặc dừng trước khi hoàn tất, có thể do yêu cầu từ hệ thống, lỗi xử lý hoặc điều kiện bất thường trong quá trình chấm.

Bảng 3.6.2: Trạng thái của bài nộp

Các trạng thái trên phản ánh tiến trình xử lý của bài nộp từ khi được đưa vào hàng đợi cho đến khi hoàn tất hoặc gặp lỗi. Trong đó, Queued, Processing và Grading là các trạng thái trung gian thể hiện bài nộp đang được xử lý. Completed là trạng thái kết thúc khi hệ thống đã có kết quả chấm cuối cùng. Các trạng thái Internal Error, Compile Error và Aborted thể hiện những trường hợp bài nộp không hoàn tất theo quy trình chấm thông thường.

Cần phân biệt giữa trạng thái xử lý của bài nộp và kết quả chấm của bài nộp. Trạng thái xử lý cho biết bài nộp đang ở bước nào trong hệ thống, còn kết quả chấm cho biết lời giải đúng hay sai sau khi chạy với test case. Khi bài nộp ở trạng thái Completed, hệ thống có thể hiển thị kết quả chấm như Accepted, Wrong Answer, Time Limit Exceeded, Memory Limit Exceeded hoặc Runtime Error tùy theo kết quả thực thi chương trình.

3.6.3. Thiết kế chế độ chấm điểm

Bên cạnh việc xác định kết quả đúng hoặc sai của bài nộp, hệ thống BKDNOJ còn hỗ trợ nhiều chế độ chấm điểm khác nhau nhằm phù hợp với nhiều hình thức luyện tập và tổ chức kỳ thi lập trình. Chế độ chấm điểm của mỗi bài tập được xác định thông qua trường `scoring_mode` trong bảng Problem. Trường này quy định cách hệ thống tính điểm dựa trên kết quả của từng test case, cách tổ chức test case theo batch và cách tổng hợp điểm cuối cùng của bài nộp.

Chế độ chấm điểm	Giá trị	Nguyên tắc	Trường hợp sử dụng
------------------	---------	------------	--------------------

Short Circuit	short_circuit	Chấm toàn phần, nếu sai một test case thì dừng chấm và bài nộp nhận 0 điểm.	Phù hợp với kỳ thi dạng ICPC, nơi bài nộp chỉ được chấp nhận khi đúng toàn bộ test case.
Partial Batch	partial_batch	Chấm theo nhóm test hoặc subtask. Một batch chỉ có điểm khi tất cả test case trong batch đều đúng.	Phù hợp với bài toán có chia subtask, thường dùng trong các kỳ thi dạng IOI.
Partial Testcase	partial_testcase	Chấm theo từng test case. Mỗi test case đúng đều được tính điểm riêng, không phụ thuộc vào các test case khác trong cùng batch.	Phù hợp khi muốn người dùng nhận điểm thành phần ngay cả khi chưa giải đúng toàn bộ một nhóm test.

Bảng 3.6.3.1: Các chế độ chấm điểm

Ở chế độ Short Circuit, hệ thống áp dụng nguyên tắc chấm toàn phần. Điều này có nghĩa là bài nộp chỉ đạt điểm tối đa khi vượt qua toàn bộ test case của bài toán. Nếu một test case cho kết quả sai, chương trình chạy quá thời gian, vượt giới hạn bộ nhớ hoặc phát sinh lỗi khi thực thi, hệ thống có thể dừng việc chấm các test case còn lại và bài nộp nhận 0 điểm. Cách chấm này phù hợp với các kỳ thi lập trình kiểu ICPC, trong đó kết quả bài nộp thường được đánh giá theo tiêu chí đúng hoàn toàn hoặc chưa được chấp nhận.

Công thức tính điểm của chế độ short_circuit có thể được biểu diễn như sau:

$$\text{Điểm} = \begin{cases} P_{\max}, & \text{nếu đúng tất cả test case} \\ 0, & \text{nếu sai bất kỳ test case nào} \end{cases}$$

Trong đó, $P_{\max}$ là điểm tối đa của bài tập.

Ở chế độ Partial Batch, các test case được tổ chức thành từng batch, tương ứng với nhóm test hoặc subtask. Mỗi batch có điểm riêng và chỉ được tính điểm khi toàn bộ test case trong batch đó đều đúng. Nếu có bất kỳ test case nào trong batch bị sai, toàn bộ batch sẽ nhận 0 điểm. Các batch được tính điểm độc lập với nhau, do đó người dùng vẫn có thể đạt điểm ở những batch đã vượt qua, ngay cả khi sai ở các batch khác.

Với mỗi batch b , điểm của batch được xác định dựa trên điểm nhỏ nhất trong các test case thuộc batch đó:

$$P_b = \min points_t \in b$$

Trong đó P_b là điểm nhận được từ batch b và $points_t$ là điểm nhận được từ test case t .

Tổng điểm của bài nộp của bài tập như sau:

$$\text{Tổng điểm} = \sum P_b$$

Cách lấy giá trị nhỏ nhất giúp đảm bảo rằng nếu một test case trong batch sai, điểm của toàn bộ batch sẽ bằng 0. Đây là cơ chế phù hợp với các bài toán có chia subtask, trong đó mỗi nhóm test thường đại diện cho một mức ràng buộc hoặc một mức độ khó nhất định.

Ở chế độ Partial Testcase, hệ thống tính điểm chi tiết hơn theo từng test case. Mỗi test case đúng đều được ghi nhận điểm riêng, không yêu cầu toàn bộ test case trong cùng một batch phải đúng. Điều này giúp người dùng có thể nhận điểm thành phần dựa trên số lượng test case vượt qua. Chế độ này phù hợp với những bài toán hoặc kỳ thi muốn đánh giá mức độ hoàn thiện của lời giải theo từng phần nhỏ hơn.

Với mỗi test case t trong batch b , hệ số điểm của test case được chuẩn hóa theo công thức:

$$c_t = \frac{D_t}{P_t}$$

Trong đó D_t là điểm nhận được từ test case và P_t là điểm của test case t và P_t được mặc định bằng 1.

Điểm của batch được tính dựa trên trung bình hệ số đúng của các test case trong batch:

$$\text{Điểm } b = \frac{(\sum c_t \in b)}{|b|} \times P_b$$

Trong đó $|b|$ là số lượng test case trong batch b và P_b là điểm của batch b .

Tổng điểm cuối cùng của bài tập:

$$\text{Tổng điểm} = \sum P_b$$

Trong quá trình chấm điểm, hệ thống sử dụng trường batch trong bảng SubmissionTestCase để xác định test case thuộc nhóm nào. Nếu giá trị batch là NULL, test case được xem là test

độc lập và điểm có thể được tính trực tiếp. Nếu batch có giá trị là số nguyên như 1, 2, 3,... thì test case thuộc về batch hoặc subtask tương ứng.

Giá trị batch	Ý nghĩa
NULL	Test case độc lập, không thuộc batch nào.
Số nguyên 1, 2, 3,...	Test case thuộc batch hoặc subtask tương ứng.

Bảng 3.6.3.2: Bảng giá trị batch

Cơ chế batch giúp hệ thống hỗ trợ việc chia bài toán thành nhiều nhóm test có độ khó hoặc ràng buộc khác nhau. Trong một số trường hợp, các batch có thể có quan hệ phụ thuộc với nhau. Nếu batch trước chưa đạt yêu cầu, các batch phụ thuộc phía sau có thể bị bỏ qua nhằm giảm thời gian chấm không cần thiết. Tuy nhiên, ở chế độ `partial_testcase`, cơ chế phụ thuộc batch không được áp dụng, vì mục tiêu của chế độ này là chấm đầy đủ từng test case để tính điểm thành phần cho người dùng.

Quy trình chấm điểm một bài nộp bắt đầu khi người dùng gửi mã nguồn lên hệ thống. Django Site tiếp nhận bài nộp, tạo bản ghi Submission và chuyển thông tin bài nộp đến Bridge. Bridge tiếp tục chuyển bài nộp sang máy chấm để thực hiện biên dịch và chạy chương trình với các test case tương ứng.

Khi máy chấm bắt đầu xử lý, hệ thống khởi tạo lại dữ liệu chấm của bài nộp, xóa các bản ghi SubmissionTestCase cũ nếu có và chuẩn bị trạng thái batch. Sau đó, máy chấm chạy mã nguồn với từng test case và trả về kết quả như Accepted, Wrong Answer, Time Limit Exceeded, Memory Limit Exceeded hoặc Runtime Error. Mỗi kết quả test case được lưu lại thành một bản ghi SubmissionTestCase, bao gồm trạng thái, thời gian chạy, bộ nhớ sử dụng và điểm của test case.

Cuối cùng, hệ thống cập nhật điểm số, trạng thái hoàn tất của bài nộp và lưu kết quả vào cơ sở dữ liệu. Đồng thời, hệ thống gửi sự kiện thông qua WebSocket để giao diện người dùng có thể cập nhật kết quả chấm theo thời gian thực. Nhờ thiết kế này, BKDNOJ có thể hỗ trợ nhiều cách tính điểm khác nhau, phù hợp với cả luyện tập thông thường, kỳ thi dạng ICPC và kỳ thi có chia subtask theo kiểu IOI.

3.7. Thiết kế IDE trực tuyến và luồng chạy thử

Bên cạnh luồng nộp bài chính thức, hệ thống BKDNOJ được thiết kế thêm IDE trực tuyến nhằm hỗ trợ người dùng viết mã và kiểm tra chương trình trực tiếp trên trình duyệt. Đây là một chức năng quan trọng giúp người học có thể thử nghiệm lời giải nhanh chóng trước khi quyết định nộp bài chính thức lên hệ thống chấm.

Khác với bài nộp chính thức, luồng chạy thử không nhằm mục đích tính điểm hoặc cập nhật bảng xếp hạng. Thay vào đó, chức năng này phục vụ quá trình kiểm tra nhanh mã nguồn với các test case mẫu của bài toán. Việc tách riêng luồng chạy thử khỏi luồng nộp bài chính thức giúp hệ thống đảm bảo dữ liệu chấm bài, điểm số và kết quả thi không bị ảnh hưởng bởi các lần chạy thử của người dùng.

IDE trực tuyến được thiết kế theo hướng tích hợp trực tiếp vào trang làm bài. Người dùng có thể đọc đề, viết mã, chạy thử và nộp bài trên cùng một giao diện. Thiết kế này giúp giảm thao tác chuyển đổi giữa nhiều công cụ khác nhau, đồng thời hỗ trợ tốt hơn cho các kỳ thi có cơ chế khóa tập trung, nơi người dùng cần làm bài trong phạm vi giao diện của hệ thống.

3.7.1. Mục tiêu của IDE trực tuyến

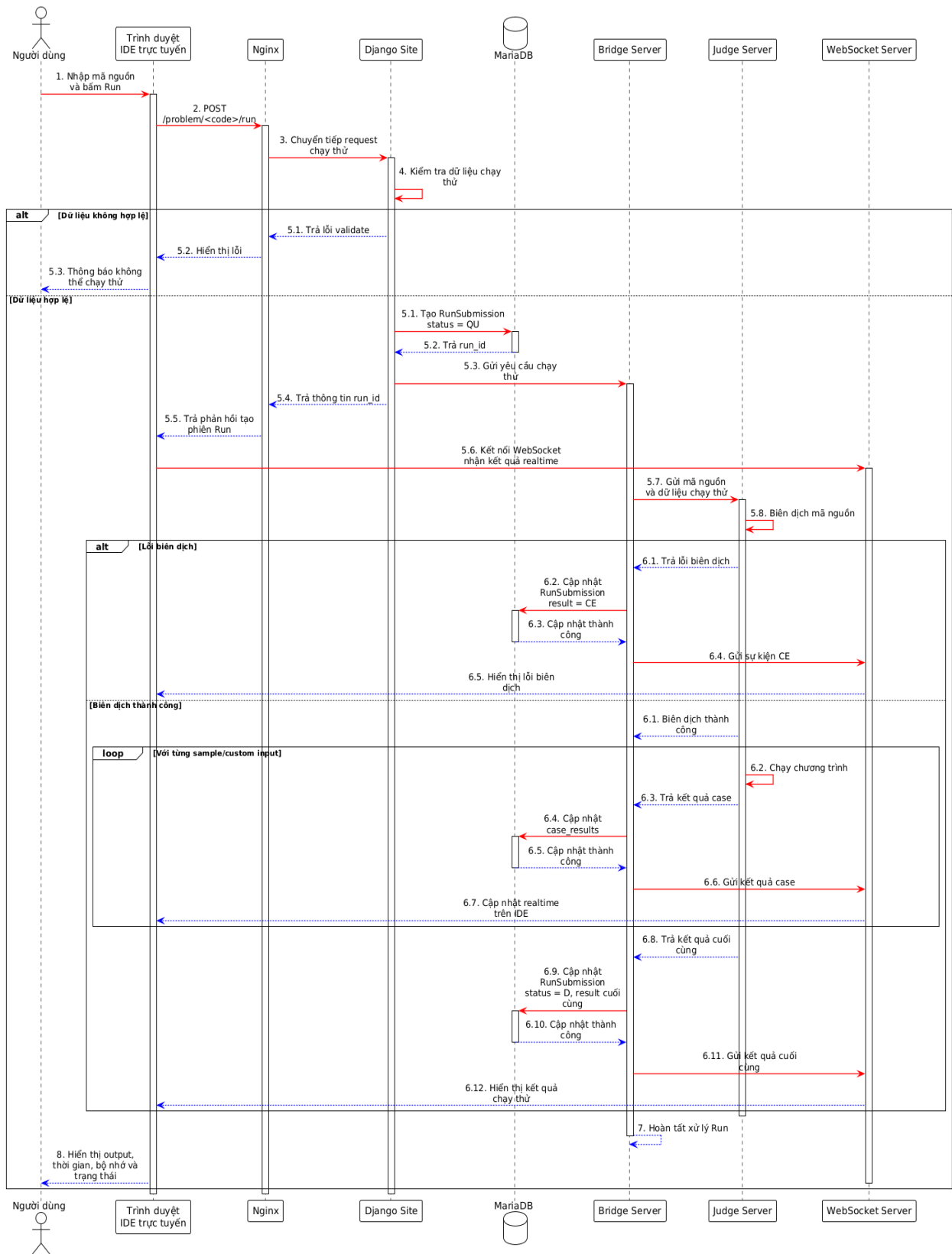
Mục tiêu chính của IDE trực tuyến là cung cấp cho người dùng một môi trường viết mã thuận tiện ngay trên hệ thống BKDNOJ. Thay vì phải sử dụng trình soạn thảo hoặc công cụ lập trình bên ngoài, người dùng có thể trực tiếp viết lời giải trong giao diện bài toán, lựa chọn ngôn ngữ lập trình và thực hiện chạy thử chương trình.

IDE trực tuyến giúp cải thiện trải nghiệm học tập và luyện tập lập trình. Người dùng có thể nhanh chóng kiểm tra chương trình với test case mẫu, phát hiện lỗi biên dịch, lỗi thực thi hoặc kết quả sai trước khi gửi bài nộp chính thức. Điều này giúp quá trình làm bài trở nên liền mạch hơn, đặc biệt đối với người mới học lập trình hoặc người dùng cần kiểm tra nhanh ý tưởng giải bài.

Bên cạnh đó, IDE trực tuyến còn có ý nghĩa quan trọng trong bối cảnh tổ chức kỳ thi trực tuyến. Khi kết hợp với cơ chế khóa tập trung trong kỳ thi, hệ thống có thể cung cấp đầy đủ môi trường làm bài ngay trong trình duyệt, bao gồm đọc đề, viết mã, chạy thử và nộp bài. Nhờ vậy, người dùng không cần rời khỏi giao diện thi để sử dụng công cụ bên ngoài, góp phần hỗ trợ hạn chế gian lận trong quá trình làm bài.

3.7.2. Luồng chạy thử chương trình

Luồng chạy thử chương trình được thiết kế tách biệt với luồng nộp bài chính thức. Khi người dùng viết mã trong IDE trực tuyến và chọn chức năng chạy thử, hệ thống sẽ tạo một yêu cầu chạy thử riêng thay vì tạo bài nộp chính thức. Yêu cầu này được lưu và xử lý thông qua dữ liệu chạy thử, không ghi nhận vào bảng bài nộp chính thức và không tham gia vào quá trình tính điểm.



Hình 3.7.2: Sơ đồ tuần tự của luồng chạy thử chương trình

Quy trình chạy thử bắt đầu từ giao diện người dùng. Người dùng nhập hoặc chỉnh sửa mã nguồn trong IDE, chọn ngôn ngữ lập trình và nhấn nút chạy thử. Sau đó, Django Site tiếp nhận yêu cầu, tạo bản ghi chạy thử tương ứng và gửi yêu cầu xử lý đến thành phần chấm. Máy chấm biên dịch mã nguồn và thực thi chương trình với test case mẫu của bài toán.

Sau khi quá trình chạy thử hoàn tất, hệ thống nhận kết quả trả về, bao gồm trạng thái chạy thử, thông tin lỗi nếu có, thời gian chạy, bộ nhớ sử dụng và kết quả đầu ra của chương trình. Kết quả này được cập nhật lại cho người dùng trên giao diện IDE, giúp người dùng biết chương trình có chạy đúng với test case mẫu hay không.

Điểm quan trọng của luồng chạy thử là dữ liệu chạy thử được tách riêng khỏi dữ liệu bài nộp chính thức. Các lần chạy thử không làm thay đổi điểm số của người dùng, không ảnh hưởng đến bảng xếp hạng và không được xem là một bài nộp trong quá trình chấm bài chính thức. Cách thiết kế này giúp người dùng có thể kiểm tra mã nguồn nhiều lần mà không làm sai lệch kết quả học tập hoặc kết quả kỳ thi.

3.7.3. Phân biệt chạy thử và nộp bài chính thức

Mặc dù chạy thử và nộp bài chính thức đều liên quan đến việc biên dịch và thực thi mã nguồn, hai luồng này có mục đích và cách xử lý khác nhau. Chạy thử được sử dụng để hỗ trợ người dùng kiểm tra nhanh chương trình trong quá trình làm bài, còn nộp bài chính thức được sử dụng để đánh giá lời giải và tính kết quả cho người dùng.

Tiêu chí	Chạy thử chương trình	Nộp bài chính thức
Mục đích	Giúp người dùng kiểm tra nhanh mã nguồn trước khi nộp bài.	Đánh giá lời giải chính thức của người dùng.
Dữ liệu lưu trữ	Lưu vào dữ liệu chạy thử, tách biệt với bài nộp chính thức.	Lưu vào dữ liệu bài nộp chính thức của hệ thống.

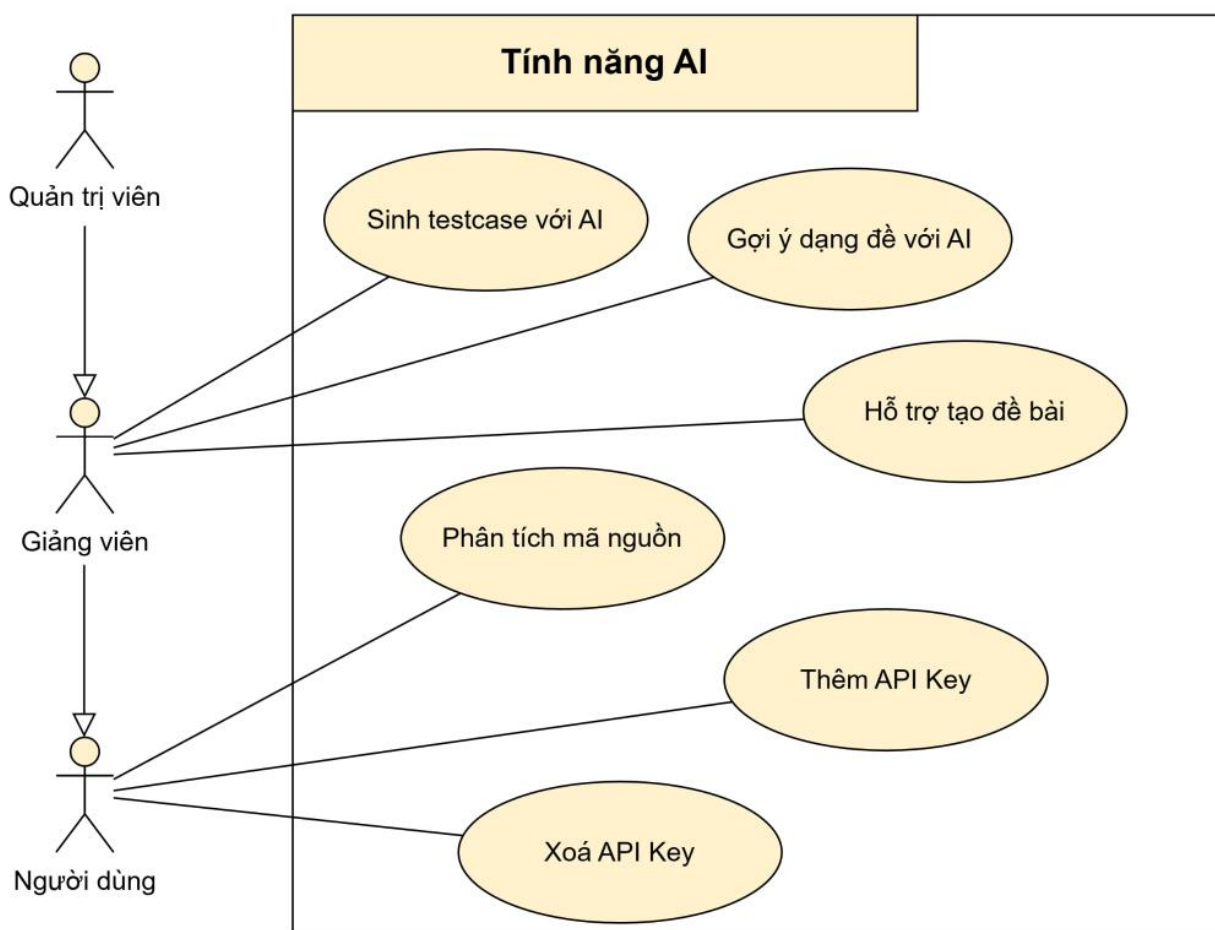
Bảng dữ liệu	Sử dụng bảng riêng cho chạy thử như RunSubmission.	Sử dụng bảng bài nộp chính thức như Submission.
Test case sử dụng	Chạy với test case mẫu của bài toán.	Chạy với bộ test case chính thức của bài toán.
Ảnh hưởng đến điểm số	Không ảnh hưởng đến điểm số.	Có ảnh hưởng đến điểm số nếu bài toán hoặc kỳ thi có tính điểm.
Ảnh hưởng đến bảng xếp hạng	Không ảnh hưởng đến bảng xếp hạng.	Có thể ảnh hưởng đến bảng xếp hạng trong luyện tập hoặc kỳ thi.
Vai trò trong kỳ thi	Hỗ trợ thí sinh kiểm tra mã ngay trong giao diện thi.	Là căn cứ để xác định kết quả làm bài của thí sinh.

Bảng 3.7.3: Phân biệt giữa chạy thử và nộp bài chính thức

Việc phân biệt rõ hai luồng xử lý này giúp hệ thống BKDNOJ vừa hỗ trợ tốt trải nghiệm làm bài của người dùng, vừa đảm bảo tính chính xác của dữ liệu chấm bài chính thức. Người dùng có thể sử dụng IDE trực tuyến để kiểm tra và hoàn thiện lời giải, sau đó mới gửi bài nộp chính thức để hệ thống chấm điểm. Đây là thiết kế phù hợp với định hướng xây dựng BKDNOJ thành một nền tảng hỗ trợ học tập, luyện tập và tổ chức kỳ thi lập trình trực tuyến.

3.8. Thiết kế các chức năng AI

Bên cạnh các chức năng cơ bản của một hệ thống chấm bài trực tuyến, BKDNOJ được thiết kế mở rộng thêm các chức năng AI nhằm hỗ trợ người học, giảng viên và quản trị viên trong quá trình sử dụng hệ thống. Các chức năng AI không thay thế quy trình chấm bài chính thức, mà đóng vai trò như công cụ hỗ trợ phân tích, gợi ý, tự động hóa một phần công việc và cung cấp thêm thông tin tham khảo.



Hình 3.8: Sơ đồ các chức năng AI

Các chức năng AI trong hệ thống bao gồm quản lý API key AI, phân tích mã nguồn, hỗ trợ tạo đề bài, sinh test case, đánh giá năng lực người dùng và dự đoán dạng đề. Mỗi chức năng được thiết kế thành một luồng xử lý riêng, có dữ liệu đầu vào, quá trình xử lý và kết quả đầu ra rõ ràng. Việc tách các chức năng AI khỏi luồng chấm bài chính thức giúp hệ thống dễ quản lý hơn, đồng thời hạn chế ảnh hưởng đến kết quả chấm bài và điểm số của người dùng.

3.8.1. Quản lý API key AI

Để sử dụng các chức năng AI, BKDNOJ cho phép mỗi người dùng tự thêm và quản lý API key của nhà cung cấp AI mà mình sở hữu. Hệ thống hỗ trợ bốn nhà cung cấp: OpenAI, Google Gemini, Anthropic Claude và DeepSeek. Mỗi người dùng chỉ lưu được tối đa một API key cho mỗi nhà cung cấp.

Khi người dùng thêm API key, giá trị khóa được mã hóa ngay bằng thuật toán Fernet trước khi lưu vào cơ sở dữ liệu. Khóa gốc (plaintext) không được lưu lại và bị xóa khỏi bộ nhớ ngay sau khi mã hóa xong. Giao diện chỉ hiển thị bốn ký tự cuối của khóa để người dùng nhận biết. Giá trị mã hóa chỉ được giải mã tạm thời khi thực sự cần gọi API của nhà cung cấp AI, sau đó bị loại bỏ ngay lập tức.

Sau khi thêm key, trạng thái ban đầu là **Pending** (chờ xác nhận). Người dùng cần chọn model cụ thể và nhấn **Test** để hệ thống gửi một yêu cầu thử nghiệm đến API của nhà cung cấp. Kết quả kiểm tra sẽ cập nhật trạng thái thành **Verified** (hợp lệ) hoặc **Failed** (lỗi, kèm thông báo nguyên nhân như sai khóa, không có quyền, model không tồn tại, hoặc bị giới hạn tốc độ). Mỗi lần kiểm tra được ghi vào nhật ký bao gồm thời gian, model đã kiểm tra, kết quả và thời gian phản hồi.

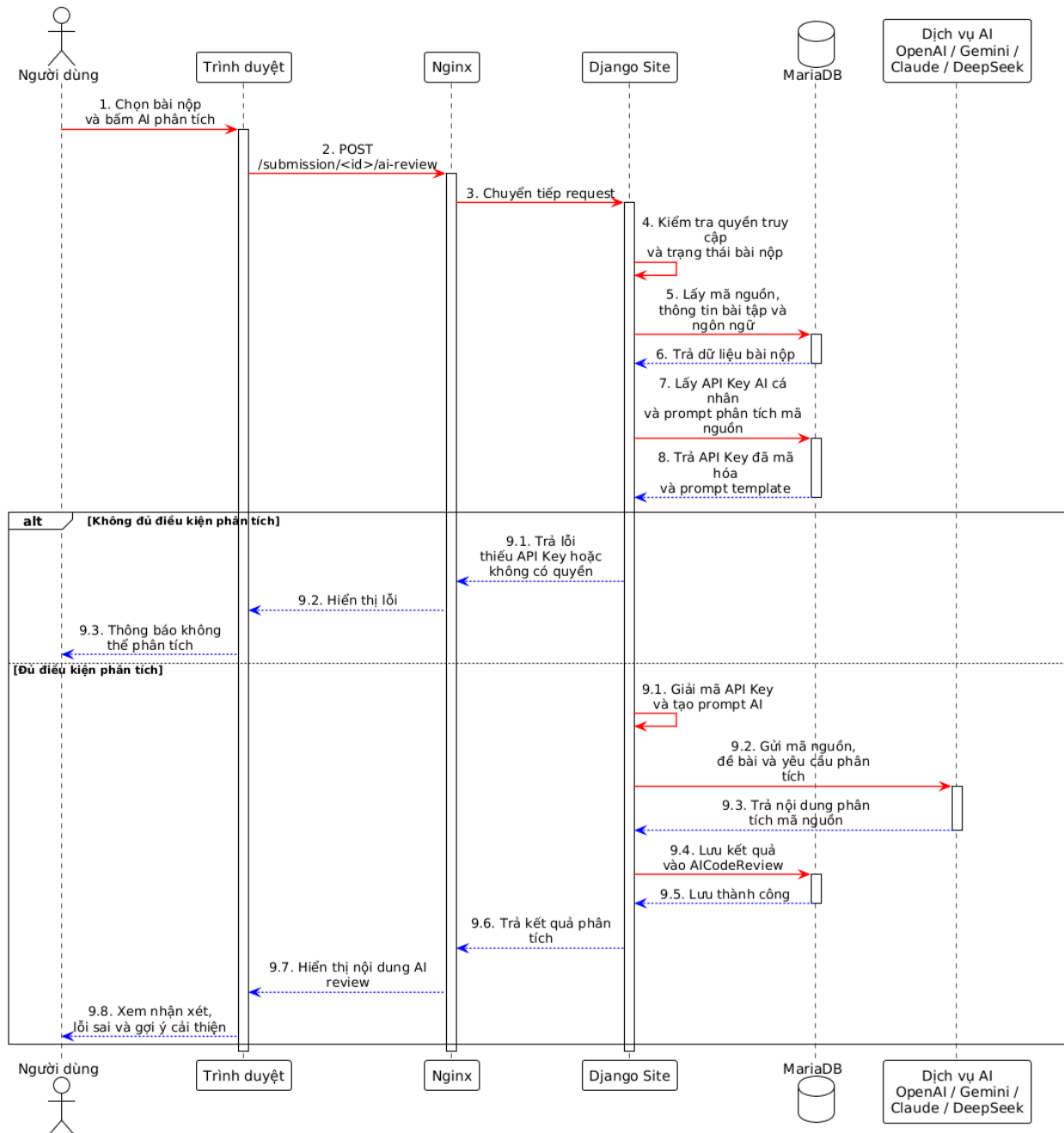
Thành phần	Mô tả
Người sở hữu	Người dùng tạo và quản lý API key; mỗi người dùng tối đa một key/nhà cung cấp.
Nhà cung cấp AI	OpenAI, Google Gemini, Anthropic Claude, DeepSeek.
Khóa truy cập	Mã hóa bằng Fernet (dẫn xuất từ SECRET_KEY); giao diện chỉ hiển thị 4 ký tự cuối.
Model mặc định	Model được chọn khi thêm key; người dùng có thể thay đổi khi kiểm tra.
Kiểm tra kết nối	Gửi prompt thử nghiệm đến API nhà cung cấp; ghi nhật ký kết quả và thời gian phản hồi.
Trạng thái	Ba trạng thái: Pending (mới thêm), Verified (đã kiểm tra thành công), Failed (lỗi kèm mô tả).

Bảng 3.8.1: Quản lý API key

Khi một chức năng AI được kích hoạt, hệ thống tra cứu API key của người dùng theo nhà cung cấp tương ứng, giải mã tạm thời và gửi yêu cầu, đồng thời cập nhật thời điểm sử dụng cuối.

3.8.2. AI phân tích mã nguồn

Chức năng AI phân tích mã nguồn cho phép người dùng yêu cầu nhận xét tự động từ AI về bài nộp của mình. Người dùng chọn nhà cung cấp AI và model, hệ thống sẽ gửi mã nguồn của bài nộp cùng một prompt phân tích có cấu trúc đến dịch vụ AI. API key được giải mã tạm thời ngay lúc gọi và bị xóa khỏi bộ nhớ ngay sau đó.



Hình 3.8.2: Sơ đồ tuần tự của AI phân tích mã nguồn

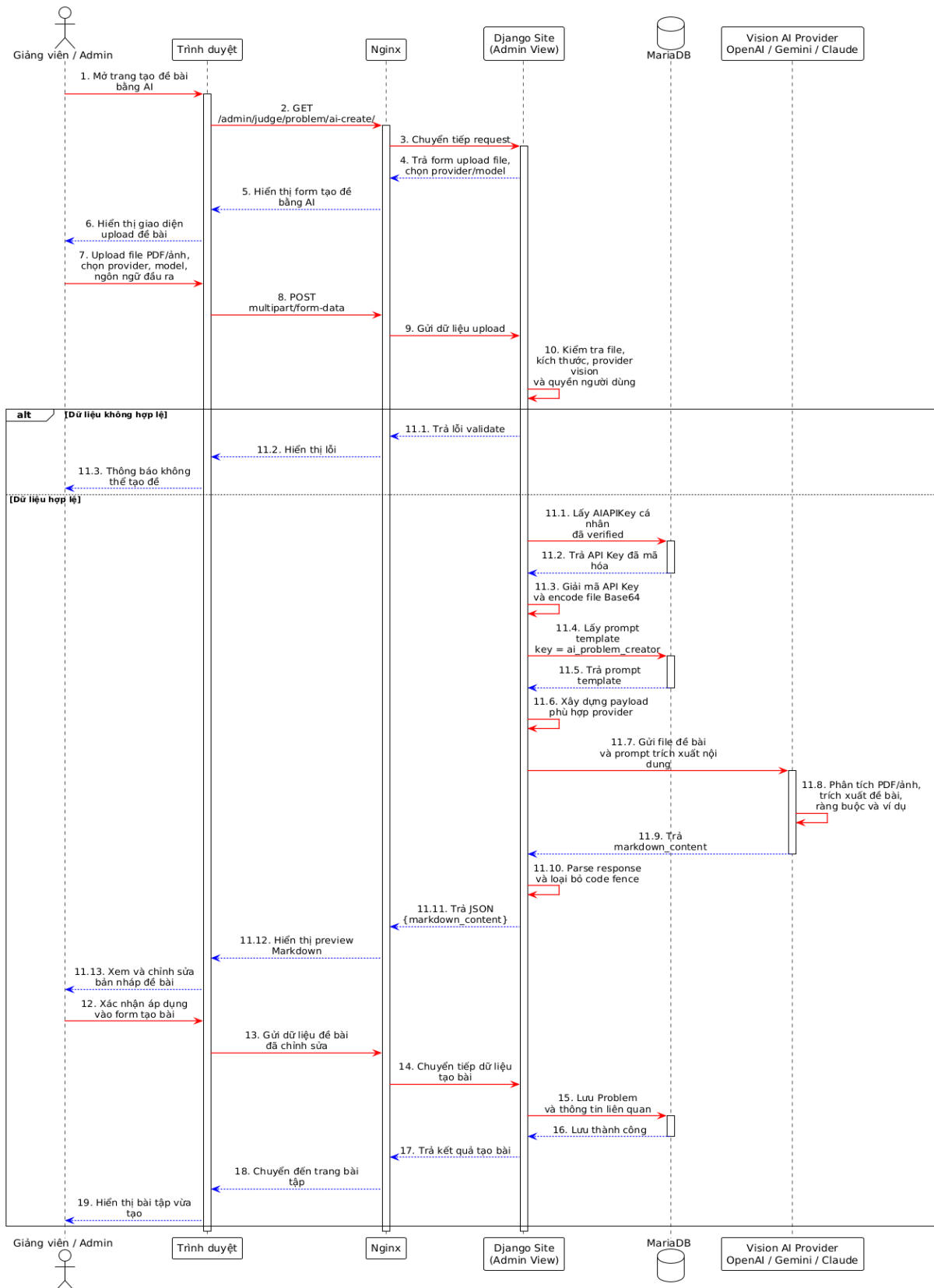
Prompt phân tích bao gồm tên bài toán, ngôn ngữ lập trình, kết quả và điểm của bài nộp. AI được yêu cầu trả lời theo năm mục cố định: thuật toán và cấu trúc dữ liệu sử dụng, luồng thực thi, độ phức tạp thời gian, độ phức tạp không gian, và chất lượng mã nguồn. Kết quả trả về dưới dạng văn bản thuần túy, không sử dụng Markdown hay HTML.

Ngoài nội dung phân tích, AI còn được yêu cầu trả về danh sách tag dạng bài ở cuối phản hồi theo định dạng TAGS: tag1, tag2, Hệ thống trích xuất phần tag này, đối chiếu với danh sách ProblemType trong cơ sở dữ liệu và lưu vào bảng UserProblemTag gắn với bài nộp đó. Phần tag bị loại bỏ trước khi lưu nội dung phân tích vào bảng AICodeReview. Kết quả phân tích được lưu lại để lần xem sau không cần gọi AI lại.

Chỉ người dùng sở hữu bài nộp mới có thể yêu cầu phân tích. Ngôn ngữ đầu ra có thể được chọn (mặc định là tiếng Việt). Bài nộp dạng file-only hoặc không có mã nguồn sẽ bị từ chối.

3.8.3. AI hỗ trợ tạo đề bài

Chức năng AI hỗ trợ tạo đề bài cho phép người ra đề tải lên file ảnh (PNG, JPG, WebP) chứa nội dung bài toán để AI trích xuất và chuyển đổi thành văn bản Markdown theo cấu trúc chuẩn của hệ thống. Chức năng này chỉ hỗ trợ các nhà cung cấp có khả năng nhận ảnh, gồm OpenAI, Google Gemini và Anthropic Claude (DeepSeek không được hỗ trợ).



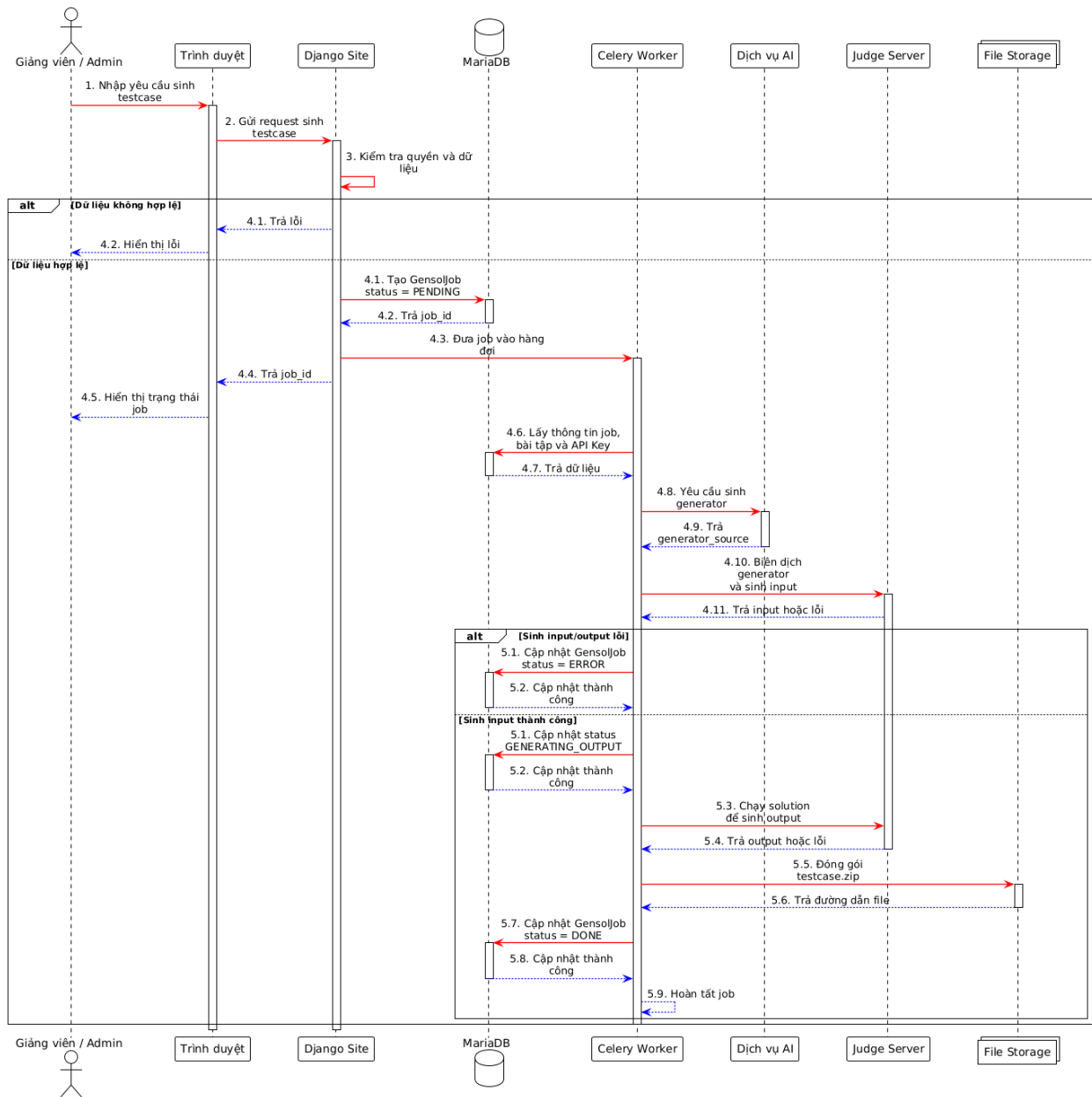
Hình 3.8.3: Sơ đồ tuần tự AI hỗ trợ tạo đề bài

File được đọc, mã hóa base64 và đưa vào payload gửi đến API của nhà cung cấp. AI được yêu cầu trả về nội dung theo cấu trúc cố định gồm: phần mô tả bài toán, định dạng Input, định dạng Output, bảng Subtask (nếu có) và các ví dụ minh họa. Kết quả trả về được điền sẵn vào form tạo bài toán để người dùng kiểm tra và chỉnh sửa trước khi lưu.

Người dùng vẫn phải tự cấu hình các thông số kỹ thuật như giới hạn thời gian, giới hạn bộ nhớ, điểm số, quyền truy cập và thiết lập chấm. Kích thước file tối đa mặc định là 10 MB.

3.8.4. AI sinh test case

Chức năng AI sinh test case hỗ trợ người ra đề xây dựng bộ dữ liệu kiểm thử tự động. Thay vì AI sinh trực tiếp các file input/output, hệ thống dùng AI để tạo ra chương trình sinh input viết bằng C++, sau đó hệ thống máy chấm sẽ biên dịch và chạy chương trình đó để sinh ra dữ liệu thực tế.



Hình 3.8.4: Sơ đồ tuần tự của AI sinh test case

AI được cung cấp nội dung bài toán và số lượng test case cần sinh (tối đa 50). Chương trình sinh do AI tạo ra nhận một số nguyên T từ stdin (chỉ số test case, bắt đầu từ 1), dùng T làm seed ngẫu nhiên và xuất đúng một test input ra stdout. Nếu bài toán có subtask, AI phân bổ test case theo tỉ lệ điểm của từng subtask và đẩy ràng buộc lên mức tối đa của subtask đó.

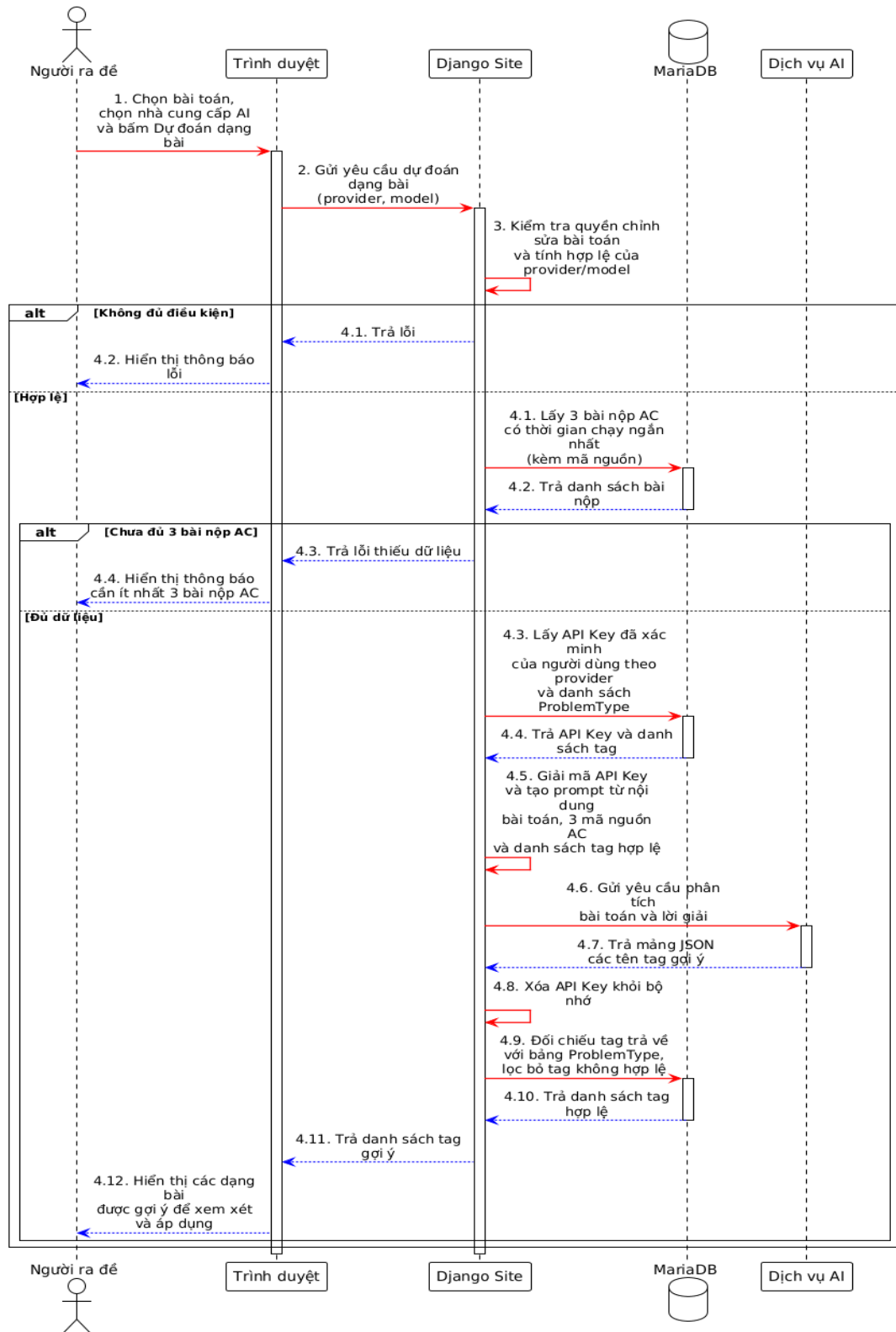
Ngoài chương trình sinh, người dùng cũng cần cung cấp chương trình lời giải chuẩn (viết bằng bất kỳ ngôn ngữ nào). Hệ thống chạy từng bước: sinh input bằng chương trình generator, chạy lời giải chuẩn với input đó để tạo output tương ứng, rồi đóng gói thành bộ test case và lưu vào bài toán. Nếu bài toán đã có test case, hệ thống yêu cầu xác nhận ghi đè trước khi tiến hành.

Thành phần	Vai trò
Nội dung bài toán	Cung cấp ràng buộc, định dạng input/output, thông tin subtask
AI	Tạo chương trình sinh input dạng C++ dựa trên mô tả bài toán
Chương trình sinh input	Nhận chỉ số test T, xuất một test input; chạy N lần để tạo N test case
Lời giải chuẩn	Do người ra đề cung cấp; hệ thống chạy để tạo output tương ứng
Hệ thống máy chấm	Biên dịch và thực thi cả hai chương trình trên, thu file input và output
Người ra đề	Kiểm tra kết quả và xác nhận trước khi lưu vào bài toán

Bảng 3.8.4: Thành phần và vai trò trong AI sinh test case

3.8.5. AI gợi ý dạng bài

Chức năng AI thứ tư trong BKDNOJ là gợi ý dạng bài cho bài toán dưới góc nhìn của quản trị viên. Chức năng này chỉ dành cho người có quyền chỉnh sửa bài toán.



Hình 3.8.5: Gợi ý dạng bài với AI

Hệ thống lấy tối đa 3 bài nộp đúng (chấp nhận được) có thời gian chạy ngắn nhất của bài toán, trích xuất mã nguồn và kết hợp với nội dung đề bài. AI phân tích tổng thể và trả về một mảng JSON chứa danh sách tên tag. Hệ thống đối chiếu với bảng ProblemType trong cơ sở dữ liệu và áp dụng các tag hợp lệ vào bài toán. Nếu bài toán chưa có đủ 3 bài nộp đúng, tính năng từ chối và thông báo thiếu dữ liệu.

Dữ liệu đầu vào	Mục đích
Nội dung bài toán	Cung cấp ngữ cảnh về yêu cầu và ràng buộc
3 bài nộp AC nhanh nhất	Phân tích kỹ thuật được dùng trong lời giải thực tế
Danh sách dạng bài	Giới hạn AI chỉ chọn các dạng bài đã có trong hệ thống

Bảng 3.8.5: Dữ liệu đầu vào và mục đích của AI gợi ý dạng bài

3.9. Thiết kế cơ chế khóa tập trung trong kỳ thi

Trong các kỳ thi lập trình trực tuyến, việc đảm bảo người dùng tập trung trong giao diện làm bài là một yêu cầu quan trọng. Khác với hình thức thi trực tiếp trong phòng máy, thi trực tuyến khiến người tổ chức khó kiểm soát hoàn toàn môi trường làm bài của thí sinh. Người dùng có thể chuyển sang tab khác, mở công cụ bên ngoài, tìm kiếm tài liệu hoặc sử dụng các công cụ hỗ trợ không được phép trong quá trình thi.

Để hỗ trợ hạn chế các hành vi này, BKDNOJ được thiết kế thêm cơ chế khóa tập trung trong kỳ thi. Cơ chế này giúp hệ thống theo dõi trạng thái tập trung của người dùng khi làm bài, ghi nhận các hành vi rời khỏi giao diện thi và hỗ trợ người tổ chức trong việc đánh giá tính nghiêm túc của quá trình làm bài.

Cơ chế khóa tập trung không được xem là giải pháp chống gian lận tuyệt đối, nhưng đóng vai trò là một lớp kiểm soát bổ sung trong hệ thống. Khi kết hợp với IDE trực tuyến và luồng chạy thử chương trình, người dùng có thể làm bài, chạy thử và nộp bài ngay trong giao diện thi mà không cần rời khỏi hệ thống để sử dụng công cụ bên ngoài. [14], [15]

3.9.1. Mục tiêu của cơ chế khóa tập trung

Mục tiêu chính của cơ chế khóa tập trung là yêu cầu người dùng duy trì quá trình làm bài trong giao diện kỳ thi của hệ thống. Khi kỳ thi được bật chế độ khóa tập trung, người dùng cần thực hiện bài làm trong môi trường trình duyệt được kiểm soát, hạn chế việc chuyển tab, thoát khỏi chế độ toàn màn hình hoặc rời khỏi cửa sổ thi.

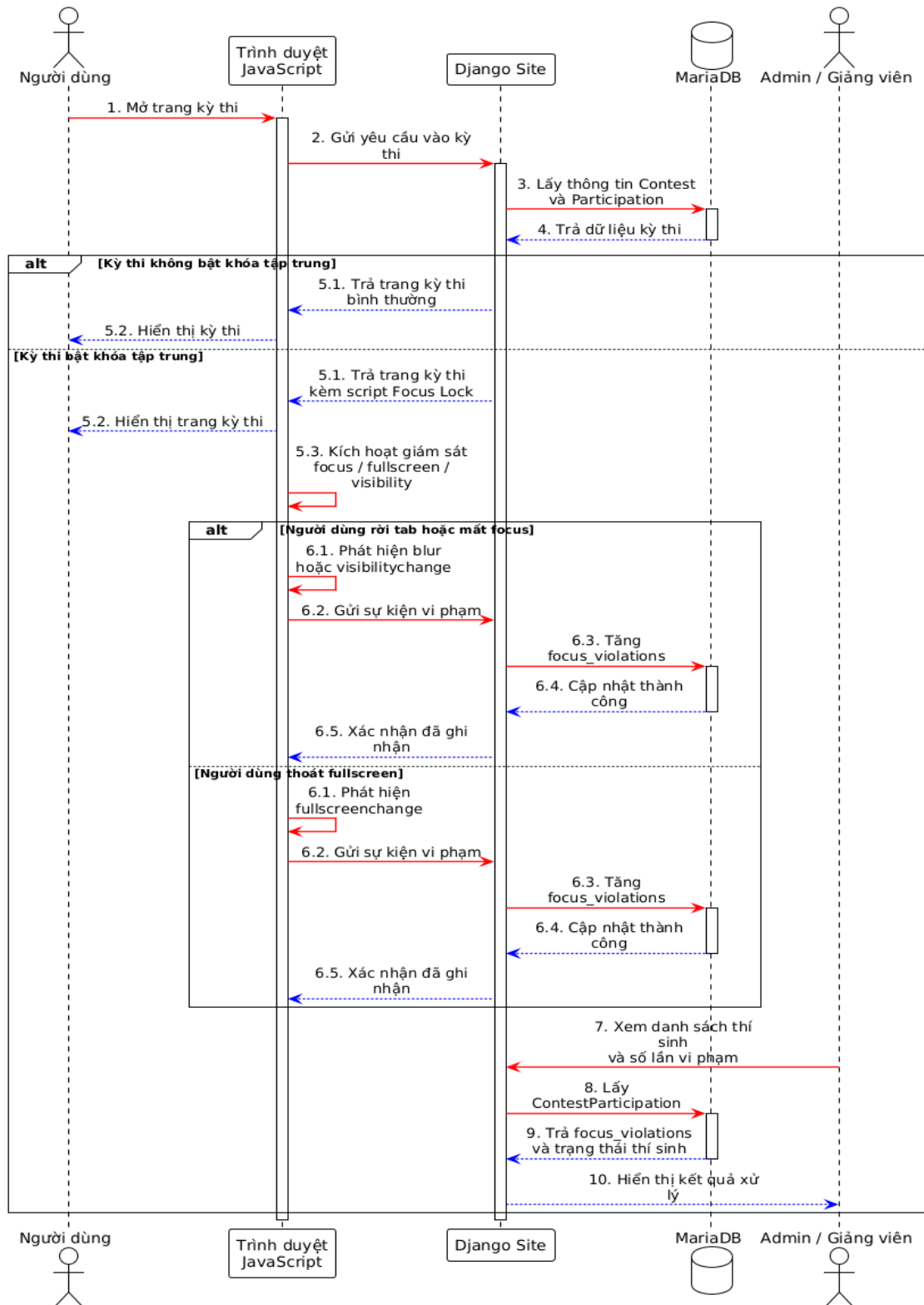
Cơ chế này giúp nâng cao tính nghiêm túc và công bằng trong quá trình tổ chức kỳ thi trực tuyến. Khi người dùng có hành vi mất tập trung hoặc rời khỏi giao diện làm bài, hệ thống có thể ghi nhận thông tin vi phạm để phục vụ quá trình theo dõi và xử lý sau đó. Nhờ vậy, người tổ chức kỳ thi có thêm dữ liệu tham khảo khi đánh giá quá trình làm bài của thí sinh.

Bên cạnh đó, cơ chế khóa tập trung còn có ý nghĩa trong bối cảnh các công cụ AI ngày càng phổ biến. Nếu không có cơ chế kiểm soát phù hợp, người dùng có thể dễ dàng chuyển sang công cụ bên ngoài để tìm kiếm lời giải, phân tích đề hoặc tạo mã nguồn. Việc yêu cầu người dùng làm bài trong giao diện tập trung giúp giảm khả năng sử dụng các công cụ không được phép trong thời gian thi.

Cơ chế khóa tập trung được thiết kế để kết hợp với IDE trực tuyến. Người dùng vẫn có thể viết mã, chạy thử chương trình và nộp bài trong hệ thống, nhưng bị hạn chế việc rời khỏi giao diện thi. Điều này vừa đảm bảo trải nghiệm làm bài cần thiết, vừa hỗ trợ hạn chế gian lận trong quá trình tổ chức kỳ thi lập trình trực tuyến.

3.9.2. Luồng xử lý khi người dùng tham gia kỳ thi

Luồng xử lý của cơ chế khóa tập trung bắt đầu khi người dùng truy cập vào một kỳ thi có bật chế độ khóa tập trung. Trước khi vào giao diện làm bài, hệ thống kiểm tra trạng thái kỳ thi, quyền tham gia của người dùng và cấu hình khóa tập trung. Nếu người dùng đủ điều kiện tham gia, hệ thống chuyển người dùng vào giao diện thi được kiểm soát.



Hình 3.9.2: Sơ đồ tuần tự cơ chế khóa tập trung

Khi kỳ thi bắt cơ chế khóa tập trung, thí sinh tham gia chính thức sẽ được chuyển đến một trang trung gian thay vì vào thẳng giao diện thi. Trang này yêu cầu thí sinh kích hoạt chế độ toàn màn hình trước khi bắt đầu làm bài. Sau khi vào toàn màn hình, giao diện thi được tải bên trong một khung nội dung chiếm toàn bộ màn hình, đồng thời thanh điều hướng của hệ thống được ẩn đi để hạn chế thao tác rời khỏi môi trường thi.

Trong quá trình thi, hệ thống theo dõi các hành vi như thoát khỏi chế độ toàn màn hình, chuyển tab hoặc chuyển sang cửa sổ khác. Khi phát hiện các hành vi này, trình duyệt gửi yêu cầu đến máy chủ để ghi nhận vi phạm. Máy chủ sẽ tăng số lần vi phạm tập trung trong dữ liệu tham gia kỳ thi của thí sinh. Sau đó, lớp phủ toàn màn hình được hiển thị lại để yêu cầu thí sinh quay về chế độ toàn màn hình và tiếp tục làm bài.

Để tăng hiệu quả kiểm soát, hệ thống chặn một số phím tắt như tải lại trang, bật hoặc tắt toàn màn hình và hạn chế thoát toàn màn hình bằng bàn phím trên các trình duyệt hỗ trợ. Phía máy chủ cũng kiểm tra các yêu cầu truy cập trong thời gian thi; nếu thí sinh cố truy cập ra ngoài trang thi được kiểm soát, hệ thống sẽ chặn hoặc chuyển hướng trở lại trang làm bài.

Cơ chế khóa tập trung không làm thay đổi luồng làm bài và chấm bài. Thí sinh vẫn có thể xem đề, viết mã, chạy thử và nộp bài trong giao diện thi. Cơ chế này chỉ áp dụng cho thí sinh tham gia chính thức, không áp dụng cho người xem hoặc người tham gia thi ảo.

Sau khi kỳ thi kết thúc, số lần vi phạm được lưu lại để giảng viên hoặc quản trị viên xem xét. Hệ thống không tự động kết luận gian lận hay loại thí sinh, mà chỉ cung cấp dữ liệu hỗ trợ cho quá trình đánh giá.

3.10. Tổng kết chương

Nội dung chương đã trình bày quá trình phân tích và thiết kế hệ thống BKDNOJ từ mức tổng quan đến các thành phần chức năng cụ thể. Trước hết, hệ thống được xác định thông qua mục tiêu thiết kế, các nhóm người dùng, yêu cầu chức năng, yêu cầu phi chức năng và sơ đồ Use Case tổng quát. Đây là cơ sở để làm rõ phạm vi hoạt động của hệ thống cũng như vai trò của từng nhóm tác nhân trong quá trình sử dụng.

Về mặt kiến trúc, BKDNOJ được thiết kế theo mô hình nhiều thành phần, bao gồm giao diện người dùng, máy chủ web, ứng dụng chính, cơ sở dữ liệu, bộ nhớ đệm, tác vụ nền, WebSocket, Bridge, máy chấm và dịch vụ AI. Cách tổ chức này giúp hệ thống phân tách rõ trách nhiệm giữa các thành phần, hỗ trợ xử lý các nghiệp vụ quan trọng như nộp bài, chấm bài, chạy thử chương trình, cập nhật kết quả theo thời gian thực và thực hiện các chức năng AI.

Bên cạnh kiến trúc tổng thể, nội dung chương cũng đã trình bày thiết kế cơ sở dữ liệu và các luồng xử lý chính của hệ thống. Các luồng như nộp bài chính thức, chấm bài, tính điểm, chạy thử chương trình bằng IDE trực tuyến, quản lý API key AI, phân tích mã nguồn, tạo đề bài, sinh test case, đánh giá năng lực và dự đoán dạng đề được thiết kế nhằm đáp ứng yêu cầu học tập, luyện tập và tổ chức kỳ thi lập trình trực tuyến.

Ngoài ra, cơ chế khóa tập trung trong kỳ thi cũng được thiết kế như một giải pháp hỗ trợ hạn chế gian lận. Cơ chế này giúp theo dõi trạng thái tập trung của thí sinh trong quá trình làm bài, ghi nhận số lần vi phạm và cung cấp dữ liệu tham khảo cho giảng viên hoặc quản trị viên sau kỳ thi.

Nhìn chung, các nội dung phân tích và thiết kế đã tạo nên nền tảng đầy đủ cho việc xây dựng hệ thống BKDNOJ. Hệ thống không chỉ đáp ứng các chức năng cốt lõi của một nền tảng Online Judge, mà còn được mở rộng theo hướng hỗ trợ học tập thông minh, cải thiện trải nghiệm làm bài và tăng khả năng quản lý trong các kỳ thi lập trình trực tuyến.

CHƯƠNG 4. XÂY DỰNG VÀ TRIỂN KHAI HỆ THỐNG

4.1 Môi trường và công nghệ sử dụng

Hệ thống BKDNOJ được xây dựng và triển khai dưới dạng ứng dụng web nhiều thành phần. Các thành phần trong hệ thống được tổ chức thành các dịch vụ riêng nhằm phục vụ các chức năng như quản lý bài tập, nộp bài, chấm bài tự động, chạy thử chương trình, tổ chức kỳ thi và tích hợp các chức năng AI. [4], [6]–[10]

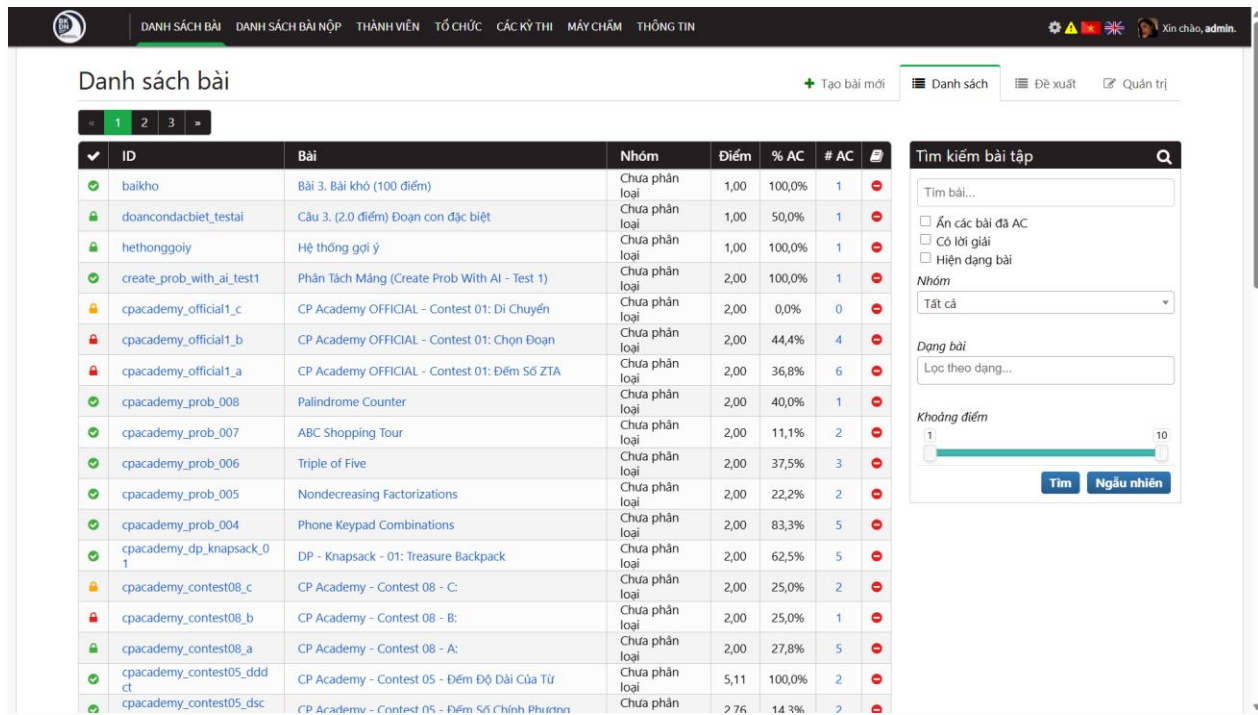
Trong quá trình xây dựng hệ thống, Django được sử dụng làm nền tảng chính để phát triển ứng dụng web và xử lý nghiệp vụ. Cơ sở dữ liệu được sử dụng để lưu trữ các thông tin quan trọng như người dùng, bài tập, test case, bài nộp, kỳ thi và các dữ liệu mở rộng. Redis và Celery hỗ trợ xử lý các tác vụ nền, trong khi WebSocket được sử dụng để cập nhật trạng thái xử lý theo thời gian thực. Bên cạnh đó, hệ thống sử dụng Bridge và máy chấm để thực hiện quá trình biên dịch, chạy chương trình và chấm bài tự động. [4], [6], [7]

Thành phần	Vai trò
Django Site	Xử lý nghiệp vụ chính của hệ thống
Cơ sở dữ liệu	Lưu trữ dữ liệu người dùng, bài tập, bài nộp, kỳ thi và cấu hình hệ thống
Redis	Hỗ trợ bộ nhớ đệm, hàng đợi và dữ liệu tạm
Celery	Xử lý các tác vụ nền hoặc tác vụ mất nhiều thời gian
WebSocket	Cập nhật trạng thái bài nộp, chạy thử và tác vụ theo thời gian thực
Nginx	Tiếp nhận request, điều hướng dịch vụ và phục vụ tài nguyên tĩnh
Bridge	Kết nối giữa hệ thống web và máy chấm
Máy chấm	Biên dịch, thực thi mã nguồn và trả kết quả chấm
Docker/Docker Compose	Đóng gói, cấu hình và triển khai các dịch vụ của hệ thống
Dịch vụ AI	Hỗ trợ phân tích mã nguồn, tạo đề bài, sinh test case và đánh giá năng lực

Bảng 4.1: Môi trường và các công nghệ sử dụng

4.2. Xây dựng các chức năng nền tảng

Các chức năng nền tảng là phần cốt lõi giúp BKDNOJ hoạt động như một hệ thống chấm bài lập trình trực tuyến. Trong quá trình xây dựng, hệ thống đã hoàn thiện các chức năng chính như quản lý người dùng, quản lý bài tập, quản lý test case, nộp bài, chấm bài tự động, quản lý kỳ thi và hiển thị kết quả chấm.



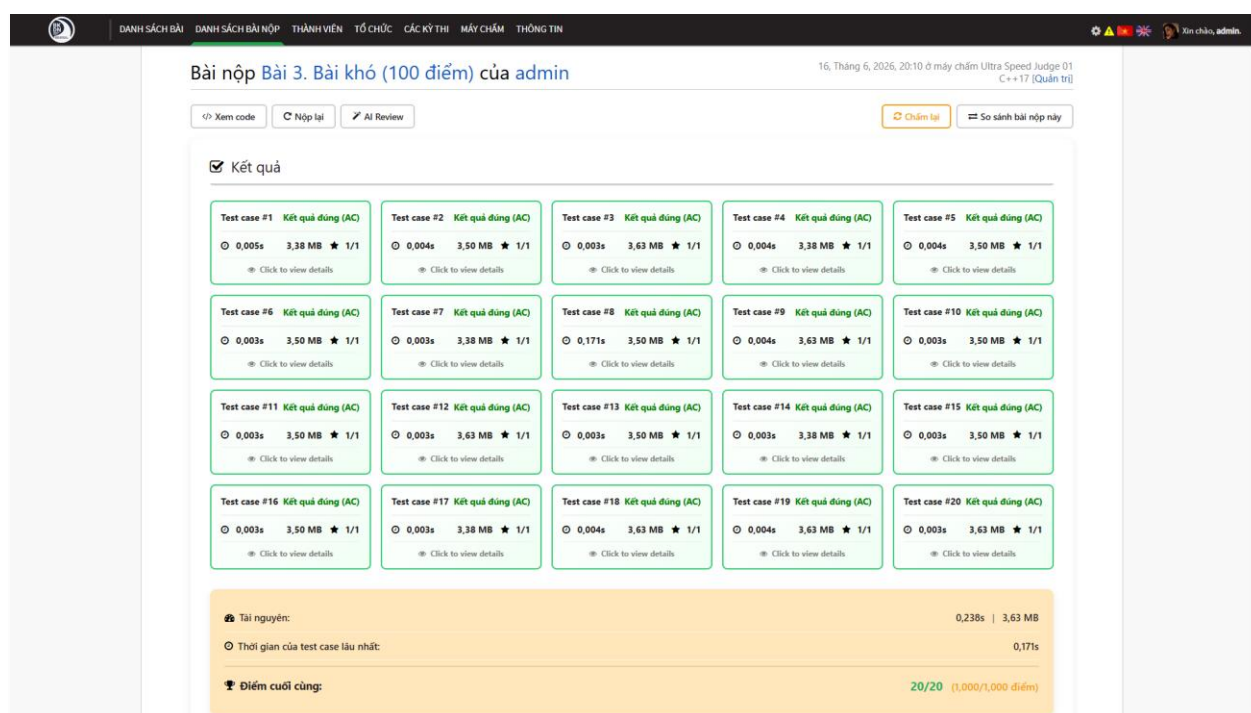
Hình 4.2.1: Giao diện xem danh sách bài tập

Hình trên thể hiện giao diện bài tập trong hệ thống. Người dùng có thể xem danh sách bài, truy cập nội dung bài toán, xem giới hạn thời gian, giới hạn bộ nhớ, dữ liệu vào, dữ liệu ra và các ví dụ minh họa. Đây là giao diện chính phục vụ quá trình luyện tập và làm bài của người dùng.

Thời gian ^{UTC+7}	Tên truy cập	Bài	Ngôn ngữ	Trạng thái	Kết quả	Thời gian	Bộ nhớ
12:10:49 sa, 17/06/2026	admin	CP Academy OFFICIAL - Contest 01: Di Chuyển	C++THEMIS	Kết quả sai (WA)	0 / 100	4 ms	1,50 MB
8:10:19 ch, 16/06/2026	admin	Bài 3. Bài khó (100 điểm)	C++17	Kết quả đúng (AC)	20 / 20	171 ms	3,63 MB
2:51:34 sa, 16/06/2026	admin	Câu 3. (2.0 điểm) Đoạn con đặc biệt	C++17	Kết quả đúng (AC)	20 / 20	141 ms	14,75 MB
2:46:01 sa, 16/06/2026	admin	Câu 3. (2.0 điểm) Đoạn con đặc biệt	C++17	Kết quả sai (WA)	0 / 20	5 ms	3,88 MB
12:09:24 sa, 16/06/2026	admin	Hệ thống gợi ý	C++17	Kết quả đúng (AC)	50 / 50	12 ms	4,63 MB
3:39:41 sa, 14/06/2026	admin	Phân Tách Mảng (Create Prob With AI - Test 1)	C++17	Kết quả đúng (AC)	50 / 50	21 ms	14,00 MB
3:21:13 sa, 08/06/2026	admin	A Plus B	PY3	Lỗi khi chạy chương trình (IR)	0 / 40	55 ms	11,88 MB
9:49:02 ch, 01/06/2026	admin	A Plus B	C++17	Kết quả sai (WA)	0 / 40	8 ms	3,50 MB
9:48:35 ch, 01/06/2026	admin	A Plus B	C++17	Kết quả đúng (AC)	40 / 40	113 ms	3,50 MB

Hình 4.2.2: Giao diện xem lịch sử bài nộp

Thông qua chức năng nộp bài, người dùng có thể gửi mã nguồn lên hệ thống để được chấm tự động. Sau khi bài nộp được tiếp nhận, hệ thống chuyển bài nộp đến máy chấm, chạy chương trình với các test case và cập nhật kết quả về giao diện người dùng.



Hình 4.2.3: Xem chi tiết kết quả bài nộp

Kết quả chấm bài được hiển thị với các trạng thái như Accepted, Wrong Answer, Time Limit Exceeded, Memory Limit Exceeded, Runtime Error hoặc Compilation Error. Thông tin này giúp người dùng biết được lời giải của mình có đúng hay không và cần điều chỉnh ở điểm nào.

The screenshot shows the 'Bách Khoa Đà Nẵng Online Judge' dashboard. The left sidebar contains navigation links: Dashboard, Bài (selected), Nhóm bài, Đăng đề, Giấy phép, Tag problems, Bài nộp, Ngôn ngữ, Kỳ thi, Tickets, Người sử dụng, Tổ chức, Thanh điều hướng, Bài đăng blog, Nhận xét, Flat pages, and Cấu hình khác. The main content area is titled 'Bài' and includes filters for 'Lọc' (All, Hide, Show), 'Bối cảnh thi công khai' (All, Hide, Show), and 'Bối cảnh creator' (All, Hide, Show). It also shows the year '2017 2025 2026' and a search bar. Below the filters is a table of problems with columns: Mã Bài, Tên Bài Toán, Tác Giả, Điểm, and Hiện Thị Công Khai. The table lists 16 problems, including 'A Plus B', 'Bài 3. Bài khó (100 điểm)', and various 'CP Academy' contests.

Mã Bài	Tên Bài Toán	Tác Giả	Điểm	Hiện Thị Công Khai
aplusb	A Plus B	admin	5,0	Xem trên trang web
baikho	Bài 3. Bài khó (100 điểm)	admin	1,0	Xem trên trang web
cpacademy_contest02_a	CP Academy - Contest 02 - Ở Năng Lượng Đặc Biệt	admin	1,374	Xem trên trang web
cpacademy_contest02_b	CP Academy - Contest 02 - Cường Độ Năng Lượng	admin	1,374	Xem trên trang web
cpacademy_contest02_c	CP Academy - Contest 02 - Mua Hàng Với Phiếu Giảm Giá	admin	1,374	Xem trên trang web
cpacademy_contest02_d	CP Academy - Contest 02 - Trắc Nghiệm Về DPT	admin	1,374	Xem trên trang web
cpacademy_contest03_a	CP Academy - Contest 03 - A	admin	1,374	Xem trên trang web
cpacademy_contest03_b	CP Academy - Contest 03 - B	admin	1,374	Xem trên trang web
cpacademy_contest03_c	CP Academy - Contest 03 - C	admin	1,374	Xem trên trang web
cpacademy_contest03_d	CP Academy - Contest 03 - D	admin	8,215	Xem trên trang web
cpacademy_contest05_ddct	CP Academy - Contest 05 - Đếm Độ Dài Của Từ	admin	5,108	Xem trên trang web
cpacademy_contest05_dscp	CP Academy - Contest 05 - Đếm Số Chính Phương	admin	2,761	Xem trên trang web
cpacademy_contest05_hq	CP Academy - Contest 05 - Hộp Quà	admin	5,108	Xem trên trang web
cpacademy_contest06_a	CP Academy - Contest 06 - A:	admin	2,0	Xem trên trang web
cpacademy_contest06_b	CP Academy - Contest 06 - B:	admin	2,0	Xem trên trang web
cpacademy_contest06_bai4	CP Academy - Contest 06 - Bài 4	admin	6,573	Xem trên trang web

Hình 4.2.4: Giao diện quản lý bài tập

The screenshot shows the 'Chỉnh sửa các dữ liệu cho Bài 3. Bài khó (100 điểm)' form. It includes a sidebar with 'Tập tin dữ liệu nén dạng zip:', 'Grader:', 'IO Method:', 'Trình chấm:', and 'Hạn chế chiều dài đầu ra:'. The main area has a 'Hiện nay: baikho/data.zip' section with a 'Thay đổi' button and a 'Nhấn nút này nếu bạn mới thay đổi file test' button. Below this is a table with 5 rows, each representing a test case. The table has columns: +, 1, Kiểu, Tập tin đầu vào, Tập tin đầu ra, Điểm, Pretest?, Sample?, and Xóa?.

+	1	Kiểu	Tập tin đầu vào	Tập tin đầu ra	Điểm	Pretest?	Sample?	Xóa?
+	1	Test đơn	baikho/Testcase001/baikho.inp	baikho/Testcase001/baikho.out	1	☐	☐	☐
+	2	Test đơn	baikho/Testcase002/baikho.inp	baikho/Testcase002/baikho.out	1	☐	☐	☐
+	3	Test đơn	baikho/Testcase003/baikho.inp	baikho/Testcase003/baikho.out	1	☐	☐	☐
+	4	Test đơn	baikho/Testcase004/baikho.inp	baikho/Testcase004/baikho.out	1	☐	☐	☐
+	5	Test đơn	baikho/Testcase005/baikho.inp	baikho/Testcase005/baikho.out	1	☐	☐	☐

Hình 4.2.5: Giao diện quản lý test case

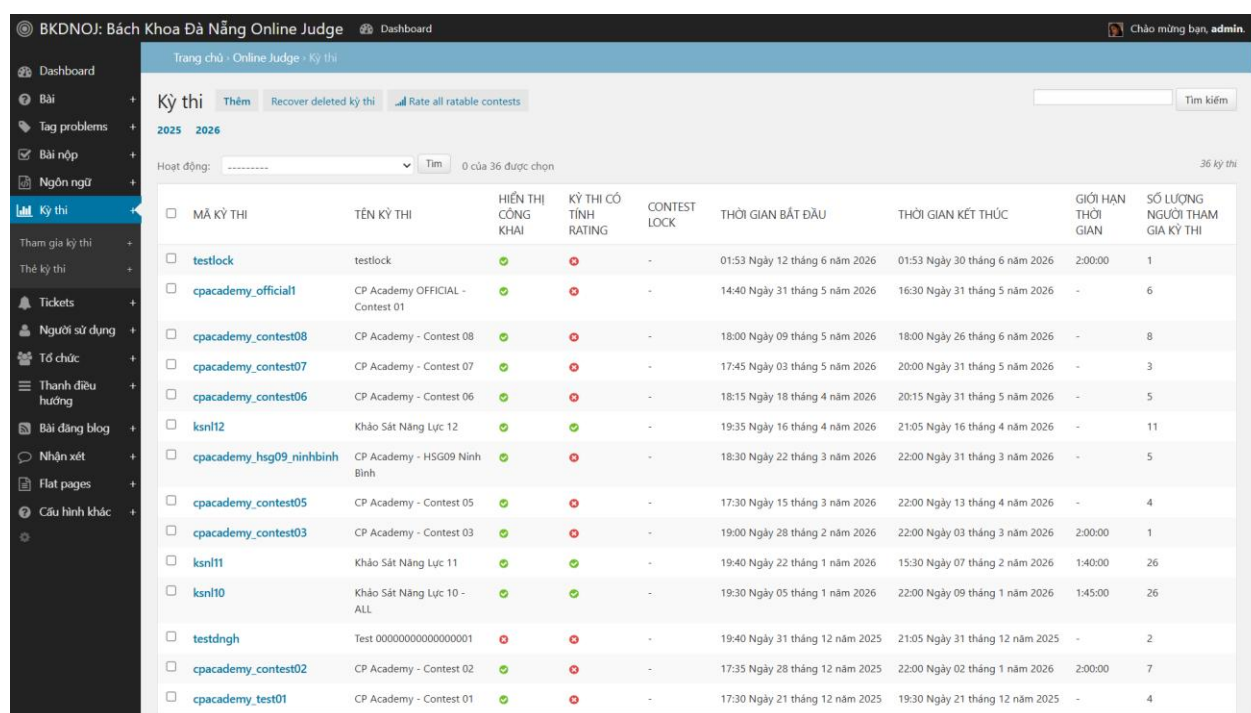
The screenshot shows the 'Thêm vấn đề' (Add problem) form in the BKDNO: Bách Khoa Đà Nẵng Online Judge system. The interface includes a sidebar with navigation options like 'Dashboard', 'Bài', 'Nhóm bài', 'Dạng đề', 'Giấy phép', 'Tag problems', 'Bài nộp', 'Ngôn ngữ', 'Kỳ thi', 'Tickets', 'Người sử dụng', 'Tổ chức', 'Thanh điều hướng', 'Bài đăng blog', 'Nhận xét', 'Flat pages', and 'Cấu hình khác'. The main form contains the following fields and options:

- Mã bài:** A text input field for the problem ID.
- Tên bài toán:** A text input field for the problem name.
- Suggester:** A dropdown menu to select the user who suggested the problem.
- Options:** Checkboxes for 'Hiện thị công khai' (Public display) and 'Quản lý test thủ công' (Manual test case management).
- Ngày đăng:** A date and time selection field.
- Người tạo:** A text input field for the creator's name.
- Giám khảo:** A text input field for the reviewer's name.
- Testers:** A text input field for the testers' names.
- Tổ chức:** A dropdown menu to select the organization.
- Hiện thị submission:** A dropdown menu to select the submission display setting.

At the bottom of the form, there are three buttons: 'Lưu' (Save), 'Lưu và thêm mới' (Save and add new), and 'Lưu và tiếp tục chỉnh sửa' (Save and continue editing).

Hình 4.2.6: Giao diện thêm bài tập

Đối với giảng viên hoặc quản trị viên, hệ thống cung cấp giao diện quản lý bài tập và test case. Người quản lý có thể tạo bài, chỉnh sửa nội dung, cấu hình giới hạn, thêm dữ liệu kiểm thử và chuẩn bị bài toán cho quá trình chấm tự động.



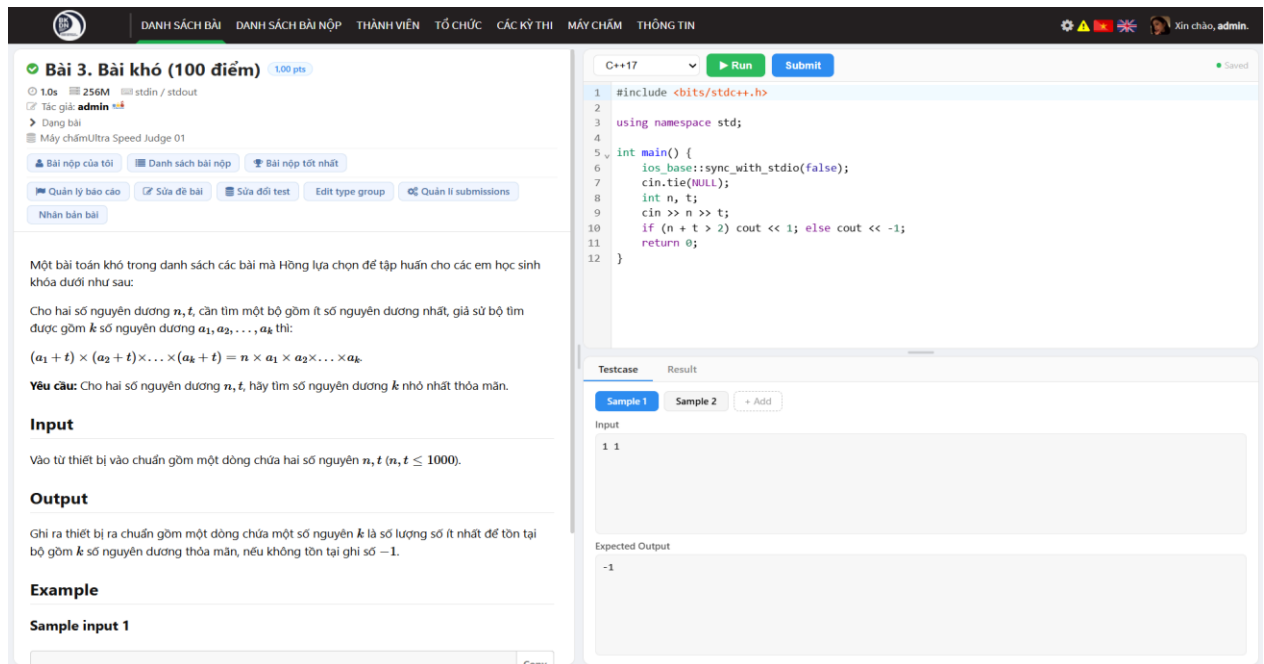
MÃ KỶ THI	TÊN KỶ THI	HIỆN THỊ CÔNG KHAI	KỶ THI CÓ TÍNH RATING	CONTEST LOCK	THỜI GIAN BẮT ĐẦU	THỜI GIAN KẾT THÚC	GIỚI HẠN THỜI GIAN	SỐ LƯỢNG NGƯỜI THAM GIA KỶ THI
testlock	testlock	✓	✗	-	01:53 Ngày 12 tháng 6 năm 2026	01:53 Ngày 30 tháng 6 năm 2026	2:00:00	1
cpacademy_official1	CP Academy OFFICIAL - Contest 01	✓	✗	-	14:40 Ngày 31 tháng 5 năm 2026	16:30 Ngày 31 tháng 5 năm 2026	-	6
cpacademy_contest08	CP Academy - Contest 08	✓	✗	-	18:00 Ngày 09 tháng 5 năm 2026	18:00 Ngày 26 tháng 6 năm 2026	-	8
cpacademy_contest07	CP Academy - Contest 07	✓	✗	-	17:45 Ngày 03 tháng 5 năm 2026	20:00 Ngày 31 tháng 5 năm 2026	-	3
cpacademy_contest06	CP Academy - Contest 06	✓	✗	-	18:15 Ngày 18 tháng 4 năm 2026	20:15 Ngày 31 tháng 5 năm 2026	-	5
ksnl12	Khảo Sát Năng Lực 12	✓	✓	-	19:35 Ngày 16 tháng 4 năm 2026	21:05 Ngày 16 tháng 4 năm 2026	-	11
cpacademy_hsg09_ninhbinh	CP Academy - HSG09 Ninh Bình	✓	✗	-	18:30 Ngày 22 tháng 3 năm 2026	22:00 Ngày 31 tháng 3 năm 2026	-	5
cpacademy_contest05	CP Academy - Contest 05	✓	✗	-	17:30 Ngày 15 tháng 3 năm 2026	22:00 Ngày 13 tháng 4 năm 2026	-	4
cpacademy_contest03	CP Academy - Contest 03	✓	✗	-	19:00 Ngày 28 tháng 2 năm 2026	22:00 Ngày 03 tháng 3 năm 2026	2:00:00	1
ksnl11	Khảo Sát Năng Lực 11	✓	✓	-	19:40 Ngày 22 tháng 1 năm 2026	15:30 Ngày 07 tháng 2 năm 2026	1:40:00	26
ksnl10	Khảo Sát Năng Lực 10 - ALL	✓	✓	-	19:30 Ngày 05 tháng 1 năm 2026	22:00 Ngày 09 tháng 1 năm 2026	1:45:00	26
testdngnh	Test 00000000000000000001	✗	✗	-	19:40 Ngày 31 tháng 12 năm 2025	21:05 Ngày 31 tháng 12 năm 2025	-	2
cpacademy_contest02	CP Academy - Contest 02	✓	✗	-	17:35 Ngày 28 tháng 12 năm 2025	22:00 Ngày 02 tháng 1 năm 2026	2:00:00	7
cpacademy_test01	CP Academy - Contest 01	✓	✗	-	17:30 Ngày 21 tháng 12 năm 2025	19:30 Ngày 21 tháng 12 năm 2025	-	4

Hình 4.2.7: Giao diện quản lý kỳ thi

Ngoài chức năng luyện tập thông thường, hệ thống còn hỗ trợ tổ chức kỳ thi lập trình. Người quản trị hoặc giảng viên có thể tạo kỳ thi, thiết lập thời gian, thêm bài toán, quản lý thí sinh và theo dõi kết quả làm bài trong kỳ thi.

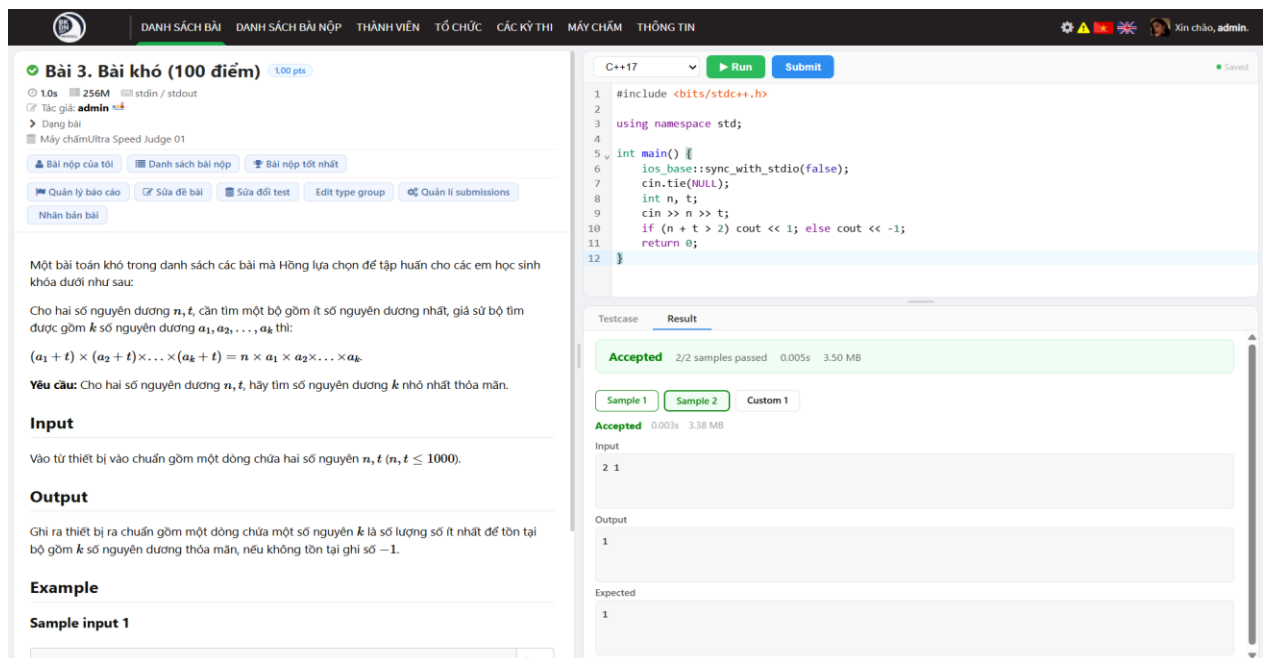
4.3. Xây dựng IDE trực tuyến và luồng chạy thử

IDE trực tuyến được xây dựng nhằm hỗ trợ người dùng viết mã trực tiếp trên giao diện bài tập. Thay vì phải sử dụng công cụ lập trình bên ngoài, người dùng có thể đọc đề, viết mã, chọn ngôn ngữ lập trình, chạy thử và nộp bài chính thức ngay trong hệ thống BKDNOJ.



Hình 4.3.1: Giao diện IDE trực tuyến

Hình trên thể hiện giao diện IDE trực tuyến được tích hợp vào trang bài tập. Khu vực soạn thảo mã nguồn cho phép người dùng nhập lời giải, lựa chọn ngôn ngữ lập trình và sử dụng các chức năng như chạy thử hoặc nộp bài. Việc tích hợp IDE giúp quá trình làm bài trở nên liền mạch và thuận tiện hơn.



Hình 4.3.2: Thao tác chạy thử chương trình với IDE trực tuyến

Chức năng chạy thử cho phép người dùng kiểm tra nhanh chương trình trước khi nộp bài chính thức. Khi người dùng thực hiện chạy thử, hệ thống xử lý yêu cầu thông qua luồng riêng và trả kết quả ngay trên giao diện IDE.

Kết quả chạy thử hiển thị các thông tin như trạng thái thực thi, thời gian chạy, bộ nhớ sử dụng, dữ liệu đầu ra và thông báo lỗi nếu có. Các lần chạy thử chỉ phục vụ mục đích kiểm tra mã nguồn, không ảnh hưởng đến điểm số, bảng xếp hạng hoặc kết quả chấm chính thức của bài toán.

Việc xây dựng IDE trực tuyến và chức năng chạy thử giúp nâng cao trải nghiệm làm bài của người dùng. Đồng thời, chức năng này cũng hỗ trợ tốt cho các kỳ thi có bật cơ chế khóa tập trung, vì thí sinh có thể viết mã, chạy thử và nộp bài ngay trong giao diện thi mà không cần chuyển sang công cụ bên ngoài.

4.4. Xây dựng các chức năng AI

Các chức năng AI được tích hợp vào BKDNOJ nhằm hỗ trợ người học, giảng viên và quản trị viên trong quá trình sử dụng hệ thống. Các chức năng này bao gồm quản lý API key AI, phân tích mã nguồn, hỗ trợ tạo đề bài, sinh test case, đánh giá năng lực người dùng và dự đoán dạng đề. [12], [13]

AI API Keys

Create new blog post About Progress Statistics Blogs Comments Admin User Admin Profile Edit profile AI API Keys

Add API keys from AI providers to enable AI-powered features. Keys are encrypted before storage.

Add API Key

-- Select provider -- Paste your API key here + Add

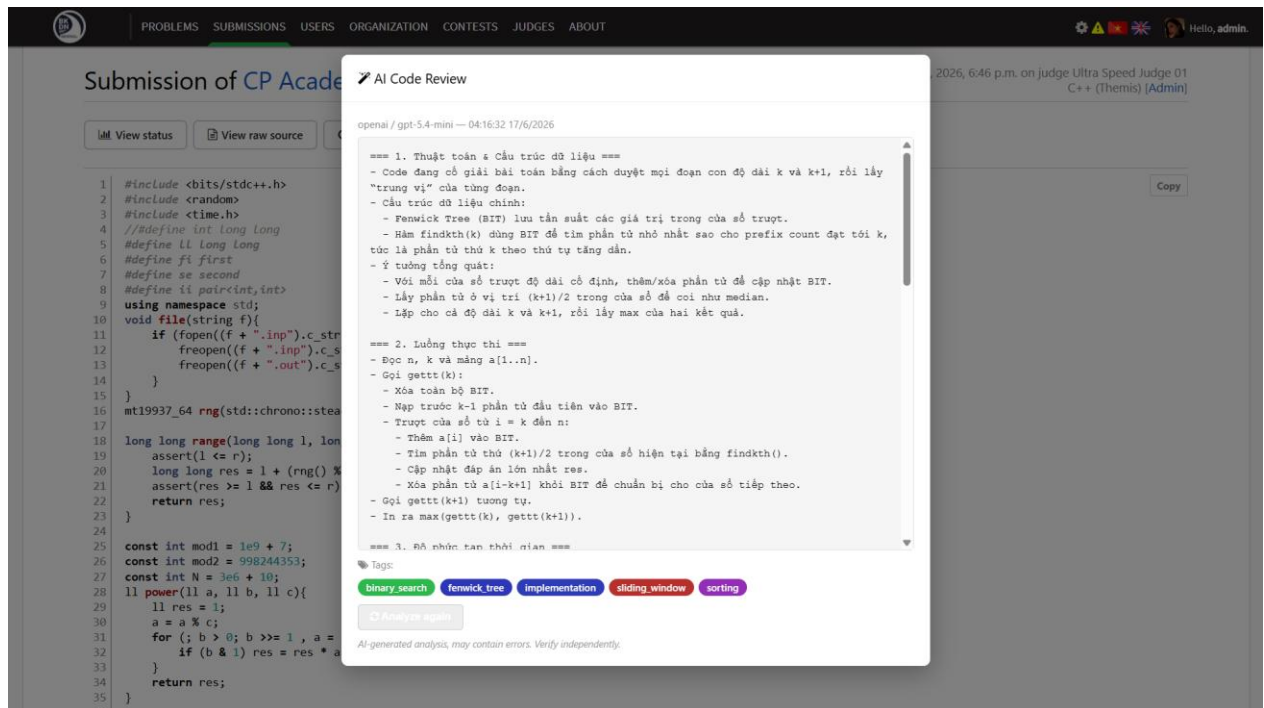
Provider	API Key	Status	Model	Actions
OpenAI	****4mYA	Verified	gpt-4o-mini	Test
Google Gemini	****gzH	Verified	gemini-2.5-flash	Test

Test Logs

Time	Provider	Model	Result	Time (ms)	Detail
04:13:59 17/6/2026	Google Gemini	gemini-3.1-flash-lite	OK	3678ms	Connection successful
04:13:46 17/6/2026	OpenAI	gpt-5.4-mini	OK	1087ms	Connection successful
04:13:37 17/6/2026	Google Gemini	gemini-2.5-pro	Failed	267ms	Rate limited, try later
04:13:34 17/6/2026	Google Gemini	gemini-3.1-pro-preview	Failed	305ms	Rate limited, try later
04:13:30 17/6/2026	OpenAI	gpt-5.4-nano	OK	2043ms	Connection successful

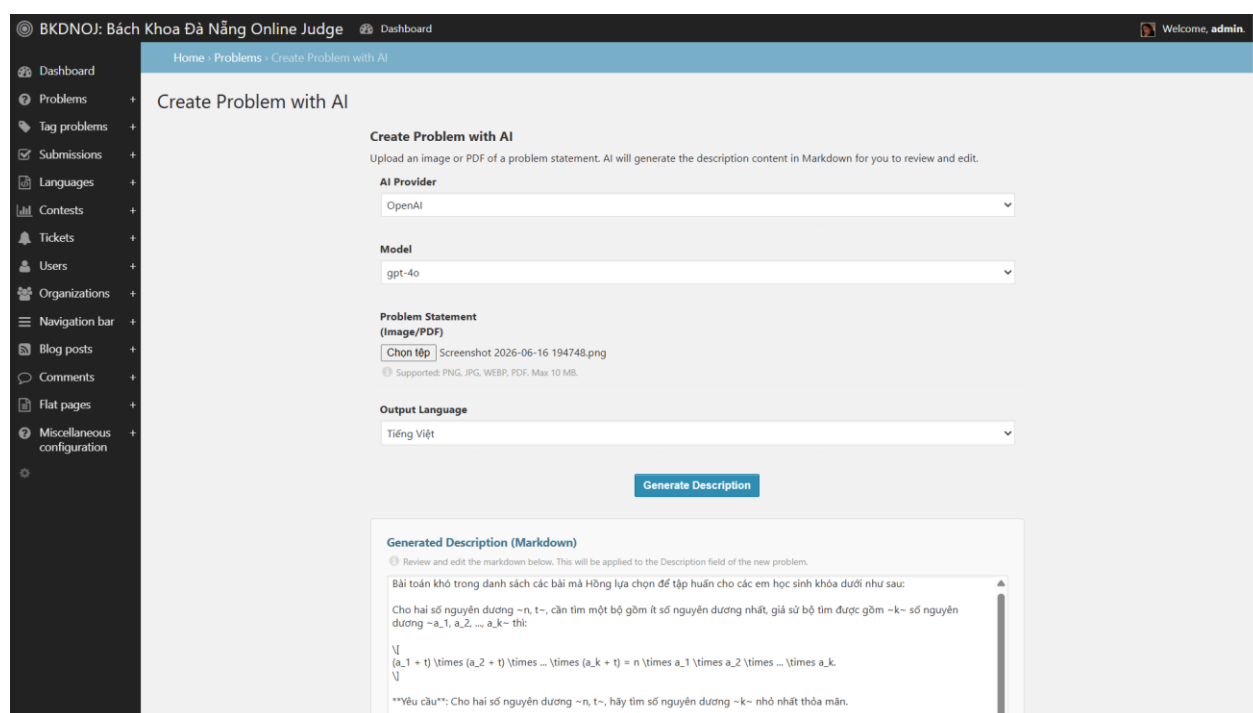
Hình 4.4.1: Giao diện quản lý API Key

Hình trên thể hiện giao diện quản lý API key AI trong hệ thống. Người dùng có thể thêm API key, lựa chọn nhà cung cấp AI, kiểm tra kết nối và sử dụng khóa này cho các chức năng AI. Việc quản lý API key theo từng người dùng giúp hệ thống linh hoạt hơn và không phụ thuộc vào một khóa dùng chung.



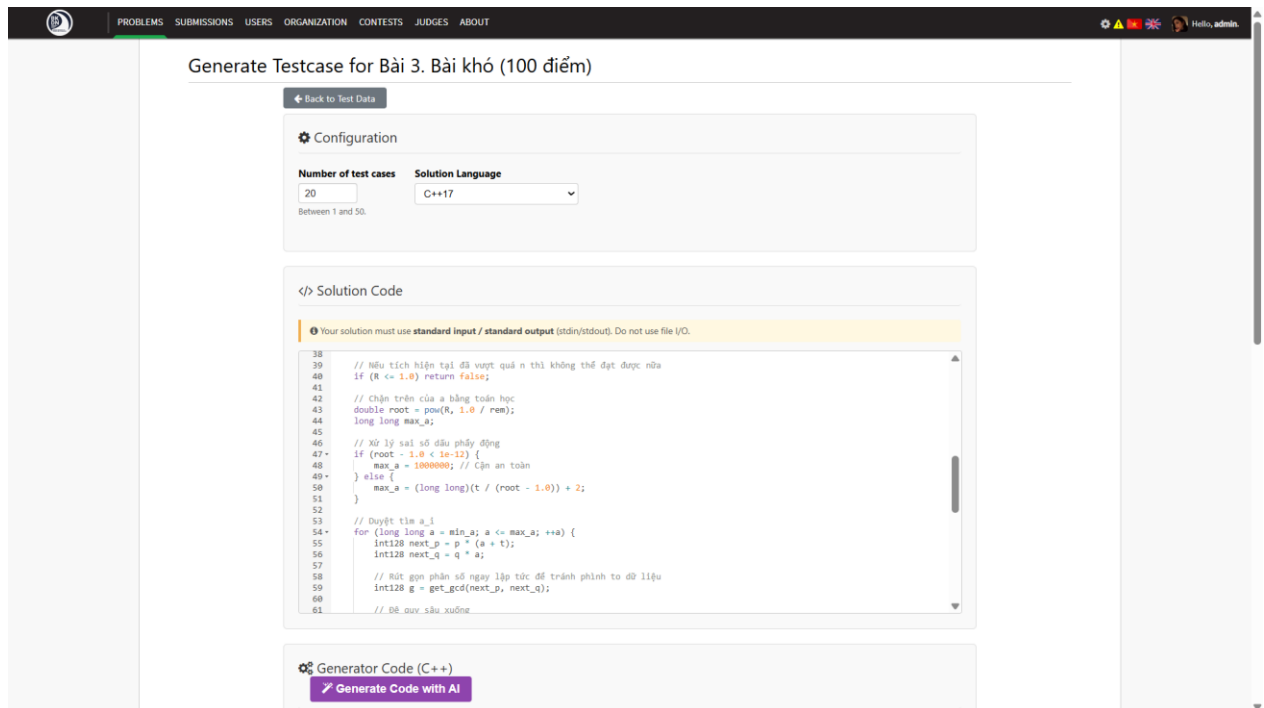
Hình 4.4.2: Giao diện AI phân tích mã nguồn

Chức năng AI phân tích mã nguồn cho phép người dùng gửi mã nguồn để nhận nhận xét từ AI. Kết quả phân tích có thể bao gồm đánh giá ý tưởng thuật toán, độ phức tạp, lỗi thường gặp và gợi ý cải thiện lời giải. Chức năng này giúp người học hiểu rõ hơn về chất lượng mã nguồn của mình, thay vì chỉ nhận kết quả đúng hoặc sai từ máy chấm.



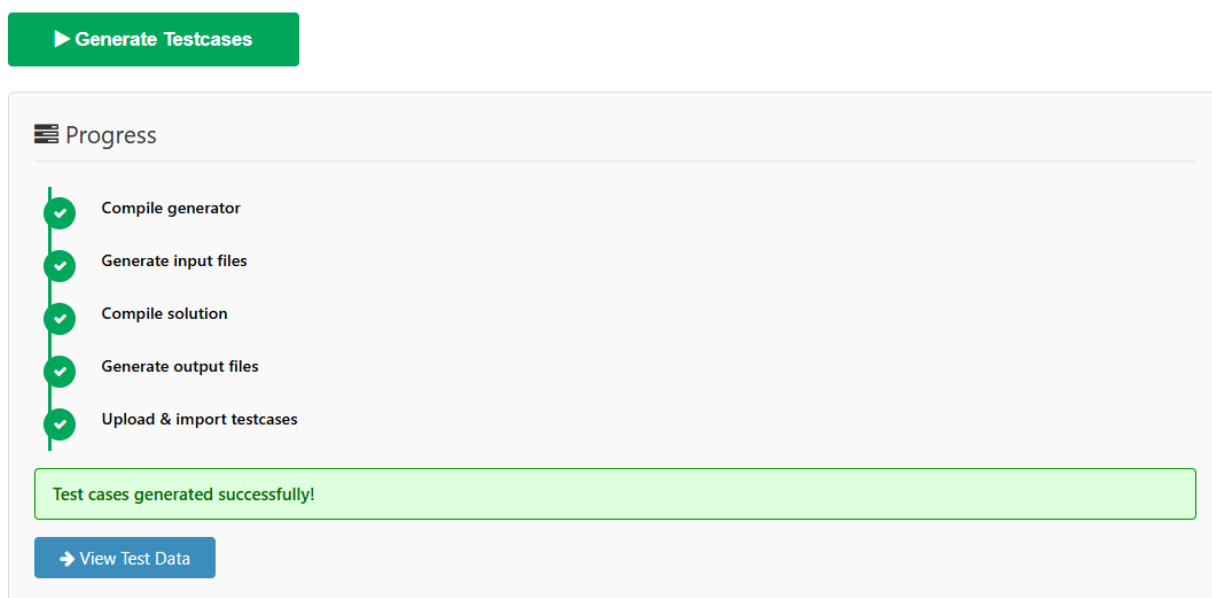
Hình 4.4.3: Giao diện AI hỗ trợ tạo đề bài

Chức năng AI hỗ trợ tạo đề bài được xây dựng để hỗ trợ người ra đề chuẩn hóa nội dung bài toán từ ảnh hoặc văn bản đầu vào. Sau khi AI xử lý, hệ thống có thể trả về các phần nội dung như tên bài, mô tả bài toán, dữ liệu vào, dữ liệu ra và ví dụ minh họa. Người dùng vẫn cần kiểm tra lại nội dung trước khi lưu chính thức vào hệ thống.



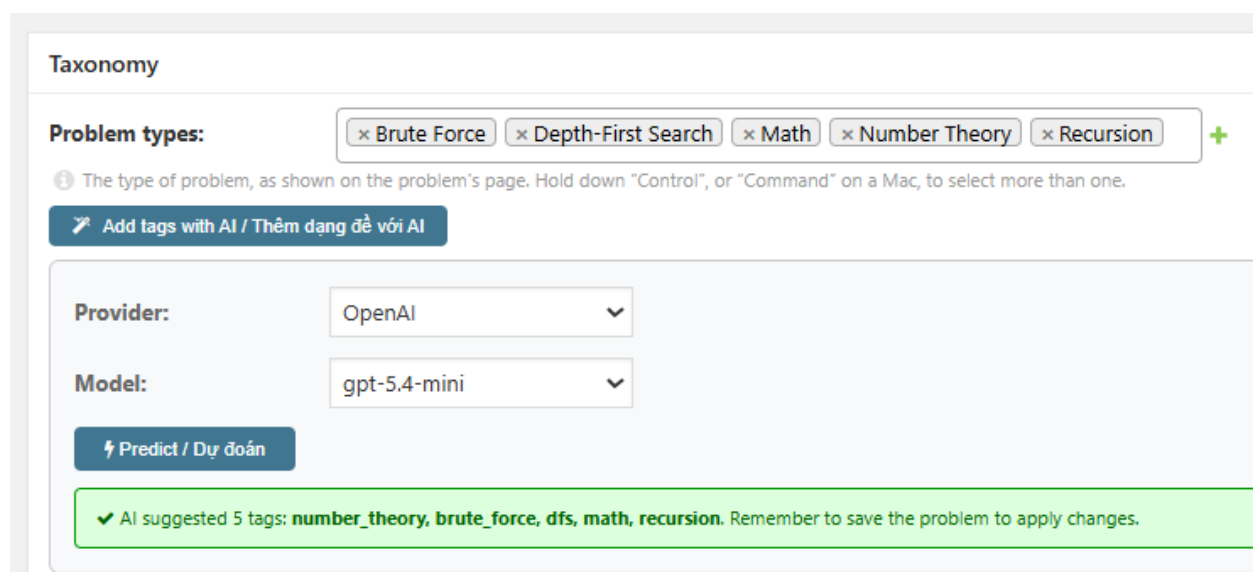
Hình 4.4.4: Giao diện AI sinh test case

Chức năng AI sinh test case hỗ trợ người ra đề trong quá trình xây dựng dữ liệu kiểm thử. Thay vì sinh trực tiếp các file input/output lớn, AI được dùng để tạo ra chương trình sinh input viết bằng C++. Người ra đề cũng cần cung cấp thêm chương trình lời giải chuẩn bằng bất kỳ ngôn ngữ nào. Sau khi có đủ hai chương trình này, hệ thống gửi tác vụ đến máy chấm: biên dịch và chạy chương trình sinh để tạo từng file input (mỗi lần chạy nhận chỉ số test case làm seed ngẫu nhiên), sau đó chạy lời giải chuẩn với từng input đó để tạo output tương ứng. Kết quả là một bộ test case hoàn chỉnh được lưu trực tiếp vào bài toán trên hệ thống.

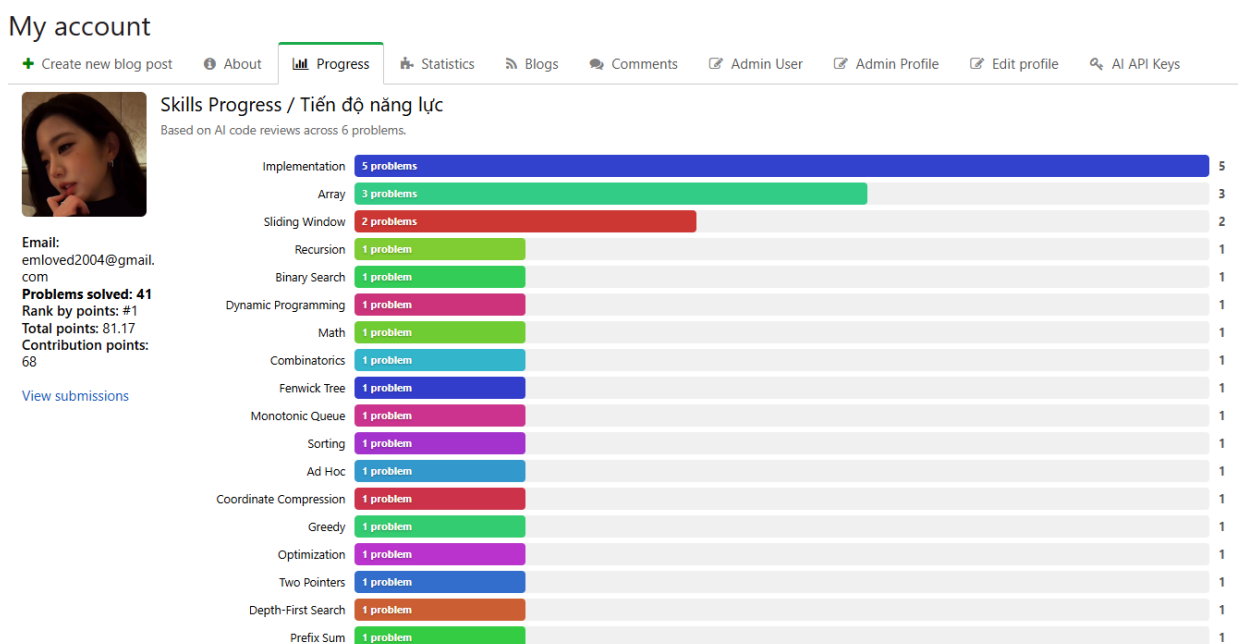


Hình 4.4.5: Quá trình xử lý AI sinh test case

Sau khi tác vụ sinh test case hoàn tất, bộ dữ liệu kiểm thử được lưu vào bài toán và sẵn sàng dùng cho chấm bài. Nếu bài toán đã có test case từ trước, hệ thống yêu cầu người ra đề xác nhận ghi đè trước khi tiến hành. Người ra đề nên kiểm tra lại kết quả để đảm bảo dữ liệu đúng định dạng, đúng ràng buộc và phù hợp với yêu cầu bài toán trước khi sử dụng chính thức.



Hình 4.4.6: Giao diện AI dự đoán dạng đề



Hình 4.4.7: Tiến độ năng lực người dùng

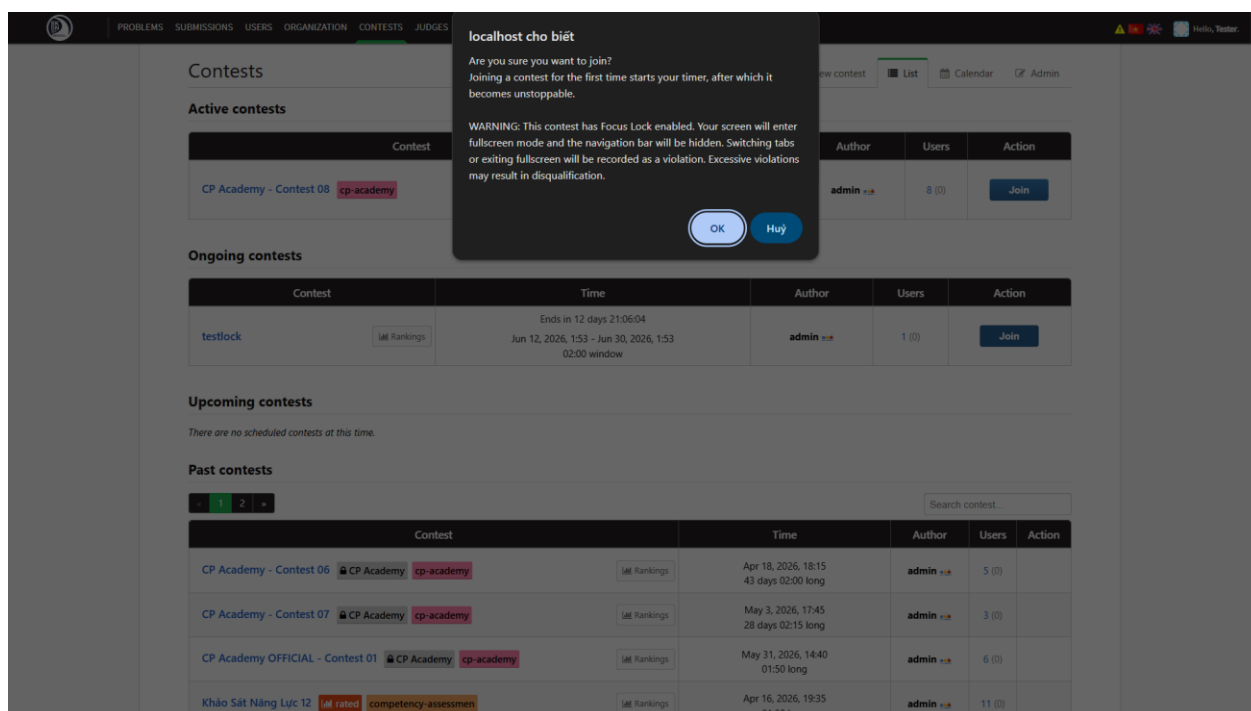
BKDNOJ sử dụng AI cho hai mục đích liên quan đến phân loại dạng bài, phục vụ hai đối tượng khác nhau.

Đối với người ra đề, hệ thống cung cấp chức năng dự đoán dạng đề cho một bài toán cụ thể. Hệ thống lấy ba bài nộp AC có thời gian chạy ngắn nhất của bài toán đó, kết hợp với nội dung đề bài, rồi gửi cho AI phân tích. AI trả về danh sách tên tag dạng mảng JSON; hệ thống đối chiếu với danh sách dạng bài có sẵn trong cơ sở dữ liệu và đề xuất các tag phù hợp để người ra đề xem xét áp dụng. Nếu bài toán chưa đủ ba bài nộp AC, chức năng từ chối và thông báo thiếu dữ liệu.

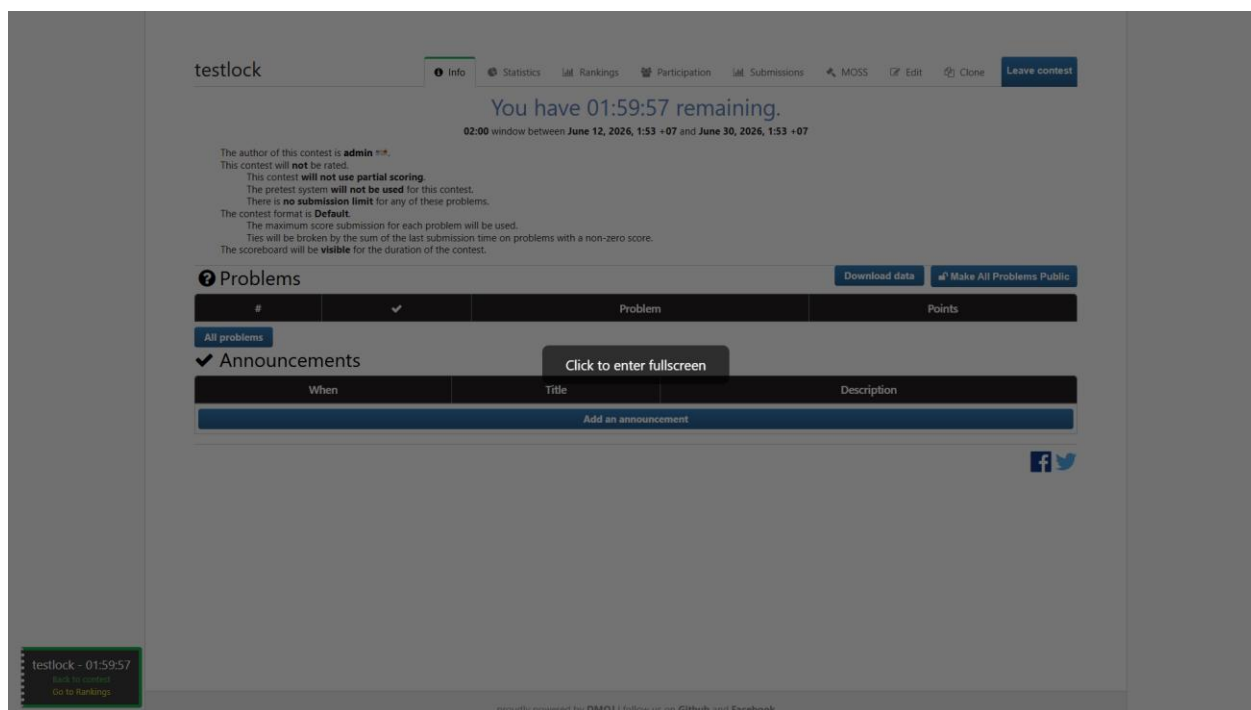
Đối với người dùng thông thường, hệ thống hiển thị trang tiến độ kỹ năng dựa trên dữ liệu được tích lũy từ quá trình sử dụng chức năng phân tích mã nguồn bằng AI. Mỗi lần AI phân tích một bài nộp, phản hồi kèm theo danh sách tag dạng bài và hệ thống lưu các tag này vào bảng UserProblemTag gắn với bài nộp đó. Trang tiến độ tổng hợp dữ liệu này thành biểu đồ thanh ngang, thể hiện số lượng bài toán đã giải theo từng nhóm thuật toán. Đây là kết quả phụ tự động cập nhật mỗi khi người dùng dùng AI để phân tích bài nộp, không phải một lần gọi AI riêng biệt.

4.5. Xây dựng cơ chế khóa tập trung trong kỳ thi

Cơ chế khóa tập trung được xây dựng nhằm hỗ trợ hạn chế gian lận trong các kỳ thi lập trình trực tuyến. Khi một kỳ thi bật tùy chọn khóa tập trung, thí sinh tham gia chính thức sẽ nhận được cảnh báo ngay tại bước xác nhận tham gia: màn hình sẽ vào chế độ toàn màn hình, thanh điều hướng sẽ bị ẩn, và các hành vi thoát toàn màn hình hoặc chuyển tab sẽ bị ghi nhận là vi phạm. [14], [15]



Hình 4.5.1: Giao diện bắt đầu tham gia kỳ thi với khóa tập trung



Hình 4.5.2: Giao diện trước khi vào khóa tập trung

Sau khi xác nhận tham gia, thí sinh được chuyển đến một trang wrapper riêng biệt thay vì vào thẳng giao diện thi. Trang này hiển thị một lớp phủ tối yêu cầu thí sinh nhấp để kích hoạt chế độ toàn màn hình. Toàn bộ giao diện thi, bao gồm xem đề, viết mã, chạy thử và nộp bài được tải bên trong một iframe chiếm toàn bộ màn hình nằm phía sau lớp phủ đó. Thanh điều hướng của hệ thống bị ẩn hoàn toàn trong suốt thời gian thi. Middleware phía server kiểm tra trạng thái khóa tập trung trên mỗi request: nếu thí sinh cố truy cập URL ngoài trang wrapper, request sẽ bị chặn hoặc chuyển hướng về đúng trang.

Trong quá trình làm bài, luồng chấm bài hoạt động bình thường. Để hạn chế khả năng thoát môi trường kiểm soát, hệ thống chặn các phím Ctrl+R, F5 (tải lại trang) và F11 (bật/tắt toàn màn hình). Trên Chrome và Edge, Keyboard Lock API được sử dụng để khóa thêm phím Escape nhằm ngăn thoát toàn màn hình bằng phím tắt.

CP Academy - Contest 08

[Info](#)
[Statistics](#)
[Rankings](#)
[Participation](#)
[Submissions](#)
[MOSS](#)
[Edit](#)
[Clone](#)
[Spectate contest](#)

[Filter](#)
☐ Show full name/organization
 ☐ Show virtual participations
 [Download as CSV](#)

Rank	Username	Points	A 25	B 30	C 25	Violations
1	 c2o1_HuaGiaBao	80 80	25 10	30 21	25 (4) 60	0
2	 Cuong	74 84	25 (8) 31	24 19	25 44	0
3	 C3A1_Minh	49 23	25 (1) 12	24 18		0
4	 C3A1_PGIAKHANGDZ	49 57	25 (2) 47	24 15		0
5	 Yuu	0 0				8
5	 c2o1_ducbao	0 0				0
5	 Tester	0 0				48
5	 nkk_	0 0				0
Total AC			4	1	2	

Hình 4.5.3: Giao diện bảng xếp hạng cho biết số lần vi phạm

Sau khi kỳ thi kết thúc, số lần vi phạm được lưu lại để giảng viên hoặc quản trị viên xem xét. Hệ thống không tự động kết luận gian lận hay loại thí sinh, mà chỉ cung cấp dữ liệu tham khảo cho quá trình đánh giá. Cơ chế này chỉ áp dụng cho thí sinh tham gia chính thức, không áp dụng cho người xem hoặc tham gia ảo.

4.6. Triển khai hệ thống

Hệ thống BKDNOJ được triển khai bằng Docker và Docker Compose nhằm quản lý các dịch vụ cần thiết trong quá trình vận hành. Các thành phần như Django Site, cơ sở dữ liệu, Redis, Celery, Websocket, Nginx, Bridge và máy chấm được cấu hình thành các dịch vụ riêng, giúp quá trình khởi chạy và quản lý hệ thống thuận tiện hơn.

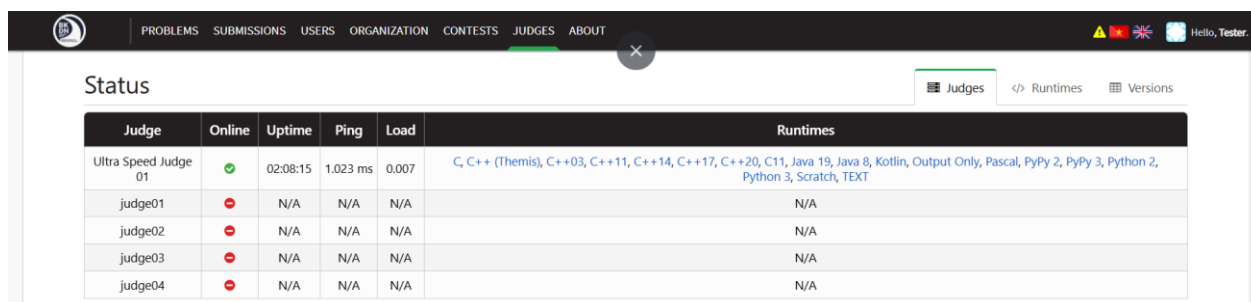
CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
e195aaa1ca69	bkdnoj_nginx	0.00%	12.2 MiB / 7.62 GiB	0.16%	8.92 MB / 11.6 MB	3.72 MB / 160 kB	13
3c374014ad21	bkdnoj_bridged	0.04%	117.8 MiB / 7.62 GiB	1.51%	394 kB / 455 kB	168 kB / 0 B	9

0bc0147018a4	bkdnoj_celery	0.24%	202.5 MiB / 7.62 GiB	2.60%	4.38 MB / 4.56 MB	5.92 MB / 4.1 kB	5
d34c5db7a4b8	bkdnoj_site	0.02%	703 MiB / 7.62 GiB	9.01%	10 MB / 15.1 MB	47 MB / 1.04 MB	11
5ccd719b57d1	bkdnoj_wsevent	0.00%	21.29 MiB / 7.62 GiB	0.27%	153 kB / 115 kB	3.58 MB / 0 B	12
ea953a204c7e	bkdnoj_redis	0.32%	5.961 MiB / 7.62 GiB	0.08%	8.23 MB / 6.16 MB	2.39 MB / 1.41 MB	6
f4337cd72f7f	bkdnoj_mysql	0.02%	127.1 MiB / 7.62 GiB	1.63%	3.1 MB / 7.52 MB	68.6 MB / 8.12 MB	13
0031b594fc4e	ultra_speed_judge_01	0.05%	161.8 MiB / 7.62 GiB	2.07%	0 B / 0 B	1.13 GB / 39.9 MB	8

Bảng 4.6: Danh sách container đang chạy bằng Docker Compose

Máy chấm không nằm trong Docker Compose mà chạy độc lập và kết nối vào bridged qua cổng 9998 và 9999. Cách tách biệt này cho phép triển khai nhiều máy chấm trên các máy chủ khác nhau mà không cần thay đổi cấu hình hệ thống chính. Dữ liệu bài toán và test case được chia sẻ giữa site, celery và bridged qua volume.

Sau khi khởi động thành công, hệ thống có thể truy cập qua trình duyệt. Người dùng có thể đăng nhập, xem và nộp bài, sử dụng IDE trực tuyến, tham gia kỳ thi và dùng các chức năng AI theo phân quyền tương ứng.



Judge	Online	Uptime	Ping	Load	Runtimes
Ultra Speed Judge 01	●	02:08:15	1.023 ms	0.007	C, C++ (Themis), C++03, C++11, C++14, C++17, C++20, C11, Java 19, Java 8, Kotlin, Output Only, Pascal, PyPy 2, PyPy 3, Python 2, Python 3, Scratch, TEXT
judge01	●	N/A	N/A	N/A	N/A
judge02	●	N/A	N/A	N/A	N/A
judge03	●	N/A	N/A	N/A	N/A
judge04	●	N/A	N/A	N/A	N/A

Hình 4.6: Giao diện trạng thái của máy chấm

Để xác nhận hệ thống hoạt động đúng sau triển khai, cần kiểm tra ít nhất hai luồng chính. Luồng chấm bài được kiểm tra bằng cách nộp một bài toán đơn giản và quan sát kết quả trả về trên giao diện, nếu máy chấm kết nối thành công qua bridge và WebSocket hoạt động, trạng thái bài nộp sẽ cập nhật theo thời gian thực

4.7. Tổng kết chương

Nội dung chương đã trình bày quá trình xây dựng và triển khai các chức năng chính của hệ thống BKDNOJ. Các chức năng nền tảng như quản lý bài tập, quản lý test case, nộp bài, chấm bài tự động, quản lý kỳ thi và hiển thị kết quả đã được xây dựng để đáp ứng vai trò cơ bản của một hệ thống chấm bài lập trình trực tuyến.

Bên cạnh các chức năng nền tảng, hệ thống còn được mở rộng với IDE trực tuyến, luồng chạy thử chương trình, các chức năng AI và cơ chế khóa tập trung trong kỳ thi. Những chức năng này giúp nâng cao trải nghiệm làm bài, hỗ trợ người học trong quá trình luyện tập, giảm công sức cho người ra đề và hỗ trợ hạn chế gian lận khi tổ chức kỳ thi trực tuyến.

Quá trình triển khai bằng Docker/Docker Compose giúp hệ thống được tổ chức thành nhiều dịch vụ rõ ràng, thuận tiện cho việc cài đặt, vận hành và kiểm tra. Sau khi triển khai, các chức năng chính cần được kiểm tra thông qua giao diện thực tế, bài nộp thử, kết quả chấm, chức năng chạy thử, chức năng AI và cơ chế khóa tập trung để đảm bảo hệ thống hoạt động ổn định.

CHƯƠNG 5. THỰC NGHIỆM VÀ ĐÁNH GIÁ

5.1. Mục tiêu thực nghiệm

Quá trình thực nghiệm được thực hiện nhằm đánh giá bốn nhóm chức năng chính của hệ thống BKDNOJ. Thứ nhất là luồng nộp bài và chấm bài, bao gồm quá trình người dùng nộp mã nguồn, hệ thống chuyển bài nộp đến máy chấm, thực thi với test case và cập nhật kết quả về giao diện. Thứ hai là IDE trực tuyến và luồng chạy thử, nhằm kiểm tra việc chạy thử chương trình thông qua RunSubmission có hoạt động độc lập với bài nộp chính thức hay không.

Thứ ba là các chức năng AI, bao gồm quản lý API key, phân tích mã nguồn, hỗ trợ tạo đề bài từ ảnh, sinh test case và gợi ý dạng bài. Thứ tư là cơ chế khóa tập trung trong kỳ thi, nhằm kiểm tra việc chuyển hướng thí sinh vào trang kiểm soát, yêu cầu toàn màn hình, theo dõi hành vi rời khỏi giao diện thi và ghi nhận số lần vi phạm.

Các thực nghiệm chủ yếu tập trung vào kiểm thử chức năng, đồng thời có bổ sung kiểm thử tải đọc API để đánh giá khả năng phản hồi của hệ thống khi số lượng người dùng đồng thời tăng lên.

5.2. Thiết lập môi trường thực nghiệm

Hệ thống BKDNOJ được triển khai bằng Docker Compose với các dịch vụ chính như cơ sở dữ liệu MariaDB, Redis, Django Site, Celery, Bridge, WebSocket và Nginx. Máy chấm chạy độc lập bên ngoài Docker Compose và kết nối với Bridge để nhận bài nộp, biên dịch, thực thi chương trình và trả kết quả chấm.

Dữ liệu kiểm thử bao gồm các bài toán có test case đầy đủ, tài khoản người dùng thường, tài khoản quản trị, một kỳ thi có bật cơ chế khóa tập trung và API key AI đã được xác minh.

5.3. Kịch bản thực nghiệm

Các kịch bản kiểm thử được xây dựng theo từng nhóm chức năng chính của hệ thống.

Nhóm kiểm thử	Nội dung kiểm thử
Nộp bài và chấm bài	Kiểm thử các kết quả AC, WA, CE, TLE/MLE

IDE trực tuyến	Kiểm thử chạy thử chương trình đúng, sai và lỗi biên dịch
Chức năng AI	Kiểm thử API key, phân tích mã nguồn, tạo đề từ ảnh, sinh test case và gợi ý dạng bài
Khóa tập trung	Kiểm thử vào kỳ thi, toàn màn hình, ghi nhận vi phạm, chặn phím tắt và nộp bài trong iframe
Tải đọc API	Kiểm thử khả năng phản hồi của trang chủ, danh sách bài tập và danh sách kỳ thi với nhiều người dùng đồng thời

Bảng 5.3: Nhóm và nội dung kiểm thử

Đối với luồng nộp bài và chấm bài, hệ thống được kiểm thử bằng các bài nộp có kết quả khác nhau như Accepted, Wrong Answer, Compilation Error, Time Limit Exceeded và Memory Limit Exceeded. Đối với IDE trực tuyến, hệ thống kiểm tra việc chạy thử thông qua RunSubmission và xác nhận dữ liệu chạy thử không xuất hiện trong danh sách bài nộp chính thức.

Đối với các chức năng AI, hệ thống kiểm tra quá trình thêm và xác minh API key, gửi mã nguồn để AI phân tích, tạo nội dung đề bài từ ảnh, sinh chương trình tạo test case và gợi ý dạng bài dựa trên các bài nộp đúng. Đối với cơ chế khóa tập trung, hệ thống kiểm tra việc chuyển hướng vào trang wrapper, yêu cầu toàn màn hình, ghi nhận vi phạm khi thoát toàn màn hình hoặc chuyển tab, đồng thời đảm bảo thí sinh vẫn có thể xem đề, chạy thử và nộp bài trong giao diện thi.

Ngoài ra, kiểm thử tải đọc API được thực hiện bằng công cụ K6 với bốn mức tải gồm 50, 100, 200 và 300 người dùng ảo. Mỗi người dùng ảo thực hiện tuần tự các request đến trang chủ, danh sách bài tập và danh sách kỳ thi. [16]

5.4. Kết quả thực nghiệm

Kết quả kiểm thử chức năng cho thấy toàn bộ các kịch bản chính đều đạt yêu cầu. Hệ thống xử lý đúng luồng nộp bài và chấm bài, trả về đúng các trạng thái như AC, WA, CE, TLE và MLE. IDE trực tuyến hoạt động đúng, kết quả chạy thử được trả về giao diện và không tạo bài nộp chính thức.

Nhóm chức năng	Số kịch bản	Đạt	Không đạt
Luồng nộp bài và chấm bài	4	4	0
IDE trực tuyến	2	2	0
Các chức năng AI	5	5	0
Cơ chế khóa tập trung	4	4	0
Tổng cộng	15	15	0

Bảng 5.4.1: Kết quả thực nghiệm với các chức năng chính

Đối với các chức năng AI, hệ thống thêm, kiểm tra và xóa API key đúng yêu cầu. Chức năng AI phân tích mã nguồn trả về nhận xét có cấu trúc và có thể trích xuất tag dạng bài. Chức năng tạo đề từ ảnh trả về nội dung Markdown theo cấu trúc đề bài. Chức năng sinh test case tạo được chương trình generator và thực hiện được luồng tạo bộ test case. Chức năng gợi ý dạng bài hoạt động khi bài toán có đủ dữ liệu bài nộp đúng.

Đối với cơ chế khóa tập trung, hệ thống chuyển hướng thí sinh vào trang wrapper, yêu cầu toàn màn hình, tải giao diện thi trong iframe, ẩn thanh điều hướng và ghi nhận số lần vi phạm khi thí sinh thoát toàn màn hình hoặc chuyển tab. Các chức năng xem đề, chạy thử và nộp bài vẫn hoạt động bình thường trong giao diện thi.

Kết quả kiểm thử tải đọc API được tổng hợp như sau:

Chỉ số	50 người dùng ảo	100 người dùng ảo	200 người dùng ảo	300 người dùng ảo
Request/s	108,1	104,4	90,1	77,9
Error rate	0,00%	0,00%	0,19%	1,63%
P50 tổng thể	101 ms	588 ms	1.040 ms	1.110 ms
P95 tổng thể	246 ms	735 ms	3.800 ms	6.760 ms
Max	1.180 ms	1.410 ms	61.000 ms	61.000 ms

Bảng 5.4.2: Bảng kiểm thử tải đọc API

5.5. Đánh giá hệ thống

Qua quá trình thực nghiệm, hệ thống BKDNOJ đã đáp ứng được các chức năng chính đặt ra. Luồng chấm bài hoạt động ổn định, xử lý đúng các trạng thái bài nộp khác nhau và cập nhật kết quả theo thời gian thực. IDE trực tuyến và chạy thử chương trình hoạt động độc

lập với bài nộp chính thức, giúp người dùng kiểm tra mã nguồn mà không ảnh hưởng đến điểm số hoặc bảng xếp hạng.

Các chức năng AI đã được tích hợp và kiểm thử thành công, bao gồm phân tích mã nguồn, tạo đề từ ảnh, sinh test case và gợi ý dạng bài. Việc quản lý API key theo từng người dùng giúp hệ thống linh hoạt khi kết nối với các dịch vụ AI bên ngoài. Cơ chế khóa tập trung cũng hoạt động đúng theo thiết kế, hỗ trợ ghi nhận hành vi rời khỏi giao diện thi và cung cấp dữ liệu tham khảo cho giảng viên hoặc quản trị viên.

Tuy nhiên, hệ thống vẫn còn một số hạn chế. Khả năng chịu tải của cấu hình hiện tại phù hợp hơn với quy mô vừa và nhỏ; từ 200 người dùng ảo trở lên, độ trễ bắt đầu tăng rõ rệt. Các chức năng AI phụ thuộc vào chất lượng phản hồi và thời gian xử lý của dịch vụ bên ngoài, do đó người dùng vẫn cần kiểm tra lại kết quả trước khi sử dụng chính thức. Cơ chế khóa tập trung hoạt động ở tầng trình duyệt nên không thể chống gian lận tuyệt đối. Ngoài ra, máy chấm hiện vẫn chạy độc lập bên ngoài Docker Compose, việc theo dõi và phục hồi khi máy chấm gặp sự cố còn cần cải thiện.

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

1. Kết quả đạt được

Đề án đã thực hiện nghiên cứu, xây dựng và phát triển hệ thống BKDNOJ nhằm hỗ trợ học tập, luyện tập và tổ chức kỳ thi lập trình trực tuyến. Hệ thống được xây dựng theo mô hình Online Judge, cho phép người dùng xem bài tập, viết mã, nộp bài, chấm bài tự động và theo dõi kết quả chấm. Bên cạnh các chức năng nền tảng, hệ thống còn được mở rộng thêm IDE trực tuyến, luồng chạy thử chương trình, các chức năng AI và cơ chế khóa tập trung trong kỳ thi.

Về chức năng chấm bài, hệ thống đã hỗ trợ luồng nộp bài chính thức từ khi người dùng gửi mã nguồn đến khi máy chấm biên dịch, thực thi với test case và trả kết quả về giao diện. Các trạng thái và kết quả chấm được cập nhật cho người dùng, giúp quá trình luyện tập diễn ra nhanh chóng và khách quan. Hệ thống cũng hỗ trợ các chế độ chấm điểm khác nhau, phù hợp với nhiều dạng bài toán và hình thức tổ chức kỳ thi.

Về chức năng hỗ trợ làm bài, IDE trực tuyến và luồng chạy thử chương trình đã được xây dựng tách biệt với luồng nộp bài chính thức. Người dùng có thể viết mã và chạy thử chương trình ngay trên giao diện hệ thống mà không ảnh hưởng đến điểm số, bảng xếp hạng hoặc kết quả chấm chính thức. Chức năng này đặc biệt hữu ích trong các kỳ thi có bật cơ chế khóa tập trung.

Về tích hợp AI, hệ thống đã xây dựng các chức năng như quản lý API key AI, phân tích mã nguồn, hỗ trợ tạo đề bài từ ảnh hoặc văn bản, sinh test case, hỗ trợ đánh giá năng lực người dùng và dự đoán dạng đề. Các chức năng này góp phần hỗ trợ người học hiểu rõ hơn về lời giải của mình, đồng thời giúp người ra đề giảm bớt công sức trong quá trình xây dựng bài toán và dữ liệu kiểm thử.

Về tổ chức kỳ thi, cơ chế khóa tập trung đã được xây dựng nhằm hỗ trợ hạn chế gian lận trong thi lập trình trực tuyến. Cơ chế này giúp yêu cầu thí sinh làm bài trong giao diện toàn màn hình, theo dõi một số hành vi rời khỏi giao diện thi và ghi nhận số lần vi phạm để giảng viên hoặc quản trị viên xem xét sau kỳ thi.

Kết quả thực nghiệm cho thấy các chức năng chính của hệ thống hoạt động đúng theo thiết kế. Hệ thống đáp ứng được mục tiêu đề ra là xây dựng một nền tảng chấm bài trực tuyến

có khả năng hỗ trợ luyện tập lập trình, tổ chức kỳ thi, chạy thử chương trình và tích hợp AI để hỗ trợ quá trình học tập, ra đề và đánh giá năng lực người dùng.

2. Hạn chế

Mặc dù hệ thống đã hoàn thành các chức năng chính, vẫn còn một số hạn chế cần tiếp tục cải thiện. Thứ nhất, khả năng chịu tải của hệ thống còn phụ thuộc vào cấu hình triển khai hiện tại, số lượng worker xử lý và tài nguyên máy chủ. Khi số lượng người dùng đồng thời tăng cao, hệ thống có thể cần được tối ưu thêm về truy vấn, cache và phân phối tải.

Thứ hai, các chức năng AI vẫn phụ thuộc vào chất lượng phản hồi của mô hình AI và dịch vụ bên ngoài. Kết quả phân tích mã nguồn, tạo đề bài, sinh test case hoặc gợi ý dạng đề chỉ nên được xem là thông tin hỗ trợ, người dùng vẫn cần kiểm tra lại trước khi sử dụng chính thức.

Thứ ba, cơ chế khóa tập trung được triển khai trên nền web nên không thể đảm bảo chống gian lận tuyệt đối. Hệ thống chỉ có thể ghi nhận một số hành vi như thoát toàn màn hình, chuyển tab hoặc rời khỏi cửa sổ thi, trong khi các hành vi bên ngoài trình duyệt vẫn cần được kiểm soát bằng quy chế và hình thức giám sát phù hợp.

3. Hướng phát triển

Trong thời gian tới, hệ thống có thể được tiếp tục phát triển theo nhiều hướng. Trước hết, cần tối ưu hiệu năng hệ thống, cải thiện khả năng chịu tải, bổ sung cache cho các trang có lượng truy cập lớn và nghiên cứu triển khai nhiều máy chấm hoặc nhiều instance ứng dụng để phục vụ số lượng người dùng lớn hơn.

Bên cạnh đó, các chức năng AI có thể được hoàn thiện hơn bằng cách chuẩn hóa prompt, bổ sung cơ chế kiểm tra kết quả AI, cải thiện chất lượng sinh test case và mở rộng khả năng đánh giá năng lực người dùng theo từng nhóm kiến thức. Hệ thống cũng có thể phát triển thêm chức năng gợi ý bài tập phù hợp dựa trên lịch sử làm bài và điểm mạnh, điểm yếu của từng người học.

Đối với cơ chế khóa tập trung, hệ thống có thể bổ sung thêm các phương thức giám sát hỗ trợ như ghi log chi tiết hơn, thống kê hành vi trong kỳ thi và cung cấp giao diện phân tích vi phạm cho giảng viên. Ngoài ra, có thể nghiên cứu kết hợp với các giải pháp giám sát khác để tăng tính tin cậy khi tổ chức kỳ thi trực tuyến.

Cuối cùng, hệ thống có thể tiếp tục được hoàn thiện về giao diện, trải nghiệm người dùng, quản trị hệ thống và quy trình triển khai. Những cải tiến này sẽ giúp BKDNOJ trở thành một nền tảng chấm bài trực tuyến ổn định, dễ sử dụng và phù hợp hơn với nhu cầu học tập, luyện tập và tổ chức kỳ thi lập trình trong thực tế.

TÀI LIỆU THAM KHẢO

- [1] S. Wasik, M. Antczak, J. Badura, A. Laskowski và T. Sternal, “A Survey on Online Judge Systems and Their Applications,” arXiv:1710.05913, 2017. [Trực tuyến]. Có tại: <https://arxiv.org/abs/1710.05913>. Truy cập: 29/06/2026.
- [2] DMOJ, “DMOJ: Modern Online Judge,” GitHub repository. [Trực tuyến]. Có tại: <https://github.com/DMOJ/online-judge>. Truy cập: 29/06/2026.
- [3] DMOJ, “DMOJ Judge Server,” GitHub repository. [Trực tuyến]. Có tại: <https://github.com/DMOJ/judge-server>. Truy cập: 29/06/2026.
- [4] Django Software Foundation, “Django documentation.” [Trực tuyến]. Có tại: <https://docs.djangoproject.com/>. Truy cập: 29/06/2026.
- [5] I. Fette và A. Melnikov, “The WebSocket Protocol,” RFC 6455, IETF, 2011. [Trực tuyến]. Có tại: <https://datatracker.ietf.org/doc/html/rfc6455>. Truy cập: 29/06/2026.
- [6] Redis, “Redis documentation.” [Trực tuyến]. Có tại: <https://redis.io/docs/latest/>. Truy cập: 29/06/2026.
- [7] Celery Project, “Celery documentation.” [Trực tuyến]. Có tại: <https://docs.celeryq.dev/>. Truy cập: 29/06/2026.
- [8] Docker Inc., “Docker documentation.” [Trực tuyến]. Có tại: <https://docs.docker.com/>. Truy cập: 29/06/2026.
- [9] Docker Inc., “Docker Compose documentation.” [Trực tuyến]. Có tại: <https://docs.docker.com/compose/>. Truy cập: 29/06/2026.
- [10] NGINX, “NGINX Reverse Proxy.” [Trực tuyến]. Có tại: <https://docs.nginx.com/nginx/admin-guide/web-server/reverse-proxy/>. Truy cập: 29/06/2026.
- [11] OWASP Cheat Sheet Series, “Docker Security Cheat Sheet.” [Trực tuyến]. Có tại: https://cheatsheetseries.owasp.org/cheatsheets/Docker_Security_Cheat_Sheet.html. Truy cập: 29/06/2026.
- [12] OpenAI, “OpenAI API Platform Documentation.” [Trực tuyến]. Có tại: <https://developers.openai.com/api/docs>. Truy cập: 29/06/2026.

[13] OpenAI, “Prompt engineering.” [Trực tuyến]. Có tại: <https://developers.openai.com/api/docs/guides/prompt-engineering>. Truy cập: 29/06/2026.

[14] MDN Web Docs, “Fullscreen API.” [Trực tuyến]. Có tại: https://developer.mozilla.org/en-US/docs/Web/API/Fullscreen_API. Truy cập: 29/06/2026.

[15] MDN Web Docs, “Page Visibility API.” [Trực tuyến]. Có tại: https://developer.mozilla.org/en-US/docs/Web/API/Page_Visibility_API. Truy cập: 29/06/2026.

[16] Grafana Labs, “Grafana k6 documentation.” [Trực tuyến]. Có tại: <https://grafana.com/docs/k6/latest/>. Truy cập: 29/06/2026.